POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Automation and Control in Autonomous Vehicles

Appunti di Lezione

Author: **Giorgia Savo,**

Professor: **Matteo Corno**

Academic Year: 2025-26

Contents

Contents **i**

1 Actuators **1**
1.1 Cosa sono gli attuatori e perché contano . . . 1
1.2 I principali attuatori . . . 2
1.2.1 Le sfide principali nella progettazione degli attuatori . . . 2
1.3 Brake Actuation . . . 3
1.3.1 L'idea di base . . . 3
1.3.2 Wet solution: Brake-by-Wire idraulico . . . 4
1.3.3 Descrizione del sistema completo . . . 6
1.3.4 Identificazione del modello . . . 6
1.3.4.1 Approccio nested (a cascata) . . . 8
1.3.4.2 Progettazione del segnale di identificazione . . . 9
1.3.4.3 Design del controllore . . . 10
1.3.4.4 Validazione . . . 10
1.3.5 Non linearità: relazione posizione-pressione . . . 11
1.3.6 Dry solution . . . 12
1.4 Steer Actuation . . . 13
1.4.1 EPS — Electric Power Steering . . . 13
1.4.2 SBW — Steer-by-Wire . . . 14
1.4.3 Retrofitting . . . 15
1.4.3.1 Verifica di linearità e saturazione . . . 15
1.4.3.2 Elasticità della trasmissione . . . 16
1.5 Traction Actuation . . . 17
1.5.1 Non linearità: elasticità della trasmissione . . . 18

2 Sensors **19**
2.1 Introduzione: proprioceptive vs exteroceptive . . . 19

2.2 Time of Flight Sensors . . . 19
2.2.1 Principio di funzionamento . . . 19
2.2.2 Ultrasound . . . 20
2.2.3 Collimated Light: il LiDAR . . . 21
2.2.3.1 Classificazione dei LiDAR . . . 21
2.2.3.2 Coordinate del LiDAR . . . 22
2.2.3.3 Sorgenti di rumore e il fenomeno del jitter . . . 23
2.2.4 Radar . . . 24
2.2.4.1 Classificazione: SISO, SIMO, MIMO . . . 24
2.2.4.2 Il ciclo di scansione (scan cycle) . . . 25
2.2.4.3 Effetto Doppler e caratteristiche distintive del radar . . . 25
2.2.5 Scattered Light . . . 25
2.3 Localization Sensors: il GNSS . . . 26
2.3.1 Cos'è il GPS/GNSS . . . 26
2.3.2 Come funziona il posizionamento . . . 26
2.3.3 Il problema della sincronizzazione . . . 27
2.3.3.1 Soluzione code-based . . . 27
2.3.3.2 Soluzione carrier-based . . . 28
2.3.4 Ricostruzione della posizione . . . 28
2.3.5 Sorgenti di errore * . . . 29
2.3.6 Tecniche per ridurre gli errori . . . 29
2.3.6.1 Approcci satellite-based . . . 29
2.3.6.2 Differential GNSS (DGNSS) . . . 30
2.3.6.3 Real-Time Kinematic (RTK) . . . 30
2.3.6.4 Satellite Based Augmentation Systems (SBAS) e PPP . . . 30
2.3.7 Confronto tra le tecniche di correzione . . . 31
2.3.8 Pro e contro del GNSS . . . 31
2.4 Vision and Cameras . . . 32
2.4.1 Perché le telecamere . . . 32
2.4.2 Elementi principali . . . 32
2.4.3 Classificazione delle telecamere . . . 32
2.4.4 Camera Projective Geometry . . . 33
2.4.4.1 Il concetto di posa (pose) . . . 33
2.4.4.2 Matrici di rotazione . . . 33
2.4.4.3 Coordinate omogenee e trasformazione rigida . . . 34
2.4.4.4 Attitude angles (angoli di Cardano) . . . 35
2.4.4.5 Dal rate angolare misurato alla derivata degli angoli . . . 36

2.4.5 Vision — l'image plane . . . 37
2.4.6 Il modello pinhole * . . . 37
2.4.6.1 Estrinseci e intrinseci . . . 38
2.4.6.2 Primo step: da coordinate mondo a coordinate camera . . 39
2.4.6.3 Secondo step: da coordinate camera a coordinate immagine 39
2.4.6.4 Il trucco delle coordinate omogenee . . . 40
2.4.7 Telecamere reali: la distorsione . . . 41
2.4.7.1 Modello di distorsione radiale . . . 41
2.4.7.2 Calibrazione tramite scacchiere . . . 41
2.4.8 Il problema inverso . . . 42
2.5 Stereo Vision * . . . 42
2.5.1 L'idea di base . . . 42
2.5.2 Assunzioni tipiche delle stereo camere . . . 43
2.5.3 Derivazione geometrica: triangoli simili * . . . 43
2.5.4 Definizione di disparità . . . 44
2.5.5 Considerazioni pratiche su disparità, risoluzione e baseline . . . 44
2.5.5.1 Baseline e distanza minima . . . 44
2.5.5.2 Risoluzione e risoluzione spaziale . . . 45
2.5.5.3 Baseline e range massimo . . . 45
2.5.5.4 Effetto della lunghezza focale . . . 46
2.5.6 Il problema della corrispondenza (correspondence problem)* . . . 46
2.5.6.1 Correlation based algorithms . . . 46
2.5.7 Il vincolo epipolare . . . 48
2.5.7.1 Perché esiste il vincolo epipolare . . . 49
2.5.7.2 Rettificazione: far coincidere l'epipolar constraint con lo scanline . . . 49
2.5.8 Procedura completa per l'uso di immagini stereo . . . 50
2.5.9 Sfide e limitazioni della stereo vision . . . 51
2.6 Model Based Image Processing . . . 52
2.6.1 Lane detection . . . 53
2.6.1.1 Pipeline della lane detection . . . 53
2.6.1.2 Pipeline dettagliata della segmentazione . . . 55
2.6.2 RANSAC (Random Sample Consensus) . . . 57
2.6.2.1 Caratteristiche del RANSAC . . . 57
2.6.2.2 Funzionamento . . . 57
2.6.2.3 Quante iterazioni servono . . . 58
2.7 Neural Network Image Processing . . . 59

2.7.1 Model based vs neural network . . . 59
2.7.2 Supervised learning . . . 59
2.7.3 Training vs inference . . . 60
2.7.4 Altre tecnologie di visione . . . 60

3 Localization **61**
3.1 Il quadro generale: perché la localizzazione è un problema a sé . . . 61
3.1.1 Pose e Stato: due concetti diversi . . . 61
3.2 SLAM: posizionamento assoluto . . . 62
3.2.1 Front-end: scan-to-scan matching . . . 63
3.2.2 Back-end: pose graph e loop closure . . . 64
3.2.3 SLAM vs. Autonomous Driving: uso asincrono . . . 65
3.3 State Estimation * . . . 66
3.3.1 Idea generale: closed-loop estimation . . . 66
3.3.2 Modelli del veicolo * . . . 67
3.3.2.1 Rigid Body Kinematic Model * . . . 67
3.3.2.2 Vehicle Bicycle Model . . . 68
3.3.2.3 Modelli di attrito pneumatico . . . 70
3.4 Stima combinata cinematica-dinamica dello stato (algoritmo di state estimation) . . . 72
3.4.1 Il problema: nessuno dei due modelli, da solo, basta . . . 72
3.4.2 L'osservatore ibrido cinematico-dinamico . . . 72
3.4.2.1 Stima diretta di V_x: min-max sulle velocità ruota . . . 73
3.4.2.2 Stima di A_y tramite modello dinamico semplificato . . . 73
3.4.3 Identificazione dei coefficienti C_r, K_x . . . 74
3.4.4 Quando fidarsi di quale modello: scheduling di K_y . . . 74
3.4.5 Dalla velocità alla pose completa: fusione con GNSS . . . 75
3.4.6 Sensor fusion generale: il filtro EKF . . . 76
3.5 Localizzazione relativa (mapless)**(warehouse)** . . . 76
3.5.1 Perché "senza mappa" . . . 76
3.5.2 Setup sensoristico . . . 77
3.5.3 Impostazione del problema . . . 77
3.5.4 Modello cinematico per il row-following . . . 78
3.5.5 Modello di misura degli ultrasonici . . . 79
3.5.6 Modello di predizione a tempo discreto . . . 80
3.5.7 EKF: perché "extended" . . . 80
3.5.8 Il problema pratico: outlier e rumore non gaussiano . . . 81

3.5.9 Measurement selection: finestra centrata sulla stima . . . 82
3.5.10 Window scheduling automatico tramite covarianza del filtro . . . 82
3.5.11 Osservazioni conclusive . . . 83

4 Target Tracking 85
4.1 Il problema: perché non basta la detection . . . 85
4.2 Due filosofie di approccio . . . 85
4.2.1 Tracking by detection . . . 86
4.2.2 End-to-end tracking . . . 86
4.2.3 Confronto . . . 86
4.3 Dalla point cloud 3D alla griglia 2D . . . 86
4.4 Detection . . . 87
4.4.1 Ground Removal . . . 87
4.4.1.1 RANSAC . . . 87
4.4.1.2 Il caso critico: pareti verticali . . . 88
4.4.2 Scan Registration . . . 88
4.4.2.1 Il problema . . . 88
4.4.2.2 ICP — Iterative Closest Point . . . 89
4.4.3 Grid Quantization e Binarization . . . 91
4.4.3.1 Da point cloud a griglia . . . 91
4.4.3.2 Binarizzazione . . . 91
4.4.4 Background Removal . . . 92
4.4.5 Denoising . . . 92
4.4.5.1 Image processing (morfologia matematica) . . . 92
4.4.5.2 Shadow management . . . 93
4.5 Tracking* . . . 93
4.5.1 Multi-target Tracking * . . . 93
4.5.1.1 Le due sotto-sfide . . . 93
4.5.1.2 Setup del problema base . . . 94
4.5.1.3 Il ciclo del Kalman Filter . . . 94
4.5.2 Extended Target Tracking * . . . 94
4.6 Architettura a due livelli . . . 95
4.6.1 Low Level Tracking — la Prediction . . . 95
4.6.1.1 Scelta del modello dinamico . . . 96
4.6.1.2 Coordinated Turn Motion Model . . . 96
4.6.1.3 Discretizzazione esatta . . . 97
4.6.1.4 Equazione di output . . . 98

4.6.1.5 Measure Association* . . . 98
4.6.1.6 Track Management . . . 100
4.6.2 Double-Threshold Clustering . . . 100
4.6.3 High Level Tracking . . . 101
4.6.3.1 Drawback residuo. . . . 101
4.6.3.2 Soluzione parziale. . . . 101
4.7 Sensor Fusion con la Camera . . . 101
4.7.1 Perché la camera aiuta . . . 101
4.7.2 Euclidean Clustering . . . 102
4.7.3 DD3D . . . 102
4.7.4 Architettura di fusione . . . 103
4.7.5 Struttura del track finale . . . 103
4.8 Sistema di riferimento: globale vs locale . . . 104
4.8.1 Definizioni . . . 104
4.8.2 Perché con ego fermo non importava . . . 104
4.8.3 Perché con ego in movimento importa . . . 104
4.8.4 Lavorare in locale . . . 105
4.8.5 Frequenza di esecuzione . . . 105

5 Control **107**
5.1 Path e traiettoria . . . 107
5.1.1 Curvatura . . . 107
5.2 Lateral Control . . . 108
5.2.1 L'approccio geometrico: Pure Pursuit . . . 109
5.2.2 L'approccio cinematico . . . 109
5.2.2.1 Il modello cinematico a bicicletta . . . 109
5.2.2.2 Perché questo modello, da solo, non basta . . . 110
5.2.2.3 Cambio di coordinate: dal riferimento globale al riferimento del path . . . 111
5.2.2.4 Derivazione delle equazioni in coordinate del path . . . 111
5.2.2.5 Lateral error feedback . . . 112
5.2.2.6 Lookahead error feedback . . . 113
5.2.2.7 Lookahead feedback vs PD: sono davvero equivalenti? . . . 114
5.2.2.8 Scelta della distanza di lookahead . . . 115
5.2.2.9 Gain scheduling . . . 115
5.2.2.10 Robustezza rispetto al cambiamento di curvatura . . . 116
5.2.3 Il modello dinamico . . . 117

5.2.3.1 Perché non basta più il modello cinematico 117
5.2.3.2 Confronto fra modello cinematico e dinamico 118
5.2.4 Curvature Preview Control**** 118
5.2.4.1 L'idea di base: controllare l'orientamento, non solo l'errore 118
5.2.4.2 Architettura cascata 121
5.3 Longitudinal Control * 123
5.3.1 Collocazione del problema 123
5.3.2 Il modello di dinamica longitudinale 124
5.3.3 Identificazione dei parametri: la prova di coasting down 124
5.3.3.1 Impostazione del problema 124
5.3.3.2 Identificazione tramite regressione lineare 125
5.3.3.3 La forza alla ruota 126
5.3.4 Le mappe motore 126
5.3.4.1 Motore a combustione interna * 126
5.3.4.2 Motore elettrico 127
5.3.5 Progettazione del controllo longitudinale 127
5.3.5.1 Architettura generale: alto livello e basso livello 127
5.3.5.2 Il controllo dell'accelerazione 128
5.3.5.3 Compensazione delle non linearità: le mappe statiche inverse 128
5.3.5.4 Il controllore a cascata 129
5.3.5.5 I vantaggi del controllo a cascata 130
5.3.5.6 Il sistema visto dall'anello esterno 130
5.4 Osservazioni conclusive 131

6 Planning 133
6.1 Il quadro generale: perché "planning" e non solo "routing" 133
6.2 Longitudinal Planner 133
6.2.1 Cosa deve fare 133
6.2.2 L'Optimizer 135
6.2.2.1 Generazione del profilo trapezoidale 135
6.2.2.2 Smoothing (Quadratic Programming) 137
6.2.2.3 Valutazione quantitativa del comfort di guida 138
6.2.3 TFSM — Tactical Finite State Machine 138
6.2.4 ACC — Adaptive Cruise Control* 139
6.3 Lateral Planner * 140
6.3.1 Impostazione del problema: parametrizzazione in Frenet 140
6.3.2 Approccio closed-form: la sigmoide 141

6.3.2.1 Overtake decision making . . . 142
6.3.2.2 Calcolo della regione da evitare . . . 142
6.3.2.3 Funzione di costo . . . 143
6.3.2.4 Vincoli di fattibilità (curvatura massima) . . . 144
6.3.2.5 Preview distance . . . 144
6.3.3 Approccio basato su ottimizzazione: ricerca su grafo * . . . 144
6.3.3.1 Parametrizzazione: waypoint . . . 145
6.3.3.2 Algoritmo . . . 145
6.3.4 Estensione a ostacoli dinamici . . . 148
6.3.4.1 Same direction, same velocity . . . 149
6.3.4.2 Same direction, different velocity . . . 149
6.3.4.3 same direction, opposite velocity . . . 149
6.3.5 Predizione del comportamento altrui . . . 150
6.3.5.1 Scenari semplici . . . 150
6.3.5.2 Scenari complessi . . . 151

7 Global Planning * 153
7.1 Dal global planning "stradale" al global planning "da corsa" . . . 153
7.1.1 Il limite fisico: la friction ellipse . . . 153
7.1.2 Due filosofie per progettare la traiettoria . . . 154
7.1.2.1 Metodo dinamico (Dynamic method) . . . 155
7.1.2.2 Metodo geometrico (Geometric method) . . . 155
7.1.2.3 Hybrid approach . . . 161

1 | Actuators

1.1 Cosa sono gli attuatori e perché contano

Una macchina a guida autonoma deve risolvere esattamente gli stessi compiti che risolve un guidatore umano: percepire l'ambiente, localizzarsi, pianificare un percorso e infine *agire* sul veicolo. L'architettura tipica è quindi:

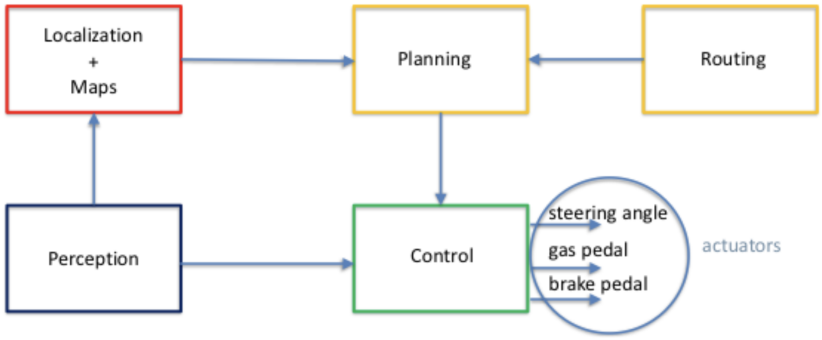


Figure 1.1: Architecture

Gli **attuatori** sono l'ultimo anello di questa catena: sono l'insieme dei dispositivi che trasformano il segnale calcolato da un controllore in un'azione fisica sul veicolo. Quando guidiamo, i tre attuatori più importanti sono l'angolo di sterzo, il pedale del gas e il pedale del freno. Esistono anche altri attuatori — la frizione (clutch) o il cambio marcia (gear), per esempio, che influenzano comunque il comportamento del veicolo, oppure le frecce (turning signals) — ma questi sono importanti pur restando concettualmente triviali rispetto ai tre principali.

L'approccio del corso a questo argomento segue il classico principio del *divide et impera*: prima si scompone il problema (sterzo, freno, motore), poi per ciascun sottosistema si trova un modo per descriverlo che sia utile al resto del sistema — cioè un modello che il controllore possa usare senza dover conoscere tutti i dettagli fisici interni.

1.2 I principali attuatori

I tre attuatori principali sono:

- **Steer** (sterzo) — governa la dinamica laterale del veicolo, ossia la direzione;
- **Brake** (freno) — governa la dinamica longitudinale (decelerazione);
- **Motor** (motore) — governa la dinamica longitudinale (accelerazione).

Un concetto chiave introdotto qui è quello di **safety level**: quanto possiamo accettare un guasto (failure) in quel particolare sistema, prima che le conseguenze diventino inaccettabili.

In base a questo criterio, i tre attuatori non sono equivalenti:

- **Sterzo** → dinamica laterale, **massima criticità di sicurezza**: un guasto qui comporta un altissimo rischio di incidente, perché se qualcosa va storto il veicolo può perdere immediatamente il controllo della direzione.
- **Freno** → dinamica longitudinale, anch'esso a **massima criticità di sicurezza**.
- **Motore** → dinamica longitudinale, ma con criticità di sicurezza **più bassa**: se perdiamo la capacità di accelerare, nella maggior parte dei casi possiamo comunque portare il veicolo in un posto sicuro (ad esempio lasciandolo rallentare fino a fermarsi). Tutti gli altri attuatori hanno quindi un livello di criticità ancora minore rispetto a sterzo e freno.

Un'osservazione importante è che stiamo lavorando in un sistema **ad anello chiuso** (closed loop): non basta descrivere un attuatore in modo statico, bisogna capire come si comporta dinamicamente nel tempo, perché è proprio questo comportamento dinamico che verrà "visto" dal resto della catena di controllo.

1.2.1 Le sfide principali nella progettazione degli attuatori

Quando si progetta un attuatore per la guida autonoma, ci sono quattro sfide fondamentali da affrontare:

1. **Reliability / Safety** — affidabilità e tolleranza ai guasti (fault tolerance). L'affidabilità è direttamente collegata alla sicurezza: un attuatore che si guasta spesso è, per definizione, meno sicuro.
2. **Bandwidth** — quanto velocemente il sistema risponde a un comando. È legata principalmente alle prestazioni, ma influisce chiaramente anche sulla sicurezza: ad

esempio, un tempo di reazione lento a un ostacolo rilevato è un problema di sicurezza, non solo di comfort.

3. **Accuracy** — quanto bene l'attuatore insegue il target che gli viene richiesto (tracking).
4. **Handover L1–L4 tra NCU e guidatore** — la Navigation Control Unit (NCU, l'unità di controllo automatico) deve poter cedere e riprendere il controllo dal guidatore in modo sicuro e coerente. Questo è un aspetto trasversale a tutti gli attuatori: ognuno deve garantire un passaggio di controllo (handover) fluido nei due sensi.

A proposito dell'ultimo punto, vale la pena richiamare i livelli di automazione definiti dalla SAE (Society of Automotive Engineers), che si basano proprio sulla possibilità (o meno) di chiedere aiuto al guidatore:

- **L1** — Driver Assistance;
- **L2** — Partial Automation;
- **L3** — Conditional Automation;
- **L4** — High Automation.

Nei livelli da 1 a 4, il sistema fa ancora affidamento, in una certa misura, sulla possibilità di richiedere l'intervento del guidatore: da qui l'importanza critica di un handover ben progettato.

1.3 Brake Actuation

1.3.1 L'idea di base

L'idea alla base della frenata è semplice: applicare una forza sul disco frenante, che viene trasformata in una coppia (torque); questa coppia rallenta la ruota e l'attrito pneumatico-strada rallenta poi il veicolo.

È importante notare che *non* è possibile applicare direttamente una forza sul telaio del veicolo per frenarlo (a parte i freni aerodinamici, un caso particolare). Il percorso è quindi sempre: coppia applicata sulla ruota → attrito pneumatico-strada che genera la forza frenante effettiva.

Con i motori elettrici è possibile applicare una forza frenante sul veicolo (in fase di frenata rigenerativa) *senza* dissipazione di energia, cosa impossibile con i freni ad attrito classici.

Il dispositivo più usato per applicare la coppia sulla ruota è il **disco freno** (disc brake), che appartiene alla classe dei "friction brakes" ed è costituito da:

- **disco / rotore** — ruota solidale con la ruota del veicolo;
- **pastiglie freno** — generano attrito con il disco, trasformando energia meccanica in calore;
- **pinza (caliper)** — applica la forza che preme le pastiglie contro il disco.

La maggior parte dei freni sono **freni idraulici**: è presente una sorta di pompa che aumenta la pressione di un fluido, il quale a sua volta muove un pistone contro la ruota. Schematicamente, la catena è:

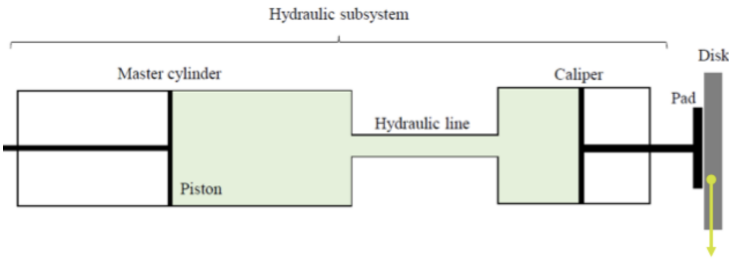
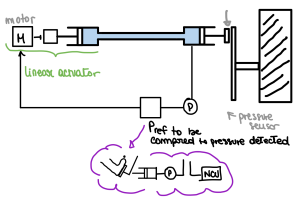


Figure 1.2: Hydraulic brake scheme

Per rendere questo sistema controllabile elettronicamente, mantenendo la stessa idea di fondo, bisogna garantire un **handover** tra controllo automatico e controllo manuale del guidatore.

1.3.2 Wet solution: Brake-by-Wire idraulico



La **wet solution**, indicata anche con la sigla BBW (Brake-by-Wire), consiste semplicemente nel sostituire il pedale e il lato "sinistro" della pompa con una pompa idraulica azionata da un motore elettrico (un attuatore lineare).

Analizziamo questa soluzione secondo i criteri visti sopra:

Reliability.

È una replica del sistema originale: se il lato manuale funzionava bene, continua a funzionare bene, perché sul lato sinistro (pedale) non cambia nulla. Tuttavia, l'aggiunta di un motore comporta un nuovo punto di guasto: se il motore o il suo sistema elettrico si

guastano, non è più possibile usare il freno tramite quella via. Questa soluzione risulta quindi **meno affidabile** di quella puramente meccanica/idraulica originale.

Bandwidth.

Se siamo soddisfatti dell'accuratezza del sistema frenante manuale, si parte già da un buon punto di riferimento: è importante progettare il motore in modo che replichi sufficientemente bene ciò che fa il piede del guidatore.

Handover.

Si può avere il pedale che spinge contro un sistema idraulico "fittizio" (fake), misurare la pressione generata e decidere in base a questa se usare il riferimento applicato dal guidatore oppure quello proveniente dalla Navigation Control Unit (NCU).

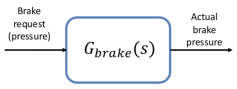
Concretamente, questo si realizza tramite una **valvola**: quando la valvola è chiusa, il primo tratto del sistema (quello collegato al pedale) viene separato dal resto, che lavora invece tramite un motore collegato a una pompa che genera la pressione. Se qualcosa va storto, la valvola si apre e il sistema torna al comportamento manuale di base (baseline).

Figure 1.3: NO (normally open) valve

Un dettaglio progettuale fondamentale: se si progettasse la valvola in modo che sia la NCU a doverla *aprire* in caso di pericolo, si avrebbe un problema, perché se si perde il controllo della NCU la valvola resterebbe incontrollata. Per questo si utilizza una **valvola NO** (*Normally Open*): è normalmente aperta, e la NCU deve attivamente *chiuderla* per poter usare il Brake-by-Wire in modalità automatica. Questo è un classico esempio di progettazione fail-safe: in assenza di segnale, il sistema torna sempre alla condizione sicura di default (controllo manuale).

Dal punto di vista della NCU non serve conoscere i dettagli implementativi del sistema idraulico; servono solo due informazioni:

- il segnale per chiudere la valvola quando si vuole il controllo automatico;
- quanto tempo impiega il segnale a essere effettivamente attuato.



Serve quindi una descrizione della relazione tra pressione desiderata P_{ref} e pressione effettiva $P_{misurata}$: manca cioè un **controllore di pressione**, di cui definiamo obiettivi e prestazioni desiderate:

- **Obiettivo**: inseguimento (tracking) di un riferimento di pressione, cioè controllare la pressione sulle pastiglie, che equivale a controllare la forza frenante.
- **Prestazioni desiderate**: banda passante elevata (ad esempio $> 5\,\text{Hz}$ con ritardo $< 40\,\text{ms}$) e overshoot nullo o piccolo. Gli overshoot sono pericolosi in frenata, perché possono causare il blocco delle ruote (wheel-locking) e l'intervento dell'ABS, con conseguente perdita di stabilità.

1.3.3 Descrizione del sistema completo

Il sistema fisico può essere scomposto in tre sottosistemi in cascata:

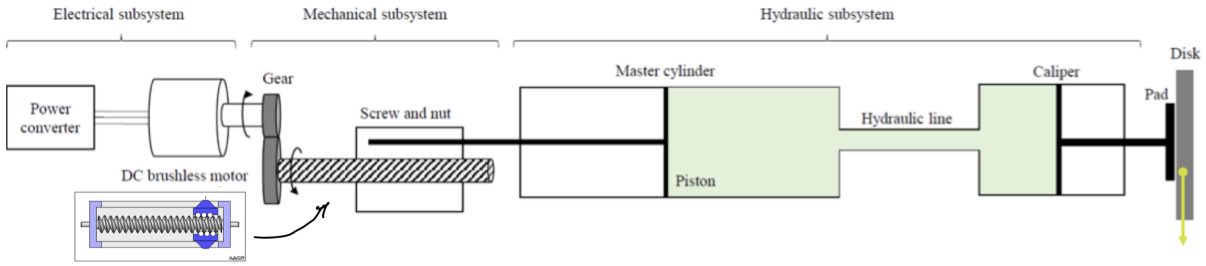


Figure 1.4: Complete System

Concretamente: un motore DC brushless, tramite un convertitore di potenza, aziona un ingranaggio (gear); questo movimento rotatorio viene trasformato in movimento lineare da una vite e madrevite (screw and nut) — è questa la *trasmissione*; il pistone così mosso agisce sul cilindro maestro (master cylinder), che tramite la linea idraulica trasmette la pressione fino alla pinza (caliper) e infine al disco.

Si assume di poter controllare tensione o corrente del motore (input), e di avere sensori per posizione del pistone, pressione e corrente motore (output misurati).

1.3.4 Identificazione del modello

Per progettare il controllore serve prima un **modello del sistema in anello aperto**. L'approccio classico prevede tre fasi:

1. **Identificazione / calibrazione** — si trova un modello del sistema che riproduca i dati sperimentali;

2. **Control tuning** — si usa il modello identificato per progettare (tarare) il controllore;
3. **Validazione** — si verifica che il comportamento in anello chiuso soddisfi le prestazioni desiderate.

Se la validazione fallisce, si torna eventualmente a rivedere l'identificazione: è un processo iterativo.

Il modello può essere derivato in due modi:

- **Approccio fisico (physical based)** — si scrivono le equazioni a partire dalla fisica del sistema, ottenendo direttamente una relazione ingresso-uscita;
- **Approccio black-box** — non servono i valori fisici dei parametri; il modello si ricava sperimentalmente. Questo approccio è possibile solo se il sistema fisico è già stato costruito, quindi non è utilizzabile in fase di puro design.

Nell'identificazione dinamica in dominio della frequenza, la procedura è:

1. si esegue un **esperimento**, registrando ingresso $u(t)$ e uscita $y(t)$;
2. si calcola lo spettro di ingresso e uscita, $U(\omega)$ e $Y(\omega)$;
3. si calcola la stima della risposta in frequenza:

$$G_{EXP}(\omega) = \frac{Y(\omega)}{U(\omega)}, \qquad \text{oppure, in modo meno sensibile al rumore,} \qquad G_{EXP}(\omega) = \frac{S_{yu}(\omega)}{S_{uu}(\omega)}$$

4. si esegue un **frequency matching**, cioè si cerca il modello parametrico $G(s)$ che meglio approssima $G_{EXP}(\omega)$, risolvendo:

$$\min_{a_0,\dots,a_{N_n},\, b_0,\dots,b_{N_d}} \sum_{k=1}^{N} \left| G_{EXP}(\omega(k)) - G(\omega(k)) \right|^2, \qquad G(s) = \frac{a_0 + \cdots + a_{N_n} s^{N_n}}{b_0 + \cdots + b_{N_d} s^{N_d}}$$

L'output del sistema può essere pressione o posizione, l'input tensione o corrente. Un punto concettualmente importante è che **ogni passaggio di questa catena porta con sé un certo livello di incertezza**: la tensione del motore diventa corrente (proporzionale alla coppia), la coppia muove il pistone, il pistone spinge il fluido, il fluido genera pressione — e in ognuno di questi passaggi ci sono parametri fisici non perfettamente noti.

1.3.4.1 Approccio nested (a cascata)

Invece di progettare un unico controllore complesso che vada direttamente da P_{ref} a V (tensione), passando "dentro" a tutti questi passaggi logici incerti, conviene un **approccio annidato (nested)**: si progetta prima un **controllore di corrente** interno (il regolatore R nello schema a blocchi, da non confondere con la resistenza elettrica del motore), che si occupa di ottenere la corrente desiderata e quindi, di conseguenza, la giusta tensione.

Questo controllore chiude un anello attorno al solo sottosistema elettrico del motore: riceve l'errore tra la corrente di riferimento I_{ref} e quella effettivamente misurata I, e decide la tensione V da applicare per correggerlo. La corrente così ottenuta prosegue poi, in catena aperta rispetto a questo anello, attraverso il sottosistema meccanico (che la trasforma in spostamento del pistone x) e infine il sottosistema idraulico (che genera la pressione P).

Il tipo di incertezza su cui bisogna progettare il controllore di corrente riguarda tipicamente la resistenza R e l'induttanza L del motore, che non sono note con precisione: il controllore va progettato per essere **robusto** rispetto a variazioni di R e L.

L'idea di fondo è la stessa di un controllo in cascata: si isola il tratto della catena con i parametri più incerti (quello elettrico, R e L) e lo si rende affidabile con un anello dedicato, così che un eventuale anello più esterno (ad esempio un controllore di pressione, che agirebbe fornendo I_{ref} a questo blocco) possa trattare l'intero sottosistema elettrico come un attuatore già ben controllato, senza doversi occupare direttamente della sua incertezza interna.

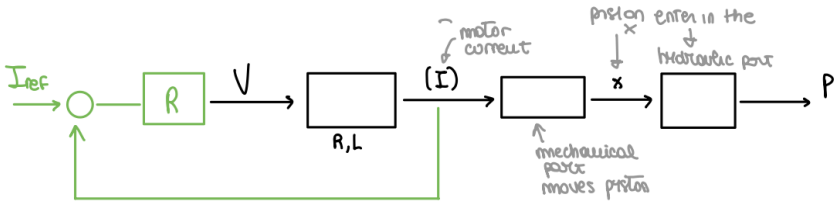


Figure 1.5: Current Controller

Una volta progettato il controllore di corrente, si può progettare sopra di esso, in cascata, un **controllore di posizione**: input l'errore di posizione e, output il riferimento di corrente. Il punto chiave è che questo controllore "vede" solo la risposta meccanica del sistema, perché la parte elettrica è già stata presa in carico dal controllore interno — *se le cose sono fatte correttamente, si possono progettare controllori in sequenza senza dover conoscere cosa c'è "dentro"* allo stadio precedente.

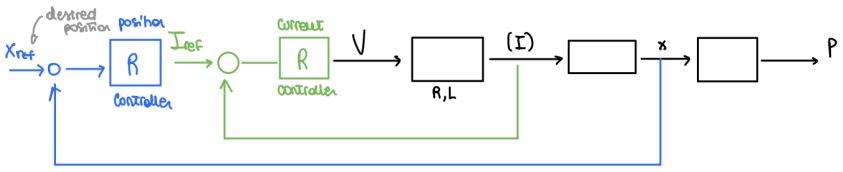


Figure 1.6: Position Controller

Analogamente, sopra il controllore di posizione si può progettare un **controllore di pressione** esterno: prende la differenza tra pressione desiderata e misurata e restituisce in output la posizione di riferimento, cosa possibile perché la posizione è già controllata in modo affidabile dal loop interno.

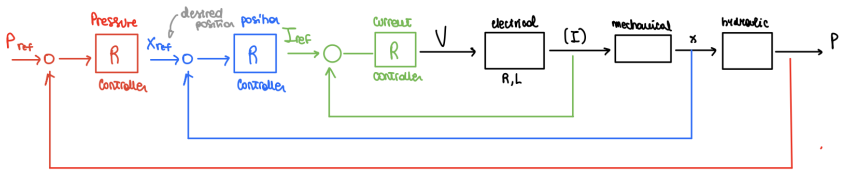


Figure 1.7: Pressure Controller

1.3.4.2 Progettazione del segnale di identificazione

Per identificare correttamente la dinamica servono segnali che eccitino tutte le frequenze di interesse. Un segnale a rampa lenta, ad esempio, mostra solo le basse frequenze — non va bene. Anche il rumore bianco non è ideale, essendo un segnale casuale di cui non conosciamo a priori i valori assunti.

Una soluzione pratica è il segnale **PRBS** (Pseudo Random Binary Sequence): una particolare sequenza di gradini a durata variabile, capace di eccitare il sistema fino a una frequenza scelta. Ha due caratteristiche utili:

- è **binario** — assume solo due valori (0 e un valore massimo scelto), il che rende conveniente limitare l'ampiezza del segnale;
- è **pseudo-random** — passa da basso ad alto in modo pseudo-casuale, quindi eccita tutte le frequenze pur restando ripetibile (a differenza del rumore bianco puro).

Il requisito generale è che l'esperimento sia **persistentemente eccitante**: solo con dati di questo tipo si può ricavare un modello affidabile su tutto lo spettro di frequenze rilevante.

1.3.4.3 Design del controllore

Un sistema del secondo ordine viene tipicamente adattato ai dati sperimentali raccolti in anello aperto:

$$G_x(s) = \frac{\mu_x}{\frac{s^2}{\omega^2} + \frac{2\zeta s}{\omega} + 1} \qquad \text{(2 poli)}$$

Con due poli si può progettare un controllore **PID**, secondo la tecnica del *loop shaping*: si sceglie il PID in modo da cancellare la dinamica del sistema e imporre la banda passante desiderata (ad esempio 25 Hz):

$$R_x(s) = K_p + \frac{K_i}{s} + K_d s$$

tipicamente con l'aggiunta di un polo ad alta frequenza per rendere il PID fisicamente realizzabile. Si ottiene così:

$$G_x(s)\, R_x(s) = L_x(s) = \frac{2\pi f}{s}$$

Perché puntare a 25 Hz e non, per esempio, a 5 Hz direttamente? Perché la banda passante del loop *esterno* (quello di pressione) dovrà essere più piccola di quella del loop *interno*: solo così il loop interno si comporta, agli occhi del loop esterno, come un semplice guadagno statico — un requisito standard nel controllo a cascata.

1.3.4.4 Validazione

Si esegue nuovamente un esperimento di identificazione, questa volta sulla funzione di trasferimento in **anello chiuso**, misurando la dinamica da riferimento a riferimento (da x a x). Ci si aspetta che questa funzione di trasferimento sia pari a 1 fino a 25 Hz, per poi iniziare a perdere ampiezza. Il modello identificato risulta infatti essere un filtro passa-basso del primo ordine a 25 Hz, come desiderato: i dati sperimentali restano vicini a 0 dB fino a quella frequenza, confermando la bontà del design.

Questi stessi dati di validazione possono essere riutilizzati anche come dati di identificazione per il loop *esterno* di pressione, che usa come variabile di controllo la posizione di riferimento e come variabile misurata la pressione.

Per il **controllo di pressione** (loop esterno) si richiede una banda passante di 5 Hz. Si usa un controllore **PI** per cancellare il polo a 25 Hz del loop interno e fissare la banda a

5 Hz:

$$G_p(s) = \frac{\mu_p}{sT_p + 1} \quad \Rightarrow \quad R_p(s) = K_p + \frac{K_i}{s}, \qquad G_p(s)R_p(s) = L_p(s) = \frac{2\pi f}{s}$$

Anche in questo caso, la validazione conferma che la funzione di trasferimento in anello chiuso è un filtro passa-basso del primo ordine a 5 Hz. L'idea di fondo di questo intero metodo è di "ingannare" il sistema trattandolo come se fosse **lineare**, anche se non lo è perfettamente.

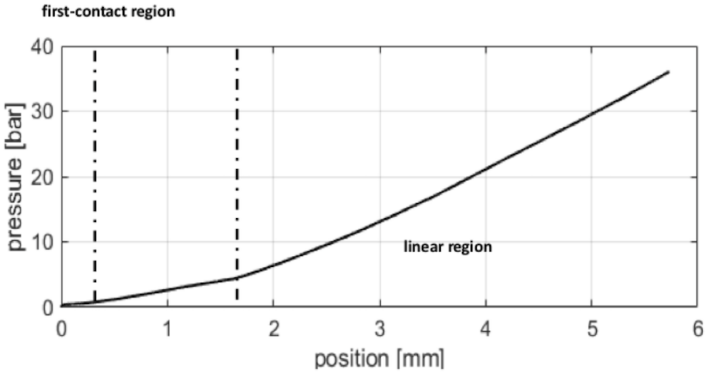
1.3.5 Non linearità: relazione posizione-pressione

Un esperimento interessante consiste nel rimuovere il disco e chiedersi: qual è la relazione statica tra posizione del cilindro maestro e pressione generata?

Finché la pastiglia non tocca il disco non c'è alcuna relazione (pressione nulla); quando inizia il contatto, la pressione comincia a crescere. La forma di questa relazione dipende dall'elasticità del sistema: i fluidi sono incomprimibili, quindi una volta che la pastiglia è a contatto con il disco non ci sarebbe più margine di movimento — ma in pratica il sistema presenta una certa elasticità (il fluido non è ideale), quindi c'è comunque una variazione continua di pressione con la posizione.

Il grafico posizione-pressione mostra quindi tre regioni:

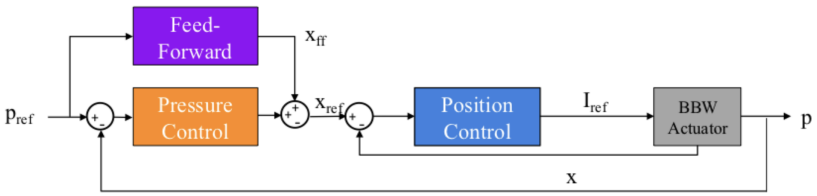
- una **deadzone** (zona morta) iniziale, in cui la pastiglia non è ancora a contatto col disco e muovere il pistone non produce alcun cambiamento di pressione;
- una regione a crescita progressiva;
- una **regione lineare** finale, dove pendenza e comportamento si stabilizzano.



Questa è chiaramente una fonte di non linearità nel sistema. Il modo più semplice per gestirla è una compensazione tramite **feed-forward**: dato un P_{ref}, si legge dalla carat-

teristica (invertita) un valore di posizione di feed-forward x_{ff}, che porta il sistema in open-loop già vicino alla posizione target corrispondente alla pressione desiderata.

Lo schema risultante è quindi a due contributi:



Il termine di feed-forward guida in anello aperto la posizione vicino al target desiderato; il controllore di pressione (in anello chiuso, progettato assumendo linearità) si occupa poi di correggere l'errore residuo. Localmente, se non ci si allontana troppo dalla regione operativa, l'approssimazione lineare resta accettabile.

In sintesi, il processo completo di progettazione del controllore del freno segue sempre lo stesso schema concettuale: identificazione → tuning → validazione, tenendo sempre presente la relazione tra input e output ad ogni stadio della cascata. Il sistema di frenata nel suo complesso può essere descritto, in modo compatto, come una funzione di trasferimento dal comando di frenata (brake request) alla pressione misurata, modellata con un **singolo polo con ritardo**:

$$G_{brake}(s) = \frac{p}{s+p} \cdot e^{-\tau s}$$

con un ritardo di circa 30 ms e una banda passante di circa 8 Hz.

1.3.6 Dry solution

Un'alternativa alla wet solution è la **dry solution** (Dry / full EM BBW, senza backup idraulico): un motore muove direttamente un pistone, senza il fluido idraulico intermedio del backup manuale.

Figure 1.8: Dry Solution

Vantaggi.

- Meno componenti nel sistema;
- Maggiore flessibilità di layout: nel sistema idraulico il cilindro maestro deve restare vicino, mentre qui il posizionamento è più libero;
- Il fluido idraulico permette comunque una risposta più **veloce** rispetto ad alternative puramente meccaniche.

Svantaggi.

- È un **single point of failure**: essendoci meno ridondanza (nessun percorso idraulico di backup manuale), l'affidabilità complessiva ne risente;
- È necessario stimare con grande precisione la forza applicata, il che è difficile: l'unica perdita tra pressione sul disco e sulla pastiglia è l'attrito della pinza, quindi in teoria la corrente del motore (facilmente controllabile) è già una buona misura della coppia e quindi della forza — ma serve comunque un sensore di forza accurato oppure una stima molto precisa, il che resta un punto critico.

1.4 Steer Actuation

L'attuazione dello sterzo è più complessa rispetto a quella del freno, per due ragioni principali:

- la **sicurezza** è ancora più importante (come visto, lo sterzo ha la massima criticità di sicurezza);
- l'handover non è solo una questione di controllo di posizione, ma coinvolge anche la **sensazione** (feeling) trasmessa al guidatore.

Esistono tre approcci principali:

1.4.1 EPS — Electric Power Steering

Nell'EPS esiste ancora un collegamento **meccanico** tra volante e ruote (attraverso il piantone di sterzo e la cremagliera / rack). Questo collegamento meccanico significa che c'è trasmissione diretta della forza dalle ruote al volante, e per questo il passaggio (switching) tra guida manuale e guida automatica risulta relativamente semplice.

In pratica, il problema si riduce a **controllare la cremagliera**: invece di usare il motore

per amplificare la coppia applicata dal guidatore (come avviene in un normale EPS assistivo), lo si usa per applicare la torsione calcolata dalla Navigation Control Unit. Serve però l'accesso al sistema EPS a basso livello.

Più nel dettaglio, il sistema EPS (ad esempio quello ad assi paralleli montato su alcune Porsche) integra: una barra di torsione flessibile con misura di coppia (torsion bar torque control), un sensore di posizione della cremagliera (encoder) e il motore stesso.

- In **modalità guida manuale**, il motore elettrico aiuta il guidatore a sterzare le ruote: il feedback di coppia dalla barra di torsione viene usato per ridurre lo sforzo richiesto al guidatore.
- In **modalità guida autonoma**, il guidatore rilascia il volante; il motore controlla direttamente la posizione della cremagliera, e il volante ruota di conseguenza per via del collegamento meccanico al piantone.

1.4.2 SBW — Steer-by-Wire

Lo SBW segue la stessa logica del Brake-by-Wire: il collegamento meccanico tra volante e cremagliera viene sostituito da un collegamento puramente elettronico. Non esiste quindi un legame meccanico tra ruote e volante.

Questo comporta:

- la necessità di attuatori e sensori sia sul lato volante sia sul lato cremagliera (quindi **due motori**);
- una maggiore complessità e gli stessi problemi già visti per il Brake-by-Wire in termini di ridondanza e affidabilità;
- la necessità, se si vuole permettere una transizione da guida autonoma a guida manuale, di garantire che questa transizione abbia senso dal punto di vista del guidatore.

Il funzionamento dei due motori è il seguente:

1. **In modalità guida manuale**: l'angolo del volante viene misurato e usato per calcolare un riferimento di posizione per l'attuatore della cremagliera. Contemporaneamente, la forza sulla barra (tie-rod force) viene stimata sul lato cremagliera e usata per calcolare una coppia di riferimento per il motore del volante — questo è il **feedback di coppia** che restituisce al guidatore la sensazione (feeling) di ciò che sta accadendo sulle ruote.

2. **In modalità guida autonoma**: il motore della cremagliera controlla direttamente la posizione seguendo il riferimento del sistema autonomo; il volante (se presente) può restare fermo, poiché non c'è più collegamento meccanico.

Il volante calcola quindi le coppie motore di feedback secondo un'equazione del tipo:

$$J_{sw}^{eq}\,\ddot{\theta}_{sw} + b_{sw}^{eq}\,\dot{\theta}_{sw} + k_{fbk}\,C_{fbk} = C_{sw}$$

mentre sul lato cremagliera:

$$m_{rack}^{eq}\,\ddot{x}_{rack} + b_{rack}^{eq}\,\dot{x}_{rack} + F_{friction}^{eq} + F_{tierod} = F_{servo}$$

Un dato interessante, indicativo dell'ordine di grandezza in gioco: la massa della cremagliera è dell'ordine di 5 kg, mentre la massa equivalente del motore è superiore a 500 kg — questa grande differenza di inerzia equivalente è tipica dei sistemi con riduzione meccanica elevata, ed è un aspetto rilevante nella progettazione del controllore.

Ad oggi, il sistema SBW è ancora poco diffuso: l'unica vettura di serie nota per adottarlo è Tesla.

1.4.3 Retrofitting

Il **retrofitting** consiste nell'aggiungere un motore addizionale che agisce meccanicamente come farebbe un guidatore, ingranato sulla cremagliera o sul piantone di sterzo esistente, insieme a sensori ed ECU dedicate. Il vantaggio è la modularità: qualunque veicolo esistente può essere trasformato in un veicolo a guida autonoma senza modificare il sistema di sterzo originale.

Anche qui, come per il freno, l'obiettivo è l'inseguimento (tracking) di un angolo di riferimento, con gli stessi requisiti prestazionali: banda passante elevata (ad esempio > 5 Hz, ritardo < 40 ms) e overshoot nullo o minimo.

1.4.3.1 Verifica di linearità e saturazione

Un esperimento tipico osserva la risposta del sistema a gradini (step) di ampiezza diversa:

- un gradino piccolo, da 0 a 1 grado;
- un gradino grande, da 0 a 11 gradi.

In un sistema **lineare**, il tempo di assestamento (settling time) dovrebbe essere lo stesso indipendentemente dall'ampiezza del gradino. Sperimentalmente si osserva invece che:

- con il gradino piccolo il tempo di assestamento è di circa 0.2 s — comportamento abbastanza lineare;
- con il gradino grande il tempo di assestamento sale a circa mezzo secondo, e la risposta cresce quasi come una **rampa a velocità costante**, senza mostrare il tipico rallentamento (decadimento esponenziale) di un sistema lineare del prim'ordine.

Questo indica che, per gradini grandi, l'attuatore è **saturato in velocità** (rate-limited): non riesce a muoversi più velocemente di un certo limite fisico, indipendentemente da quanto "grande" sia il comando.

Di fronte a questa non linearità, ci sono due strade possibili:

1. costruire un modello che descriva fedelmente anche questo comportamento saturato — un modello più complesso, utile per la simulazione, ma poco pratico per la progettazione del controllore;
2. l'approccio **preferito**: descrivere il sistema di sterzo come lineare, avendo però cura di progettare il sistema di controllo in modo da **non superare i limiti** fisici (ad esempio limitando opportunamente i riferimenti di velocità richiesti).

Il modello lineare tipico per lo sterzo è quindi ancora un polo con ritardo:

$$G_{steering}(s) = \frac{p}{s+p} \cdot e^{-\tau s}$$

(con banda passante indicativa di 4 Hz, ritardo puro di circa 0.01 s, e un limitatore di velocità di circa 20 deg/s), eventualmente affiancato da un **rate limiter** esplicito nel sistema di controllo.

Un livello di prestazioni superiore si può raggiungere solo tramite un approccio di **co-design**: progettare insieme meccanica e controllo invece di trattarli separatamente. In alcuni casi, tuttavia, questo non è possibile (ad esempio quando l'attuatore va integrato su un veicolo già esistente, come nel retrofitting).

1.4.3.2 Elasticità della trasmissione

Il sistema di sterzo comprende, oltre al motore (sorgente della coppia) e all'angolo del volante (variabile da controllare), anche una **trasmissione** (ad esempio una cinghia/puleggia tra motore e volante), che introduce inevitabilmente una certa elasticità (compliance): la trasmissione non è mai perfettamente rigida.

Questo fenomeno emerge chiaramente in prova: quando la posizione del motore è compresa in un piccolo intervallo (ad esempio tra -2% e $+2\%$), la risposta risulta piuttosto "piatta"

— segno della presenza di gioco o elasticità meccanica in quella zona. Va però osservato che, nella guida reale, le correzioni di sterzo sono nella maggior parte dei casi molto piccole: solo in poche manovre si verificano cambi di direzione rapidi e ampi, per cui bisogna sempre valutare le prestazioni del modello nell'intorno delle condizioni operative che interessano davvero.

Un modello un po' più raffinato, che tiene conto di questa elasticità, è del secondo ordine:

$$G_{steering}(s) = \frac{1}{\frac{s^2}{\omega^2} + \frac{2\xi s}{\omega} + 1} \cdot e^{-\tau s}$$

(tipicamente con ritardo di circa 20 ms, banda tra 1 e 3 Hz), anche in questo caso corredato da un eventuale rate limiter.

In generale, per i sistemi di sterzo bisogna assicurarsi che il modello lineare descriva bene le condizioni operative di reale interesse, tenendo sempre presente che la parte più critica del sistema (in termini di non linearità, saturazioni, elasticità) può facilmente diventare il vero **collo di bottiglia** (bottleneck) delle prestazioni complessive.

1.5 Traction Actuation

L'idea alla base dell'attuazione di trazione è il controllo del motore/motopropulsore. Si possono avere configurazioni con:

- **un singolo motore** centrale, con eventuale differenziale (con o senza differenziale attivo);
- **motori indipendenti**, ad esempio quattro motori indipendenti in configurazione "in-body" (motore centrale collegato alle ruote tramite trasmissione) oppure quattro motori indipendenti "in-wheel" (integrati direttamente nella ruota).

La coppia attuata è tipicamente una coppia **motrice** (driving torque), ma con i motori elettrici è possibile generare anche grandi coppie **frenanti** (tramite frenata rigenerativa), come già accennato a proposito del freno.

Un esempio concreto tratto dagli appunti: un sistema a doppio motore elettrico, trazione posteriore, con potenza massima di 378 kW (503 cv), coppia massima per motore di 200 Nm e velocità massima di 1257 rad/s (12000 RPM). In una prova di partenza da fermo a pieno gas (standing start, full throttle) si osserva un tempo di transitorio di circa 150 ms e un ritardo iniziale di circa 50 ms prima che la coppia richiesta inizi effettivamente a essere erogata.

1.5.1 Non linearità: elasticità della trasmissione

Anche nel caso della trazione, come già visto per lo sterzo, la **trasmissione meccanica** introduce un comportamento elastico (compliance) tra motore e ruota. Questo si manifesta tipicamente come una **frequenza di risonanza** (in un esempio concreto, attorno ai 6–7 Hz), un fenomeno noto come *transmission shuffle*: nella risposta a un gradino di coppia, si osservano oscillazioni smorzate attorno al valore finale, dovute proprio all'elasticità della trasmissione, prima che il sistema si stabilizzi.

In sintesi, la lezione che emerge trasversalmente da sterzo e trazione è che ogni trasmissione meccanica reale introduce elasticità (compliance elettrica/meccanica), e questa va sempre tenuta in considerazione sia nella modellazione sia nella validazione del controllore.

2 | Sensors

2.1 Introduzione: proprioceptive vs exteroceptive

I sensori sono ciò che alimenta i moduli di *Perception* e *Localization*. Si dividono in due grandi categorie:

- **proprioceptive sensors** — misurano grandezze interne al veicolo (ad esempio velocità delle ruote, accelerazione, angolo di sterzo);
- **exteroceptive sensors** — misurano ciò che sta *fuori* dal veicolo, cioè l'ambiente circostante. All'interno di questa categoria troviamo due grandi famiglie:
  - **Time of Flight sensors** — sensori attivi basati sulla misura del tempo di volo di un'onda;
  - **Vision based sensors** — sensori passivi basati sull'acquisizione di immagini (telecamere).

2.2 Time of Flight Sensors

2.2.1 Principio di funzionamento

Il principio di funzionamento è comune a tutta questa famiglia di sensori: si emette un'onda la cui velocità di propagazione è nota, e si misura il tempo che questa impiega a tornare indietro al sensore dopo aver colpito un ostacolo — da cui il nome, *Time of Flight*, che indica proprio il fatto che si sta misurando un tempo.

La misura del tempo può avvenire in due modi:

- per **conteggio diretto** del tempo di andata e ritorno: se d è la distanza dal sensore all'ostacolo e v la velocità dell'onda,

$$d = v\,\Delta t$$

- per **sfasamento di fase** (phase shift), utile quando l'onda emessa è periodica (ad esempio un'onda sinusoidale continua invece di un singolo impulso):

$$d = \frac{v}{f}\,\Delta\phi$$

dove f è la frequenza dell'onda e $\Delta\phi$ lo sfasamento misurato.

L'elemento che caratterizza e distingue le diverse tecnologie di questa famiglia è il **tipo di onda** utilizzata, poiché questo determina soprattutto il *range* operativo del sensore:

Tipo	Range	Risoluzione angolare
Ultrasound	1 – 4 m	20°
Radar	1 m – 200 m	2°
Collimated Light (LiDAR)	1 m – 200 m	0.1°
Scattered Light	1 m – 35 m	2°

2.2.2 Ultrasound

Gli ultrasuoni sono la tecnologia alla base dei classici **sensori di parcheggio**. Sono molto economici (cost effective) e facilmente integrabili nel veicolo.

Pro:

economici, elaborazione semplice, bassa potenza richiesta, robusti.

Contro:

basso range (non oltre i 5 metri circa) e bassa risoluzione — è molto difficile "focalizzare" un'onda sonora in una direzione precisa, per cui il fascio risulta piuttosto ampio.

Un esempio numerico tipico: con velocità del suono $c_0 = 350\,\mathrm{m/s}$ e un range desiderato $R = 10\,\mathrm{m}$, la frequenza di ripetizione per un singolo sensore è circa

$$f = \frac{c_0}{R} \approx 35\,\mathrm{Hz}$$

Un problema pratico quando si usano **più sensori** contemporaneamente (ad esempio più sensori di parcheggio disposti sul paraurti) è che le onde emesse da sensori diversi possono **interferire** tra loro. Il modo più semplice per evitare questo problema è far emettere le onde dei vari sensori **in sequenza** (uno alla volta) invece che simultaneamente.

2.2.3 Collimated Light: il LiDAR

Principio di funzionamento

Il **LiDAR** (Light Detection and Ranging) spara impulsi laser in diverse direzioni, misurando il tempo che ciascun impulso impiega a tornare indietro dopo essere stato riflesso da un ostacolo. Ogni impulso riflesso genera un **punto nello spazio**, caratterizzato da coordinate (x, y, z). L'insieme di tutti questi punti raccolti in una scansione prende il nome di **point cloud** (nuvola di punti).

2.2.3.1 Classificazione dei LiDAR

I LiDAR possono essere classificati secondo due assi differenti.

In base alla dimensionalità della scansione:

- **1D** — o *range finder*: misura semplicemente una distanza lungo una singola direzione;
- **2D** — scansiona l'ambiente orizzontalmente;
- **3D** — scansiona l'ambiente in tutte le direzioni.

In base alla tecnologia di emissione:

- **Elettromeccanico** (LiDAR a scansione) — un range finder che viene fisicamente mosso (ruotato) per esplorare diverse direzioni;
- **Solid state** — gli emettitori ottici inviano impulsi di fotoni con pattern e fasi specifiche per creare un'emissione direzionale, *senza parti meccaniche in movimento*;
- **Flash** — l'intero campo visivo viene illuminato con un unico ampio fascio laser divergente, in un solo impulso.

Solid state — il principio.

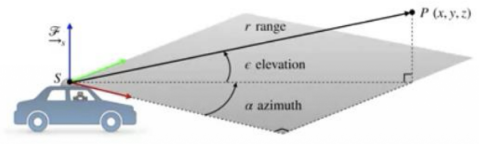
Sincronizzando l'emissione di più emettitori disposti in serie, ciascuno con una fase leggermente diversa (indicata schematicamente come A_0e^{j0}, $A_1e^{j\Delta\phi}$, $A_2e^{j2\Delta\phi}$, ...), si può controllare la direzione del fronte d'onda risultante: variando lo sfasamento relativo tra gli emettitori, il fronte d'onda si muove in una direzione specifica. In questo modo si "sceglie" elettronicamente la direzione di puntamento, senza muovere nulla fisicamente; bisogna però esplorare le diverse direzioni una alla volta.

Flash — il principio.

Invece di avere emettitori multipli, si ha un unico emettitore che illumina tutto il campo visivo in un colpo solo, e si usano invece **ricevitori multipli** per registrare il segnale riflesso in tutte le direzioni contemporaneamente. Questo approccio ha il vantaggio di un'**acquisizione sincrona**: tutti i punti della scansione sono acquisiti nello stesso istante.

Sia i LiDAR solid state sia quelli flash condividono due vantaggi rispetto alla soluzione elettromeccanica: **nessuna parte in movimento** e maggiore **compattezza**. Il flash, in aggiunta, offre l'acquisizione sincrona sopra citata.

2.2.3.2 Coordinate del LiDAR



I LiDAR misurano tipicamente i punti in **coordinate sferiche**: range r (la distanza dal sensore, misurata tramite time of flight), **azimuth** α (l'angolo orizzontale che descrive la direzione di puntamento del raggio attorno al sensore, sul piano orizzontale) ed **elevazione** ϵ (l'angolo verticale che descrive quanto il raggio punta verso l'alto o verso il basso rispetto al piano orizzontale) — entrambi gli angoli sono misurati tramite encoder, nel caso di LiDAR elettromeccanici.

Inverse Sensor Model

(da sferiche a cartesiane):

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} r\cos(\alpha)\cos(\epsilon) \\ r\sin(\alpha)\cos(\epsilon) \\ r\sin(\epsilon) \end{bmatrix}$$

Forward Sensor Model

(da cartesiane a sferiche):

$$\begin{bmatrix} r \\ \alpha \\ \epsilon \end{bmatrix} = \begin{bmatrix} \sqrt{x^2+y^2+z^2} \\ \tan^{-1}\left(\frac{y}{x}\right) \\ \sin^{-1}\left(\frac{z}{\sqrt{x^2+y^2+z^2}}\right) \end{bmatrix}$$

Oltre alla posizione, ogni punto porta con sé anche un'informazione di **colore/intensità**:

quanto è intensa la luce riflessa dall'ostacolo (su un oggetto nero, ad esempio, l'intensità sarà più bassa). Un aspetto concettualmente importante, spesso trascurato: *anche il suolo è a tutti gli effetti un ostacolo* per il LiDAR — genera punti riflessi esattamente come qualunque altro oggetto.

2.2.3.3 Sorgenti di rumore e il fenomeno del jitter

Quanto possiamo fidarci di un LiDAR? Bisogna capire quali tipi di rumore lo affliggono. Il modello classico, in coordinate cartesiane, è:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} r\cos(\alpha)\cos(\epsilon) \\ r\sin(\alpha)\cos(\epsilon) \\ r\sin(\epsilon) \end{bmatrix} + v$$

dove v rappresenta il rumore di misura. Le principali sorgenti di rumore sono:

- incertezza nella misura del tempo di arrivo;
- incertezza nella misura dell'orientamento esatto;
- interazione con il target (il modo in cui la superficie riflette la luce);
- **jitter** (specifico dei LiDAR a scansione).

Il modello sopra assume che il rumore che affligge ciascun punto sia **scorrelato** da punto a punto. Il **jitter** è invece un fenomeno diverso, perché genera un rumore **correlato**: anche se siamo abituati a pensare a una point cloud come se fosse acquisita in un unico istante temporale, in realtà, guardando da vicino, **ogni punto viene misurato in un istante di tempo leggermente diverso** dagli altri, dato che la scansione richiede un certo tempo per esplorare tutte le direzioni.

Un esempio intuitivo.

Immaginiamo di stare guidando verso un muro. Iniziamo la scansione e mettiamo un punto in quella direzione; mentre il LiDAR cambia direzione di scansione, l'auto nel frattempo si è mossa, quindi la distanza misurata al prossimo impulso sarà diversa — e così via. Se mettiamo insieme (fondiamo) le informazioni di questa scansione *senza* considerare che sono state prese in istanti diversi — cioè assumendo, erroneamente, che siano tutte relative allo stesso istante — otteniamo una ricostruzione distorta della scena.

Il jitter nel LiDAR è concettualmente simile a scattare una foto panoramica da un'auto in movimento: gli oggetti appaiono leggermente disallineati perché la telecamera (o l'auto)

non è perfettamente ferma durante l'acquisizione.

Soluzione.

Bisogna considerare, insieme ai dati del LiDAR, un'informazione **esterna** sulla velocità del veicolo: serve cioè fare **sensor fusion**, combinando le informazioni del LiDAR con quelle di altri sensori (ad esempio sensori inerziali che misurano il moto del veicolo durante la scansione).

Vantaggi e svantaggi del LiDAR

Vantaggi:

range fino a 200–300 m e risoluzione angolare molto elevata.

Svantaggi:

generano una grande quantità di dati e lavorano tipicamente a frequenze relativamente basse (circa 20 Hz), un aspetto che va tenuto in conto insieme al problema del jitter appena descritto.

2.2.4 Radar

2.2.4.1 Classificazione: SISO, SIMO, MIMO

I radar si classificano in base al numero di canali di trasmissione (Tx) e ricezione (Rx):

- **SISO** (Single Input Single Output) — un solo canale di trasmissione e uno di ricezione. Caratteristiche: misura di range e velocità; la stima della direzione di arrivo (DoA, Direction of Arrival) richiede lo **steering meccanico o elettronico** dell'antenna.
- **SIMO** (Single Input Multiple Output) — un canale di trasmissione, canali di ricezione multipli. Fornisce range/velocità e permette la stima della DoA *senza* bisogno di steering meccanico del pattern d'antenna.
- **MIMO** (Multiple Input Multiple Output) — canali di trasmissione e ricezione multipli. Offre range/velocità, una stima della DoA migliorata, e in più la capacità di stimare l'**elevazione**.

È importante notare che i radar SISO e SIMO **non permettono di risolvere l'elevazione**: forniscono informazioni essenzialmente in un piano.

2.2.4.2 Il ciclo di scansione (scan cycle)

Un **scan cycle** radar è tipicamente composto da fino a cinque tipi di messaggi estratti dal bus CAN:

1. **Status** — ricevuto una volta per ciclo di scansione, contiene il numero di oggetti rilevati nella scansione;
2. **General** — ricevuto n volte (una per ogni oggetto rilevato), contiene informazioni su **posizione, velocità** e **sezione radar equivalente** (radar cross section) di ciascun oggetto tracciato;
3. **Quality** — ricevuto n volte, contiene informazioni su **incertezza cinematica**, **probabilità di esistenza** e **ID univoco** dell'oggetto;
4. **Extended** — ricevuto n volte, contiene proprietà aggiuntive dell'oggetto, come la sua classe;
5. **Warning** — ricevuto n volte, contiene lo stato dell'avviso di collisione (collision detection warning).

2.2.4.3 Effetto Doppler e caratteristiche distintive del radar

Trattandosi di radar (onde radio), si può sfruttare l'**effetto Doppler**: questo rende relativamente facile ottenere informazioni sulla **velocità** degli oggetti rilevati direttamente dal segnale radar. **É il grande vantaggio che i Radar hanno**.

Un aspetto interessante riguarda il ritorno del segnale: materiali diversi riflettono le onde radio in modo diverso — un oggetto metallico, ad esempio, avrà una **sezione radar equivalente** (radar cross section) maggiore rispetto ad altri materiali. Inoltre, se in una data direzione è presente un oggetto molto grande, un oggetto più piccolo nella stessa direzione tenderà a riflettere un segnale simile/dominato da quello dell'oggetto grande.

Con il radar abbiamo quindi anche un'idea della **dimensione** dell'oggetto, e la capacità di **tracciare** gli oggetti da un frame all'altro (associando le rilevazioni nel tempo a uno stesso oggetto).

2.2.5 Scattered Light

I sensori a *scattered light* lavorano tipicamente su un range più basso (fino a circa 35 m), il che li rende meno utili per applicazioni automotive rispetto a radar e LiDAR. In generale, per la guida autonoma, sono i LiDAR (basati su luce collimata) a essere la scelta preferita in questa fascia di prestazioni.

2.3 Localization Sensors: il GNSS

2.3.1 Cos'è il GPS/GNSS

Il **GPS** (Global Positioning System) nasce nel 1970 presso lo US Department of Defense, dalla necessità di localizzare veicoli militari sulla Terra e nello spazio. Il sistema è composto da tre elementi:

- **satelliti** (space segment);
- **ricevitori** dei segnali radio;
- **stazioni di controllo** che monitorano lo stato dei satelliti.

Il termine **GNSS** (Global Navigation Satellite System) è quello generico usato per riferirsi a *tutti* i sistemi satellitari, di cui il GPS è solo uno:

- **GPS** — sistema satellitare statunitense (32 satelliti);
- **Galileo** — sistema satellitare europeo (30 satelliti);
- **GLONASS** — sistema satellitare russo (24 satelliti);
- **BeiDou** — sistema satellitare cinese (30 satelliti);
- **IRNSS** — sistema di navigazione regionale indiano (9 satelliti);
- **QZSS** — sistema satellitare giapponese (7 satelliti).

IRNSS e QZSS sono sistemi **locali**: non coprono l'intero globo, ma aumentano la copertura nelle loro regioni di competenza (un singolo sistema globale ha bisogno di un minimo di circa 24 satelliti per garantire copertura completa). La maggior parte dei ricevitori moderni sono **multi-provider**, cioè possono ricevere segnali da più costellazioni contemporaneamente.

2.3.2 Come funziona il posizionamento

Ogni satellite trasmette un segnale radio che contiene:

- l'**ID** del satellite;
- la **posizione** del satellite;
- il **tempo** del satellite;
- un **codice pseudo-casuale** (pseudo random code).

Per ogni satellite, il ricevitore recupera l'informazione trasmessa e il tempo di trasmissione, determinando così il **tempo di propagazione** Δt. Il **range** tra ricevitore e satellite si ottiene come:

$$\text{Range} = c \cdot \Delta t = c \cdot (t_{receiver} - t_{satellite})$$

dove c è la velocità della luce nel vuoto, circa 300,000,000 m/s.

L'accuratezza di $t_{satellite}$ è molto elevata (circa 10 ns), essendo generato da un orologio atomico; l'accuratezza di $t_{receiver}$ è invece molto peggiore, dipendendo dalla qualità dell'orologio del ricevitore.

Il messaggio di navigazione e il codice pseudo-casuale sono **modulati su frequenze portanti** (carrier frequencies): storicamente L1 (1575.42 MHz) e L2 (1227.6 MHz), a cui si è aggiunta più recentemente L5 (1176.45 MHz) nei satelliti più nuovi. Il ricevitore decodifica il messaggio a partire da questa modulazione.

Il codice **pseudo-random** (non ripetitivo, che sembra casuale ma è in realtà noto e prevedibile) è ciò che il ricevitore usa per stimare il tempo di propagazione: il satellite invia questo codice, e il ricevitore, conoscendolo a priori, sa che tipo di segnale attendersi.

2.3.3 Il problema della sincronizzazione

Non abbiamo un orologio sincronizzato tra satellite e ricevitore. Se lo avessimo, il problema si ridurrebbe a un semplice controllo di tempo. Ma l'orologio disponibile sul ricevitore non è preciso quanto quello del satellite (atomico), quindi non possiamo assumere la sincronizzazione. Esistono due soluzioni per affrontare questo problema.

2.3.3.1 Soluzione code-based

Il codice pseudo-casuale è **noto al ricevitore**: c'è quindi una correlazione tra il messaggio ricevuto e il codice che il ricevitore si aspetta di ricevere. Il ricevitore genera internamente una copia del codice pseudo-casuale identica a quella del satellite, e allinea questa copia con il segnale in arrivo; lo spostamento (shift) temporale necessario per ottenere l'allineamento rappresenta il tempo di volo del segnale.

Ogni bit del messaggio corrisponde a una distanza fisica precisa: si ottiene un'**accuratezza fino a circa 5 metri**. Poiché il codice non si ripete (non ci sono ripetizioni all'interno del periodo considerato), esiste un solo modo per allineare correttamente il segnale ricevuto con quello atteso — se il segnale si ripetesse periodicamente in tempi brevi, ci sarebbero invece più modi possibili di allinearlo, introducendo ambiguità.

Vantaggi di questo metodo: **nessuna ambiguità** grazie all'unicità del codice, buona accuratezza (circa 5 m), basso sforzo computazionale e dispositivi economici. Il codice pseudo-casuale (PRN) si ripete tipicamente ogni 1 ms.

2.3.3.2 Soluzione carrier-based

Questo metodo misura la **fase** dell'onda portante (carrier) nel momento in cui arriva al ricevitore. L'idea centrale è determinare il numero totale di lunghezze d'onda complete della portante tra satellite e ricevitore nel momento in cui il segnale viene acquisito per la prima volta, più la parte frazionaria della lunghezza d'onda; questo numero totale di cicli è noto come **ambiguità intera** (integer ambiguity).

Guardiamo cioè allo sfasamento della portante stessa. La lunghezza d'onda della portante L1 è circa 19 cm: questo significa che si può ottenere una misura molto più **precisa** rispetto al metodo code-based. Lo svantaggio è che la portante **si ripete** periodicamente (è un'onda sinusoidale): siamo molto precisi, ma non sappiamo esattamente in quale "periodo" ci troviamo — essendo periodica, si allinea ogni periodo. Esistono metodi (di risoluzione dell'ambiguità intera) per determinare la quantità reale di cicli completi trascorsi, e ottenere così una soluzione molto precisa.

Questo tipo di soluzione è più **preciso** ma anche più **costoso**; la soluzione code-based è invece computazionalmente efficiente e richiede dispositivi economici.

2.3.4 Ricostruzione della posizione

In uno spazio 3D, il ricevitore potrebbe trovarsi ovunque su una sfera di raggio pari al range misurato rispetto a un dato satellite. In teoria, la posizione $\{x, y, z\}$ si può calcolare a partire da tre misure di range, sfruttando anche la conoscenza del fatto di trovarsi sulla superficie terrestre. Tuttavia, poiché l'orologio del ricevitore non è sufficientemente accurato, occorre aggiungere anche il suo **offset temporale** t_{offset} come incognita: lo stato da stimare diventa quindi $\{x, y, z, t_{offset}\}$.

Servono quindi **almeno 4 misure satellitari** per ottenere una stima di posizione accettabile: con 4 satelliti, concettualmente, 3 vengono usati per calcolare l'intersezione delle sfere di range, e 1 per sincronizzare l'offset temporale.

2.3.5 Sorgenti di errore *

Sorgente	Errore di range
Ritardi ionosferici	±5 m
Errori orbitali	±2.5 m
Errori dell'orologio satellitare	±2 m
Multipath	±1 m
Ritardi troposferici	±0.5 m
Rumore del ricevitore	±0.3 m

I **ritardi ionosferici e troposferici** sono dovuti all'attraversamento, da parte del segnale, di strati dell'atmosfera che ne rallentano/deviano la propagazione. Il fenomeno del **multipath** avviene quando la luce (il segnale radio) viene riflessa e rifratta — nel caso di edifici, può rimbalzare sulle facciate degli edifici stessi. Il messaggio che il ricevitore ottiene dal satellite presenta quindi, in questi casi, un percorso riflesso e uno rifratto: la distanza da percorrere per le due componenti è diversa, e il ricevitore finisce per ottenere più copie dello stesso messaggio, arrivate con tempi diversi.
Oltre ai ritardi ionosferici e troposferici e al multipath già descritti, contribuiscono all'errore complessivo anche gli **errori orbitali**, gli **errori dell'orologio satellitare** e il **rumore del ricevitore**. I primi derivano dal fatto che la posizione reale del satellite non coincide mai perfettamente con quella calcolata a partire dalle effemeridi trasmesse, poiché le orbite subiscono piccole perturbazioni gravitazionali e la pressione di radiazione solare, non completamente predicibili; questo errore può essere ridotto utilizzando effemeridi *precise*, calcolate a posteriori. Gli **errori dell'orologio satellitare** nascono invece dal fatto che il posizionamento si basa sulla misura del tempo di volo del segnale, e richiede quindi una sincronizzazione temporale estremamente accurata: gli orologi atomici di bordo, per quanto precisi, presentano piccole derive nel tempo, solo parzialmente compensate dalle correzioni trasmesse nel messaggio di navigazione. Il rumore del ricevitore, infine, è l'errore intrinseco dell'hardware — dovuto al rumore termico dei componenti elettronici, alla precisione del quarzo interno e alla qualità dell'antenna — e rappresenta il contributo più piccolo tra le sorgenti elencate, ponendo comunque un limite fisico alla precisione ottenibile.

2.3.6 Tecniche per ridurre gli errori

2.3.6.1 Approcci satellite-based

- **Multi-Frequency** — dispositivi capaci di ricevere segnali GNSS su due o più frequenze possono osservare e correggere errori dipendenti dalla frequenza, come i ritardi ionosferici e troposferici.

- **Multi-Constellation** — dispositivi capaci di ricevere segnali da costellazioni diverse (GPS, Galileo, ...) possono migliorare la copertura, ridurre gli errori causati da ostruzioni e ottenere una stima del range con maggiore accuratezza.

2.3.6.2 Differential GNSS (DGNSS)

Nel **DGNSS** si utilizza un **ricevitore aggiuntivo** sul terreno (base station) posizionato in una posizione **nota**. Questa posizione nota viene usata per calcolare l'errore tra la posizione misurata (dalla base) e quella reale: attraverso un collegamento dati (radio o internet), la base station invia queste correzioni agli altri ricevitori (rover).

Gli errori sono attribuibili principalmente a ritardi atmosferici e a errori di orbita/orologio. Se il rover si trova entro pochi chilometri dalla base station, il DGNSS funziona molto bene, poiché le condizioni atmosferiche in quella zona sono simili a quelle osservate dalla base.

La soluzione richiede: la conoscenza della posizione della base station, e un modo per comunicare la correzione.

2.3.6.3 Real-Time Kinematic (RTK)

L'**RTK** ha la stessa struttura del DGNSS, ma usa un metodo **carrier-based** per il calcolo del range invece del code-based usato dal DGNSS classico.

Un limite comune sia a DGNSS sia a RTK è che **non correggono il multipath né il rumore del ricevitore**: questi errori sono infatti locali al singolo ricevitore e non sono comuni tra base e rover, quindi non possono essere eliminati per confronto. Ad esempio, se il ricevitore è vicino a un edificio ma l'antenna è posizionata in alto, gli altri errori vengono corretti, ma il multipath resta un problema, non essendo soggetto a correzione differenziale.

2.3.6.4 Satellite Based Augmentation Systems (SBAS) e PPP

- **SBAS** — sistemi satellitari **geostazionari** che forniscono servizi regionali per migliorare integrità, accuratezza e disponibilità dei segnali GNSS di base. Alcune stazioni di riferimento stimano gli errori di tempo e orbita del satellite, includendo i ritardi ionosferici; le correzioni vengono poi inviate ai satelliti SBAS tramite stazioni di uplink.
- **Precise Point Positioning (PPP)** — ha la stessa struttura dell'SBAS ma usa un metodo carrier-based per il calcolo del range. Le correzioni sono disponibili in tutto il mondo via internet (gratuitamente).

La differenza rispetto a DGNSS/RTK è la seguente: le soluzioni precedenti (DGNSS, RTK) sono più **locali**, quindi soggette allo stesso tipo di errore se ci si trova più o meno nella stessa area geografica; le soluzioni SBAS/PPP usano invece satelliti **lontani** dal ricevitore, il che dà il vantaggio che un singolo satellite può coprire un'**area molto ampia**.

2.3.7 Confronto tra le tecniche di correzione

- **RTK** — accuratezza fino a 3 cm, ma limitata a un range di circa 50 km dalla base;
- **PPP** — accuratezza di circa 10 cm, ma disponibile a livello mondiale, purché si abbia connessione;
- **DGNSS** — copertura fino a 1000 km dalla base station, è un metodo code-based, quindi meno accurato ma meno costoso rispetto ai metodi carrier-based.

In generale, il guadagno maggiore in accuratezza si ottiene usando metodi **carrier-based** (RTK e PPP) rispetto a quelli code-based (DGNSS e SBAS); l'accuratezza migliora ulteriormente usando una base station **locale** (RTK e DGNSS), con la quale gli errori che affliggono il ricevitore sul rover vengono stimati meglio.

Sperimentalmente, un ricevitore **RTK** fornisce misure accurate al livello del **centimetro**, con un'elevata **ripetibilità** tra run diverse (le traiettorie ricostruite in prove ripetute risultano molto vicine tra loro) — sempre tenendo conto dell'errore di path tracking introdotto dal guidatore stesso durante l'acquisizione dei dati.

2.3.8 Pro e contro del GNSS

Pro:

localizzazione globale, informazione di tempo accurata, in aree aperte è possibile ottenere un'accuratezza di posizionamento a livello centimetrico.

Contro:

prestazioni scadenti vicino a edifici; accuratezza, disponibilità, integrità e continuità sono difficili e costose da ottenere; il segnale è facilmente disturbabile (jammabile) e soggetto a interferenze (fenomeni di GPS jamming/spoofing sono un problema riconosciuto anche a livello di sicurezza dell'aviazione civile).

In sintesi: il GNSS è uno strumento utile, ma bisogna essere consapevoli dei suoi limiti. Nel migliore scenario possibile si ottiene alta accuratezza solo con correzioni (RTK/PPP/DGNSS):

il **GNSS da solo non è la soluzione** completa al problema della localizzazione in guida autonoma — va tipicamente integrato con altri sensori (fusione con IMU, odometria, mappe, LiDAR, ecc.).

2.4 Vision and Cameras

2.4.1 Perché le telecamere

Tutti i sistemi di guida autonoma hanno almeno una telecamera nella loro suite di sensori. La telecamera è il sensore che fornisce l'informazione più dettagliata sull'**aspetto** (appearance) degli oggetti nell'ambiente — ad esempio il colore di un semaforo, che nessun altro sensore tra quelli visti finora è in grado di percepire. L'informazione di appearance permette di distinguere tra segnali stradali, semafori, ecc.

Le telecamere sono inoltre relativamente economiche. Lo svantaggio principale è che richiedono un'elaborazione **pesante** delle immagini per estrarne informazione utile.

Le telecamere sono sensori **esterocettivi passivi**: a differenza di LiDAR e radar, non emettono nulla — si limitano a misurare ciò che è già presente nell'ambiente (la luce riflessa dagli oggetti).

2.4.2 Elementi principali

Una telecamera è composta da tre elementi principali:

- **Lens** (lente) — per mettere a fuoco;
- **Aperture** (apertura) — per concentrare la luce sul sensore;
- **Sensor** — trasforma l'immagine in un segnale digitale/elettronico.

La catena tipica è: la scena reale viene messa a fuoco dalla lente, la luce viene filtrata da un pattern di filtri colorati (Bayer filter, che permette di ottenere informazione di colore da un sensore intrinsecamente monocromatico), il sensore RAW acquisisce il segnale, e un image processor lo trasforma nell'immagine finale in output.

2.4.3 Classificazione delle telecamere

- **Color vs Black and White**;
- **Focal length** (lunghezza focale) — una lunghezza focale maggiore corrisponde a un'immagine più "zoomata";

- **Monocular vs Stereoscopic**;
- **Spectrum** — luce visibile vs infrarosso;
- **Tipo di sensore** (CMOS vs CCD).

I requisiti specifici per applicazioni di guida autonoma sono un **lungo range** e la **robustezza** rispetto a condizioni di illuminazione molto diverse: guidando siamo infatti soggetti a condizioni di luce molto variabili — dalla luce diretta del sole a una giornata nuvolosa.

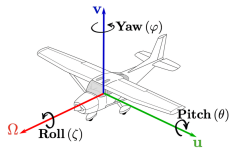
2.4.4 Camera Projective Geometry

2.4.4.1 Il concetto di posa (pose)

In un contesto 3D, la **posa** (pose) è la configurazione di un oggetto nello spazio, ed è composta da:

- **posizione** — solitamente descritta con 3 variabili (un vettore);
- **orientazione** — 3 gradi di libertà (DoF).

La posa (con eventualmente le sue derivate temporali) definisce il cosiddetto **Special Euclidean Group** $SE(3)$.



L'orientazione (rotazione) può essere descritta con diversi strumenti: matrici di rotazione, angoli di Yaw-Roll-Pitch (YRP), Axis Rotation, parametrizzazione di Rodriguez, quaternioni, tra gli altri.

2.4.4.2 Matrici di rotazione

Ogni colonna di una matrice di rotazione R è il versore del sistema di riferimento ruotato, espresso nelle coordinate del sistema di riferimento originale:

$$R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

La matrice di rotazione è **ortonormale**: la sua inversa coincide con la sua trasposta,

$$RR^T = I \quad \Rightarrow \quad R^{-1} = R^T$$

Vantaggi:

se abbiamo **rotazioni multiple**, l'effetto combinato si determina facilmente moltiplicando tra loro le matrici di rotazione — sono quindi matematicamente facili da comporre.

Svantaggi:

sono **difficili da interpretare visivamente** — data una matrice di rotazione, è difficile "leggere" a colpo d'occhio di che rotazione si tratti; inoltre **non c'è rappresentazione minimale**: servono 9 numeri per descrivere una rotazione che ha in realtà solo 3 gradi di libertà, quindi la rappresentazione non è efficiente.

Matrice come descrittore vs operatore.

La matrice di rotazione può essere usata in due modi:

- come **descrittore** — R_1^0 rappresenta la rotazione del sistema di riferimento 1 rispetto al sistema 0. Se p^0 e p^1 rappresentano le coordinate dello *stesso* vettore (fisico) nei due sistemi di riferimento, il **cambio di coordinate** è:

$$p^0 = R_1^0\, p^1 \qquad p^1 = (R_1^0)^T p^0 = R_0^1\, p^0$$

- come **operatore** — possiamo moltiplicare un vettore per la matrice di rotazione per ottenere il vettore effettivamente **ruotato**, espresso nello stesso sistema di riferimento:

$$p^0_{rot} = R_1^0\, p^0 \qquad p^0 = (R_1^0)^T p^0_{rot}$$

La differenza concettuale è sottile ma importante: nel primo caso stiamo semplicemente riscrivendo *lo stesso* vettore fisico in un diverso sistema di riferimento; nel secondo caso stiamo effettivamente *ruotando* il vettore, ottenendo un vettore fisico diverso, espresso nello stesso sistema di riferimento di partenza.

2.4.4.3 Coordinate omogenee e trasformazione rigida

Il framework delle matrici permette una rappresentazione compatta sia della rotazione sia della traslazione, ottenendo la cosiddetta **trasformazione di corpo rigido**:

$$p^0 = R_1^0\, p^1 + o_1^0$$

dove si applica prima la rotazione e poi la traslazione. Usando le **coordinate omogenee** — un "trucco" che consiste nell'aggiungere una componente unitaria al vettore, ad esempio $\tilde{p} = [x\ y\ z\ 1]^T$ — si può esprimere l'intera trasformazione roto-traslazionale come un'unica moltiplicazione matriciale:

$$\tilde{p}^0 = T_1^0\,\tilde{p}^1, \qquad T_1^0 = \begin{bmatrix} R_1^0 & o_1^0 \\ 0^T & 1 \end{bmatrix}$$

Questo è un modo compatto per combinare rotazione e traslazione in una **singola** operazione matriciale (moltiplicazione, invece di moltiplicazione + somma separate).

Composizione di trasformazioni.

Combinando diverse rotazioni e traslazioni, moltiplicando le rispettive matrici, si può passare da un sistema di riferimento a un altro attraverso una catena di trasformazioni intermedie:

$$T_3^0 = T_1^0\,T_2^1\,T_3^2 \qquad \text{(intrinsic — ogni trasformazione rispetto al frame corrente)}$$

$$\bar{T}_3^0 = \bar{T}_3^2\,\bar{T}_2^1\,\bar{T}_1^0 \qquad \text{(extrinsic — ogni trasformazione rispetto al frame originale)}$$

Questo approccio è matematicamente comodo da comporre, ma resta difficile da interpretare visivamente e, come già detto, non è una rappresentazione minimale (bisogna memorizzare più parametri del necessario).

2.4.4.4 Attitude angles (angoli di Cardano)

Un'alternativa è descrivere l'orientazione come una **sequenza di rotazioni** attorno agli assi principali, con soli 3 angoli — facile da interpretare, specialmente per angoli piccoli.

Pro:

rappresentazione minimale, facile da interpretare (specialmente per piccoli angoli).

Contro:

soggetta a **singolarità** (rare nei veicoli terrestri), e la composizione di più rotazioni non è immediata come per le matrici.

La convenzione **yaw–roll–pitch** (angoli di Cardano) è quella usata nel corso. Per ogni

rotazione è definita una corrispondente matrice di rotazione:

$$R_z(\psi) = \begin{bmatrix} c_\psi & s_\psi & 0 \\ -s_\psi & c_\psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad R_x(\rho) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_\rho & s_\rho \\ 0 & -s_\rho & c_\rho \end{bmatrix} \quad R_y(\theta) = \begin{bmatrix} c_\theta & 0 & -s_\theta \\ 0 & 1 & 0 \\ s_\theta & 0 & c_\theta \end{bmatrix}$$

dove $c_\psi = \cos(\psi)$, $s_\psi = \sin(\psi)$, e analogamente per gli altri angoli. La rotazione z è lo **yaw**, la rotazione x è il **roll**, la rotazione y è il **pitch**.

Quando si parla di convenzione yaw-roll-pitch **extrinsic**, si intende che ogni rotazione è definita rispetto al **sistema di riferimento originale** (non a quello via via trasformato). La matrice di rotazione complessiva è:

$$R_{xzy}(\psi, \rho, \theta) = R_y(\theta)\, R_x(\rho)\, R_z(\psi)$$

Se vogliamo combinare tutte le rotazioni, dobbiamo procedere **da destra a sinistra** (yaw, poi roll, poi pitch).

Intrinsic vs extrinsic:

- **intrinsic** — ogni trasformazione avviene rispetto al sistema di riferimento *corrente* (già trasformato dalle rotazioni precedenti);
- **extrinsic** — ogni trasformazione avviene rispetto al sistema di riferimento *originale*: è questa la convenzione usata per gli angoli di Cardano.

2.4.4.5 Dal rate angolare misurato alla derivata degli angoli

Un aspetto importante riguarda la relazione tra il **rate angolare** misurato (ad esempio da un giroscopio, che misura la velocità angolare attorno a un dato asse) e la derivata temporale degli angoli di Cardano. È fondamentale realizzare che, se abbiamo uno **yaw rate** di 2 gradi/secondo, *non* possiamo semplicemente integrare quei 2 gradi/secondo per ottenere l'angolo di yaw — perché queste trasformazioni angolari sono tra loro **combinate** (non indipendenti).

Se vogliamo conoscere l'effetto di un dato yaw rate sul nostro sistema di riferimento, dobbiamo prendere lo yaw rate e trasformarlo tenendo conto delle trasformazioni di pitch e roll. Analogamente, se vogliamo considerare il roll rate, dobbiamo tenere conto della sua trasformazione rispetto al pitch. **L'unico angolo che non necessita di trasformazione è il pitch** (nella sequenza scelta, è l'ultimo che viene applicato).

Il **modello del giroscopio** mette in relazione le velocità angolari misurate dal giroscopio (nel sistema di riferimento del corpo) con le derivate degli angoli di Cardano nel sistema di riferimento inerziale:

$$\begin{bmatrix} \dot{\rho} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} c_\theta & 0 & -s_\theta c_\rho \\ t_\rho s_\theta & 1 & -t_\rho c_\theta \\ -s_\theta / c_\rho & 0 & c_\theta / c_\rho \end{bmatrix} \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}$$

dove $\omega_x, \omega_y, \omega_z$ sono le misure del giroscopio nel sistema body-fixed (misurate direttamente sull'oggetto), mentre $\dot{\rho}, \dot{\theta}, \dot{\psi}$ sono le derivate degli angoli di Cardano nel sistema di riferimento inerziale.

2.4.5 Vision — l'image plane

Un'immagine è una **funzione di due variabili**, descritta come una matrice:

$$I = f(x_i, y_i)$$

Nella maggior parte dei casi, il piano immagine è **discretizzato** (in pixel); I può essere uno scalare (immagini bianco/nero) oppure un vettore a 3 componenti — RGB — nel caso di immagini a colori, dove le tre componenti dipendono dal canale di colore utilizzato.

Quando si studia una telecamera, l'obiettivo è studiare un punto che, nel mondo reale, si trova nello spazio 3D. Servono quindi **tre** sistemi di riferimento:

- il **World Reference Frame** — fisso, dove capiamo dove si trovano le cose;
- il **Camera Reference Frame** — si muove insieme alla telecamera (camera-fixed); ha generalmente l'asse Z_c allineato con la direzione di puntamento della telecamera;
- l'**Image Plane** — un terzo sistema di riferimento, dove risiede il sensore della telecamera; è un sistema di riferimento 2D dove l'immagine viene formata e generata.

2.4.6 Il modello pinhole *

Il **modello pinhole** assume che la telecamera abbia un'**apertura infinitamente piccola**. Il modo in cui si modella la telecamera consiste nel considerarla come una sorta di scatola, con un piccolo foro su un lato e l'oggetto nel mondo reale dall'altro. In pratica: matematicamente è conveniente rappresentare il foro come se fosse spostato "davanti" all'immagine (invece che dietro, come sarebbe fisicamente corretto per evitare l'immagine capovolta), poiché è un modello più comodo da trattare.

La **risoluzione** è quanto è densa la griglia del piano immagine: è un parametro **intrinseco** della telecamera.

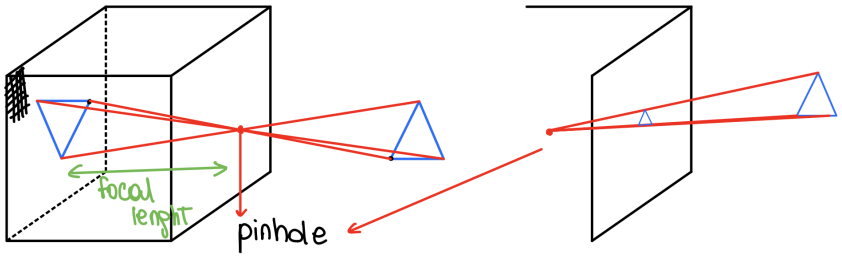


Figure 2.1: Pinhole model

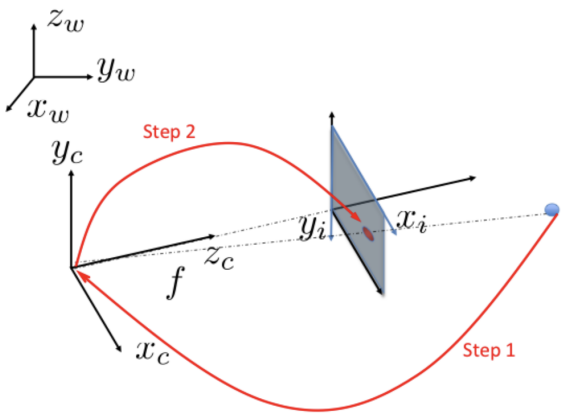
2.4.6.1 Estrinseci e intrinseci

- i **parametri estrinseci** $[R|t]$ rappresentano dove si trova la telecamera nel mondo — cioè la posizione della telecamera, ed è la roto-traslazione tra il riferimento mondo e il riferimento telecamera;
- i **parametri intrinseci** riguardano come è costruita la telecamera: secondo il modello pinhole, il parametro più importante è la **lunghezza focale**.

Definizione (lunghezza focale): è la distanza tra il centro del sistema di riferimento della telecamera e il piano dove l'immagine si forma.

Il nostro obiettivo finale è ottenere le coordinate nel World Reference Frame una volta note le coordinate nel piano immagine (dai sensori vogliamo ricostruire ciò che ci circonda). Per farlo, si risolve prima il **problema di formazione dell'immagine**, che consiste nel calcolare O_{image} a partire da O_{world}. Questo viene fatto tramite due step:

- da coordinate O_{world} a coordinate O_{camera}
- da coordinate O_{camera} a coordinate O_{image}



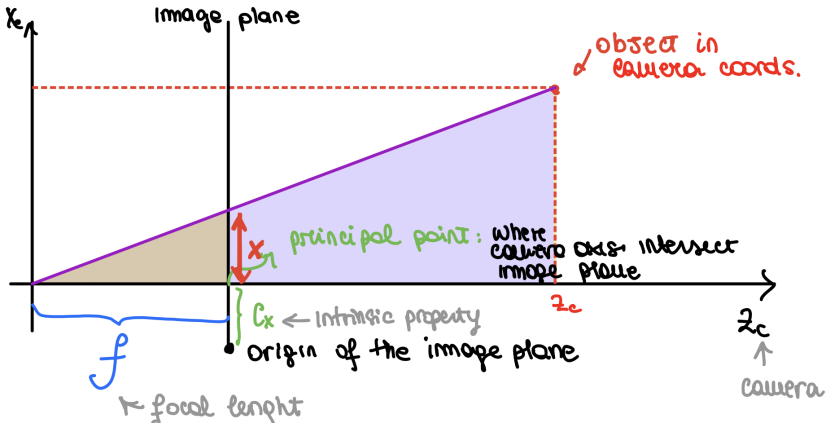


Figure 2.2: From Camera ref. to Image ref.

2.4.6.2 Primo step: da coordinate mondo a coordinate camera

$$O_{camera} = [R|t]\, O_{world} = T\, O_{world}$$

In coordinate omogenee:

$$\begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}$$

questa è semplicemente la trasformazione di corpo rigido già vista in precedenza.

2.4.6.3 Secondo step: da coordinate camera a coordinate immagine

Sfruttando la **similitudine di triangoli** (dato che l'oggetto è già stato trasformato in coordinate camera), si ottiene:

$$\frac{x}{x_c} = \frac{f}{z_c} \implies x = \frac{f}{z_c} \cdot x_c$$

Now, to obtain x_i it is necessary to know the **intrinsic property** of the camera c_x, so to obtain $x_i = x + c_x$

$$x_i = f\,\frac{x_c}{z_c} + c_x \qquad y_i = f\,\frac{y_c}{z_c} + c_y$$

dove c_x, c_y è la posizione del **principal point** (l'origine del piano immagine, cioè il punto dove l'asse ottico della telecamera interseca il piano immagine).

Questa è una relazione **non lineare**, a causa della dipendenza non lineare tra input e

output (la divisione per z_c).

2.4.6.4 Il trucco delle coordinate omogenee

Per rendere questa proiezione trattabile in forma lineare, si usa nuovamente il trucco delle coordinate omogenee, introducendo un nuovo set di variabili:

$$\begin{bmatrix} x'_i \\ y'_i \\ z'_i \end{bmatrix} = \underbrace{\begin{bmatrix} f & 0 & c_x & 0 \\ 0 & f & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}}_{K} \begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix}$$

dove K è la matrice dei parametri intrinseci. Il risultato $[x'_i, y'_i, z'_i]^T$ **non è omogeneo**: per tornare a coordinate immagine "euclidee" (cioè le coordinate pixel effettive), si divide per il terzo termine, a forzare l'omogeneità:

$$O_{image} = \frac{1}{z'_i} \begin{bmatrix} x'_i \\ y'_i \\ z'_i \end{bmatrix}$$

In questo modo si sta forzando una **struttura lineare** sulla geometria di proiezione.

Svantaggio dell'approccio.

Se prendiamo il risultato ottenuto e lo moltiplichiamo per un valore qualsiasi (uno scalare s), il risultato finale in termini di x_i, y_i (dopo la divisione per il terzo termine per forzare l'omogeneità) resta lo stesso, indipendentemente dal valore dello scalare. Combinando i due step, si ottiene la formula compatta:

$$s\, O_{image} = \underbrace{\begin{bmatrix} f & 0 & c_x & 0 \\ 0 & f & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}}_{\text{3x4, parametri intrinseci}} \underbrace{\begin{bmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{\text{4x4, parametri estrinseci}} O_{world}$$

alla fine si ottiene un risultato **a meno di un fattore di scala**: conosciamo l'immagine, ma solo a meno di questo scalare — è questo il prezzo da pagare per la comodità di scrivere l'equazione in questa forma compatta e lineare.

Definizione: il processo con cui si determinano i valori dei parametri intrinseci ed es-

trinseci si chiama **camera calibration**.

2.4.7 Telecamere reali: la distorsione

Il modello pinhole assume un foro perfetto. Nelle telecamere reali, la luce viene invece focalizzata da lenti, che introducono **distorsione**:

- **barrel distortion** — una linea che nella realtà è dritta appare **concava** nell'immagine (come se l'immagine venisse "pizzicata" verso il centro);
- **pincushion distortion** — tende a deformare l'immagine nella direzione opposta, come se venisse "tirata" dai bordi.

2.4.7.1 Modello di distorsione radiale

Il modello standard di distorsione radiale (usato ad esempio in OpenCV e Matlab) è una trasformazione dalle coordinate immagine ideali (non distorte) a quelle reali (distorte):

$$\begin{bmatrix} x_i^d \\ y_i^d \end{bmatrix} = (1 + k_1 r^2) \begin{bmatrix} x_i - c_x \\ y_i - c_y \end{bmatrix} + \begin{bmatrix} c_x \\ c_y \end{bmatrix}$$

dove

$$r^2 = (x_i - c_x)^2 + (y_i - c_y)^2$$

Un modello più avanzato, che tiene conto anche di una componente **tangenziale** della distorsione, è:

$$\begin{bmatrix} x_i^d \\ y_i^d \end{bmatrix} = (1+k_1 r^2+k_2 r^4+k_3 r^6) \begin{bmatrix} x_i - c_x \\ y_i - c_y \end{bmatrix} + \begin{bmatrix} 2k_4(x_i - c_x)(y_i - c_y) + k_5(r^2 + 2(x_i - c_x)^2) \\ 2k_5(x_i - c_x)(y_i - c_y) + k_4(r^2 + 2(y_i - c_y)^2) \end{bmatrix} + \begin{bmatrix} c_x \\ c_y \end{bmatrix}$$

Questi modelli servono a **compensare** la distorsione: dopo la correzione dell'immagine, va notato che alcuni punti dell'immagine corretta non corrispondono più a punti presenti nell'immagine originale.

2.4.7.2 Calibrazione tramite scacchiere

Per determinare lunghezza focale e coordinate del principal point si può lavorare su un sistema **lineare**: se conosciamo O_{world} e $s\,O_{image}$, si tratta di trovare la relazione tra i due valori, usando immagini del mondo reale — tipicamente tramite **scacchiere** (checkboards).

Input: un insieme di immagini che rappresentano feature con proprietà geometriche note

(ad esempio scacchiere, di cui conosciamo la dimensione fisica dei quadretti).

Facendo corrispondere pixel e posizione geometrica nota di N feature (i vertici della scacchiera), si definisce un sistema di equazioni lineari, risolto ai minimi quadrati. Ogni corrispondenza punto-pixel genera **2 equazioni e 12 incognite** (che sono f, c_x, c_y, $t_1, \ldots, t_3$ e i termini della matrice R — anche se in realtà la rotazione ha solo 3 gradi di libertà, nel sistema di equazioni si trattano come 9 incognite separate, per un totale di 12). Per questo si cerca di raccogliere **tanti punti quante più equazioni servono** a determinare univocamente tutte le incognite.

Output: le matrici dei parametri intrinseci ed estrinseci.

2.4.8 Il problema inverso

Il problema **inverso** consiste nel voler ricostruire le coordinate nel mondo a partire dalle coordinate nell'immagine. Sfortunatamente **non è possibile risolverlo direttamente**: nella relazione

$$s\, O_{image} = \underbrace{K}_{3\times4}\ \underbrace{[R|t]}_{4\times4}\ O_{world}$$

a sinistra abbiamo un vettore (3×1), ottenuto come prodotto di una matrice (3×4) per un vettore (4×1): questa relazione (combinata, $3 \times 4 \cdot 4 \times 4$) **non è invertibile**. C'è una perdita di informazione: tutti i punti nel sistema di riferimento della telecamera che giacciono sulla stessa **linea di vista** (ray) vengono proiettati sullo **stesso punto** nell'immagine — quindi, conoscendo solo la posizione di un punto nel piano immagine, non sappiamo dove si trovi esattamente lungo quella linea nel mondo reale.

2.5 Stereo Vision *

2.5.1 L'idea di base

Per risolvere il problema inverso, l'idea è quella di avere un'informazione **aggiuntiva**: invece di usare una sola immagine, se ne usano **due**, provenienti da due telecamere diverse. Serve conoscere la **posizione relativa** delle due telecamere — questo è il concetto alla base della **stereovisione** (stereo vision), che permette di costruire qualcosa che assomiglia a una point cloud, fornendo informazioni sull'ambiente 3D, sulla distanza e sulla posizione degli oggetti.

Con una singola telecamera possiamo ottenere una vista del mondo attorno a noi, condensando molte informazioni su forma e colore, ma non è facile estrarre informazione

spaziale (di profondità) da una sola immagine.

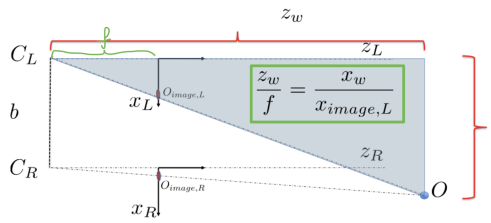
2.5.2 Assunzioni tipiche delle stereo camere

- conosciamo la roto-traslazione $[R|t]$ tra le due telecamere;
- le due telecamere hanno **assi ottici perfettamente allineati e paralleli** ($Z_L \parallel Z_R$);
- le due telecamere sono semplicemente **traslate** lungo l'asse x di una quantità b, detta **baseline**;
- le due telecamere sono **identiche** (stessi parametri intrinseci).

Il primo assunto aggiuntivo, che verrà poi rimosso, è che **sappiamo associare** ogni pixel nelle due immagini a un dato oggetto: un punto fisico dell'oggetto genera due pixel, uno per immagine, che stiamo osservando.

2.5.3 Derivazione geometrica: triangoli simili *

Consideriamo il problema in vista dall'alto (bird's eye view). Se consideriamo tutto rispetto alla **telecamera sinistra**, la posizione dell'oggetto è caratterizzata da z_w (profondità) e x_w (offset laterale). Le variabili note sono la lunghezza focale f e la baseline b; le incognite sono proprio z_w e x_w.



Dall'immagine sinistra, per similitudine dei triangoli:

$$\frac{z_w}{f} = \frac{x_w}{x_{image,L}}$$

Si ottiene quindi un'analoga relazione di similitudine, tenendo conto dell'offset dato dalla baseline:

$$\frac{z_w}{f} = \frac{x_w - b}{x_{image,R}}$$

Combinando le due equazioni:

$$z_w\, x_{image,L} = x_w f \qquad z_w\, x_{image,R} = x_w f - fb$$

sottraendo membro a membro:

$$z_w\, x_{image,R} = z_w\, x_{image,L} - fb$$

2.5.4 Definizione di disparità

Si definisce la **disparità** come la differenza tra le coordinate del punto nelle due immagini:

$$d = x_{image,L} - x_{image,R}$$

Sostituendo, si ottiene infine:

$$z_w = \frac{fb}{d}$$

e, riprendendo le equazioni di similitudine:

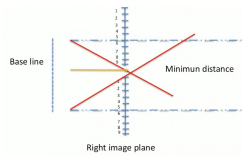
$$x_w = \frac{z_w\, x_{image,L}}{f} \qquad y_w = \frac{z_w\, y_{image,L}}{f}$$

Abbiamo così **risolto il problema**: dalle due immagini calcoliamo la disparità, e da questa — combinata con i parametri intrinseci — ricaviamo la distanza dell'oggetto, ricostruendo così le coordinate 3D del punto (rispetto al sistema di riferimento della telecamera sinistra).

2.5.5 Considerazioni pratiche su disparità, risoluzione e baseline

- Le telecamere hanno una **risoluzione fissata**, il che impone un limite inferiore alla disparità misurabile (la disparità minima rilevabile è 1 pixel, in conseguenza della quantizzazione).
- Il **range effettivo** della stereovisione è determinato dalla baseline.
- Una baseline (e una lunghezza focale) più **grande** permette di stimare la posizione di oggetti **più lontani**.
- Una baseline (e una lunghezza focale) più **grande** rende però più **difficile rilevare oggetti vicini**.

2.5.5.1 Baseline e distanza minima



L'oggetto deve essere visto da **entrambe** le telecamere per poter calcolare la disparità: esiste quindi una **distanza minima** determinata dalla geometria del sistema. Se rendiamo la baseline **più piccola**, avviciniamo le due telecamere tra loro, quindi la distanza minima si riduce. La baseline ha un impatto **diretto** sulla distanza minima rilevabile.

2.5.5.2 Risoluzione e risoluzione spaziale

Se guardiamo alla geometria del problema, tutti i punti che ricadono in una certa regione (nell'immagine, geometricamente rappresentabile come un piccolo poligono all'intersezione dei raggi visivi) appariranno associati allo **stesso pixel**, il che significa che questi punti avranno la **stessa disparità** nel sistema stereo. La risoluzione dell'immagine, insieme a baseline e lunghezza focale, ha quindi un impatto sulla **risoluzione della ricostruzione** della posizione. Se rendiamo ogni pixel più piccolo (aumentando la risoluzione), l'angolo che determina l'estensione spaziale di questi punti si restringe: se l'angolo si restringe, l'intersezione risultante è più piccola. Quindi la risoluzione ha un impatto diretto sulla **risoluzione spaziale** della ricostruzione.

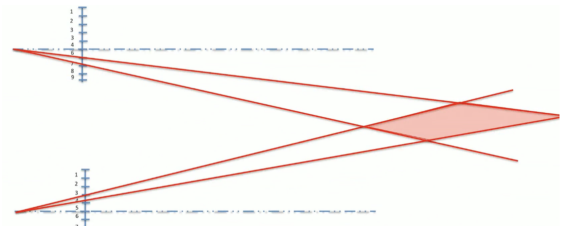


Figure 2.3: Low Resolution

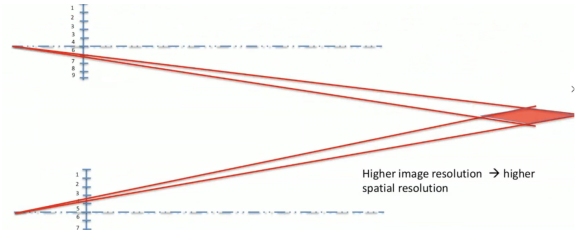


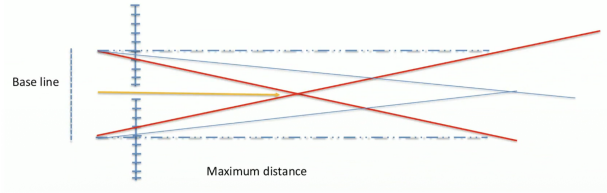
Figure 2.4: High Resolution

2.5.5.3 Baseline e range massimo

Per rilevare qualcosa, l'oggetto deve essere visto da entrambe le telecamere in modo da poter calcolare la disparità. Più siamo vicini all'asse ottico, più continuiamo a ricadere nello stesso pixel. Le vie per aumentare il range massimo sono:

- **aumentare la risoluzione** (rendendo l'angolo più piccolo);
- **cambiare la baseline**, rendendola più grande, aumentando così il range massimo.

Aumentare la risoluzione è **sempre benefico**; se invece si modifica la proprietà geometrica (baseline), si crea un compromesso: aumentare la baseline aumenta il range massimo, ma aumenta anche la distanza minima.



Trade-off riassuntivo:

- guardare **più lontano** → allontanare le telecamere (baseline maggiore);

- guardare **più vicino** → avvicinare le telecamere (baseline minore).

2.5.5.4 Effetto della lunghezza focale

Anche la distanza massima è influenzata dalla **lunghezza focale**: aumentare la lunghezza focale aumenta il range massimo. Tuttavia, aumentare la lunghezza focale della telecamera causa anche uno **zoom**: si restringe il campo visivo (field of view) — possiamo vedere oggetti più lontani, ma con un campo visivo più piccolo.

In sintesi, le telecamere stereo sono capaci di ricostruire l'ambiente 3D, ma il trade-off coinvolto è un po' più complesso rispetto a quello dei LiDAR. I LiDAR sono sensori più costosi e **attivi**, ma il dato che forniscono è più facile da elaborare per progettare sistemi di localizzazione; le telecamere hanno alcuni vantaggi (costo, informazione di appearance) ma un trade-off leggermente più complesso.

2.5.6 Il problema della corrispondenza (correspondence problem)*

Per calcolare la disparità di un punto rosso, dobbiamo sapere **dove** il corrispondente oggetto fisico visto nell'immagine sinistra si trovi nell'immagine destra — e questo non è affatto banale. Serve un modo per risolvere il cosiddetto **correspondence problem**: determinare quale pixel nell'immagine destra corrisponde a ciascun pixel nell'immagine sinistra.

Idea generale: confrontare una piccola "immagine" (template) di ciò che si vuole trovare con tutte le regioni locali dell'altra immagine.

Esistono due macro-famiglie di metodi:

- **Correlation based algorithms** — cercano regioni **simili** nell'immagine, in modo largamente indipendente dall'applicazione;
- **Feature based algorithms** — cercano cose **specifiche**, quindi sono metodi molto dipendenti dall'applicazione (la feature da cercare dipende dal contesto).

2.5.6.1 Correlation based algorithms

Non si cerca esattamente lo "stesso oggetto" in senso semantico, ma si valuta quanto due immagini (patch) siano **simili**. Gli elementi da confrontare sono **patch di dimensione**

fissa dell'immagine.

Ogni singolo pixel fornisce poco contenuto informativo (essenzialmente solo l'intensità luminosa); bisogna quindi guardare a un'**area**, cioè a un insieme di pixel. Data una patch (template) nell'immagine sinistra, si cerca il corrispondente nell'immagine destra: si sceglie prima un insieme di patch **candidate**, poi si decide quale di queste corrisponda meglio al template originale.

Appearance similarity function.

Definiamo (lavorando in bianco e nero, dove l'immagine è una funzione bidimensionale) $f(i,j)$ il pixel (i,j) nell'immagine destra, $g(i,j)$ il pixel corrispondente nell'immagine sinistra, e R il template. Esistono diverse funzioni per misurare la similarità tra due template:

1. **Massimo dell'errore assoluto:**

$$\max_{[i,j]\in R} |f(i,j) - g(i,j)|$$

   Tuttavia ci sono alcuni svantaggi:

   - il **valore assoluto** non è differenziabile — se si usano tecniche basate sul gradiente, si rischia di "saltare" da un lato all'altro senza convergere;
   - **grande sensibilità al rumore**: nel massimo c'è un problema di discontinuità — se anche un singolo pixel viene colpito da rumore (ad esempio un raggio gamma), quel singolo pixel può far apparire le due immagini come non simili.

2. **Somma dell'errore assoluto:**

$$\sum_{[i,j]\in R} |f(i,j) - g(i,j)|$$

3. **SSD** (Sum of Square Differences):

$$SSD = \sum_{[i,j]\in R} (f(i,j) - g(i,j))^2$$

   Invece del valore assoluto si può usare il **quadrato** — una parabola: il minimo si trova esattamente nella stessa posizione dell'errore assoluto, ma la funzione è **differenziabile**, il che la rende un'opzione migliore rispetto alla somma dei valori assoluti.

Tuttavia abbiamo alcuni **svantaggi**. Se stiamo guardando lo stesso oggetto, ma in alcuni casi abbiamo immagini molto **luminose** e in altri casi molto **scure** (sottoesposte, o in un ambiente buio), il valore complessivo di intensità è piccolo, e quindi la differenza tra un buon match e un match non buono risulta **poco marcata**: l'SSD **non si adatta bene** (non scala bene) all'intensità complessiva dell'immagine.

4. **NCC** (Normalized Correlation Coefficient):

$$NCC_{fg} = \frac{\sum_{[i,j]\in R} f(i,j)g(i,j)}{\sqrt{\sum_{[i,j]\in R} f^2(i,j) \sum_{[i,j]\in R} g^2(i,j)}}$$

La soluzione è la **normalizzazione**, tramite il coefficiente di correlazione normalizzato: il numeratore è la somma dei prodotti (più due immagini sono simili, più alto sarà il numeratore), il denominatore normalizza rispetto all'**autocorrelazione** di ciascuna immagine. Calcolando l'NCC si ottiene un singolo numero: abbiamo così un modo, dato un template, di trovare la posizione che meglio corrisponde a quel template in un'altra immagine.

Uno dei modi più semplici per definire quanto due template siano simili è confrontarli pixel per pixel: per fornire un unico valore che descriva la **dissimilarità**, si può usare il **massimo** della differenza (in valore assoluto, dato che la differenza di intensità è l'opposto della similarità, e valori più piccoli sono migliori).

Ricerca esaustiva.

Bisogna definire un template, poi cercare quel template nell'immagine destra scandendo l'intera immagine; una volta trovata la massima correlazione, si conosce la disparità di quel punto. Il passo successivo è definire un altro template (spostato di un pixel a destra) e ripetere la ricerca del match — questo approccio è detto **ricerca esaustiva** (exhaustive search).

2.5.7 Il vincolo epipolare

La ricerca esaustiva più semplice, ma anche la meno efficiente, richiede in linea di principio di cercare in tutta l'immagine 2D destra un punto corrispondente. In realtà, per lo **stereo matching**, non è necessario cercare in tutta l'immagine 2D: il **vincolo epipolare** (epipolar constraint) riduce lo spazio di ricerca a una **linea monodimensionale**.

2.5.7.1 Perché esiste il vincolo epipolare

Stiamo lavorando con 2 telecamere, 2 centri ottici diversi, 2 immagini diverse — ma chiaramente, se guardiamo lungo l'**asse ottico** di una telecamera, il punto corrispondente su quell'asse potrebbe trovarsi ovunque nell'altra telecamera. I possibili candidati giacciono su una **linea retta**: il punto p nell'immagine sinistra è generato da un punto lungo una retta chiamata **epipolar line**.

- i centri delle due telecamere C_l, C_r e il punto immagine p determinano il cosiddetto **piano epipolare** (epipolar plane);
- le intersezioni del piano epipolare con i due piani immagine sono le **linee epipolari** (epipolar lines);
- i punti corrispondenti devono quindi giacere lungo le linee epipolari: questo vincolo si chiama, appunto, **epipolar constraint**;
- questo vincolo riduce il problema di corrispondenza a una ricerca **monodimensionale** lungo la linea epipolare.

Il potenziale match per un punto p deve giacere sulla corrispondente linea epipolare l'. La linea epipolare è la proiezione di un raggio "retro-proiettato" $\pi^{-1}(p)$ sull'altra immagine; l'**epipolo** è la proiezione del centro ottico di una telecamera sull'immagine dell'altra — un sistema stereo ha due epipoli.

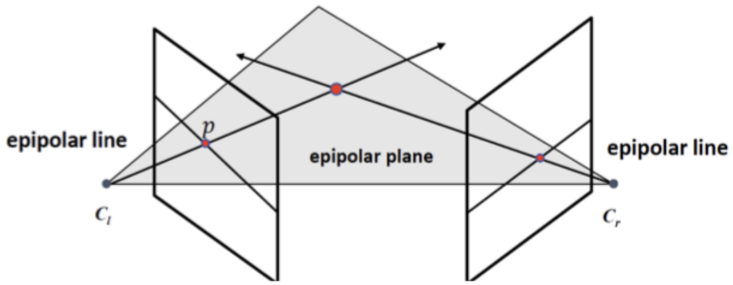


Figure 2.5: Epipolar Constraint

2.5.7.2 Rettificazione: far coincidere l'epipolar constraint con lo scanline

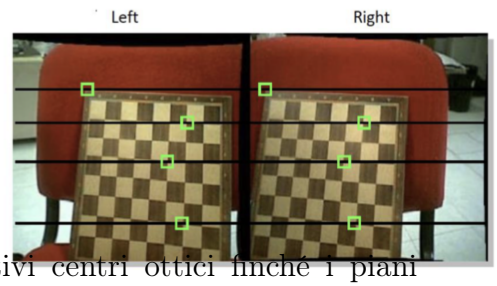
Se possiamo fare in modo che il vincolo epipolare corrisponda a una **scanline** (una riga dell'immagine), tutto diventa molto più semplice. Questo si ottiene se le due telecamere hanno:

- gli **stessi parametri intrinseci** (stessa lente, stessa lunghezza focale, stessa dimensione del piano immagine);

- assi ottici **perfettamente allineati**;
- la **stessa posizione verticale** del centro immagine;
- una **sola traslazione** verso destra della telecamera (cioè solo la baseline lungo x, nessun altro offset).

Lavorando la geometria di proiezione con queste condizioni, il vincolo epipolare corrisponde esattamente alla **scanline**.

In pratica, anche nelle stereo camere commerciali di fascia alta, le immagini sinistra e destra non sono *mai* perfettamente allineate. Per questo, in pratica, conviene forzare che le **scanline** coincidano con le linee epipolari, perché così la ricerca della corrispondenza può essere resa molto efficiente (si cerca il punto solo lungo la stessa riga).



Il processo che realizza questo si chiama **rettificazione stereo** (stereo rectification): deforma (warp) le immagini sinistra e destra in nuove immagini "rettificate" tali che le linee epipolari coincidano con le scanline. Concettualmente, si definiscono due **omografie** — matrici di proiezione geometrica identiche alle Perspective Projection Matrices originali, ma ruotate attorno ai rispettivi centri ottici finché i piani immagine non diventano paralleli tra loro. Questo garantisce che gli epipoli si trovino all'infinito, quindi le linee epipolari risultano parallele. Per avere linee epipolari orizzontali, la baseline deve essere parallela al nuovo asse X di entrambe le telecamere; inoltre, per una corretta rettificazione, i punti corrispondenti devono avere la **stessa coordinata verticale** — ottenuta richiedendo che le nuove telecamere abbiano gli stessi parametri intrinseci. Va notato che, avendo la stessa lunghezza focale, anche i nuovi piani immagine risultano complanari.

Per costruire queste omografie serve una **calibrazione**, effettuata con lo stesso tipo di struttura a scacchiera vista in precedenza, in modo da poter imporre il vincolo epipolare.

2.5.8 Procedura completa per l'uso di immagini stereo

1. **Rettificare** la coppia stereo (se non già rettificata) per far coincidere le linee epipolari con le scanline;
2. per ogni pixel nell'immagine sinistra, trovare il corrispondente punto nell'immagine destra lungo la **stessa scanline**;
3. calcolare la **disparità** per ogni coppia di corrispondenze trovata;

4. visualizzarla come immagine in scala di grigi o a colori: la **disparity map** (che può essere letta come una mappa di profondità — oggetti vicini presentano disparità maggiore, e quindi appaiono più luminosi nella disparity map).

2.5.9 Sfide e limitazioni della stereo vision

Oltre alle sfide relative alla telecamera già discusse, la stereo vision presenta ulteriori sfide specifiche:

- **Occlusione** — abbiamo lavorato sull'ipotesi che la scena sia identica da sinistra a destra; in realtà, da prospettive diverse possiamo vedere porzioni diverse dello stesso oggetto (o oggetti nascosti in un'immagine ma non nell'altra) — questo genera dei "buchi" (hole) nella disparity map.
- **Pattern ripetitivi** — se in un'immagine è presente un pattern ripetuto, molti template risulteranno simili tra loro, generando **incertezza** nella corrispondenza.
- **Mancanza di texture** — l'algoritmo di correlazione si basa su texture e variazioni di intensità; guardando una superficie priva di feature, il problema di corrispondenza risulta **indefinito**: se il match potrebbe trovarsi ovunque, l'algoritmo può "bloccarsi" e la profondità stimata continua a variare senza convergere. Il cielo, ad esempio, cambia molto in queste ricostruzioni: le nuvole sono lontane, la loro luminosità cambia continuamente, sono trasparenti (quindi non ben rilevate dal sistema di stereovisione), e osservare una superficie priva di feature rende il problema di corrispondenza indefinito.
- **Superfici non-Lambertiane** — sono superfici che non riflettono la luce nello stesso modo in tutte le direzioni (si pensi a uno specchio): da prospettive diverse si vedono cose diverse. Per superfici iridescenti, addirittura, si può osservare un colore diverso da prospettive diverse.
- **Dimensione del template** — non si può usare un'unica dimensione ottimale in ogni situazione:
  - **template piccoli**: sono più sensibili — spostando anche di un solo pixel un template 3×3, si cambia gran parte del contenuto del template. È più capace di individuare precisamente dove si trovi il massimo, ma è soggetto a **molto rumore**.
  - **template grandi**: la correlazione può cambiare solo perché si rimuove un pixel a sinistra e se ne aggiunge uno a destra — una variazione piccola rispetto

al resto del template; l'effetto è quindi "sovrastato" dal resto dei punti. Usare un template molto grande riduce la **risoluzione di profondità** — diventa difficile individuare esattamente dove si trovi il massimo della correlazione.

Un ulteriore aspetto pratico è la **sincronizzazione**: una delle nostre assunzioni è che le due immagini siano state acquisite esattamente nello stesso istante. Se si costruisce un sistema stereo a partire da telecamere USB generiche, queste tipicamente *non* sono sincronizzate tra loro; si può in parte compensare via software, ma la disparità calcolata può risultare non pienamente rappresentativa della profondità reale. Solitamente, quando si lavora con telecamere stereo professionali, non si prendono due telecamere generiche e le si posiziona a piacere, ma si costruisce un vero e proprio **stereo rig**, garantendo che la risoluzione sia la stessa per entrambe.

Assunzioni tipiche degli algoritmi stereo:

- **No occlusioni** — alcuni punti sono visibili in un'immagine ma non nell'altra;
- **Unicità** — un punto in un'immagine ha al più un punto corrispondente nell'altra (l'assunzione fallisce con oggetti trasparenti);
- **Continuità** — la disparità è a tratti (piecewise) uniforme.

Quella qui presentata è l'idea di base; esistono numerosi raffinamenti e metodi più avanzati che sfruttano lo stesso principio fondamentale.

2.6 Model Based Image Processing

Ci sono due approcci principali per estrarre informazione speciale (feature-specific) dalle immagini:

- **model based image processing**;
- **neural network image processing**.

L'idea del **model based image processing** è cercare cose **specifiche** nelle immagini, per le quali possiamo fornire una descrizione matematica, e cercare quegli elementi specifici nelle immagini stesse (queste algoritmi cercano feature specifiche). Esempi tipici: rilevamento di codici QR, rilevamento delle corsie (lane detection), rilevamento della strada (road detection), rilevamento di pali (pole detection).

2.6.1 Lane detection

Una delle applicazioni più comuni di questo tipo di elaborazione è la **lane detection**. Se guardiamo al manuale che descrive come costruire una strada, sappiamo la **definizione** di corsia (lane): *"a division of a road marked off with painted lines and intended to separate single lines of traffic according to speed or direction"*.

Le linee dipinte hanno due caratteristiche principali:

- il loro **colore** è diverso da quello della strada;
- appaiono **dritte** (o quasi dritte).

Un algoritmo può sfruttare proprio queste due feature. Il nostro compito è rilevare queste linee dipinte, che sono: dipinte (quindi hanno colore diverso) e appaiono per lo più dritte. L'idea è sviluppare un algoritmo capace di rilevare queste due feature.

2.6.1.1 Pipeline della lane detection

A partire dall'immagine, la pipeline tipica è:

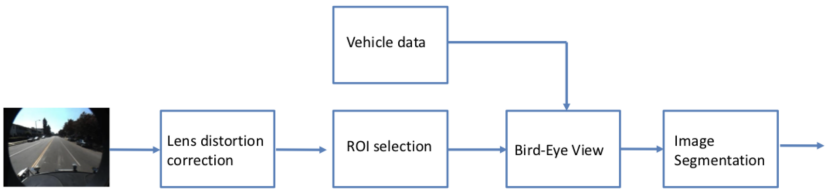


Figure 2.6: Model-based Image Processing

La ROI selection sfrutta anche i **dati del veicolo** (vehicle data) come input aggiuntivo.

Step 1: ROI selection.

ROI sta per *Region of Interest*: si seleziona la parte dell'immagine dove ci aspettiamo che si trovi la strada. Possiamo suddividere l'immagine in una regione di pixel dove *non* vale la pena guardare, escludendola dall'elaborazione successiva. Si usano informazioni qualitative note sulle proprietà della telecamera: campo visivo (field of view), posizione e orientazione.

Step 2: Bird Eye View.

L'obiettivo è ricostruire informazione spaziale a partire dall'immagine — riproiettare la vista prospettica della telecamera in una vista dall'alto (bird's eye view). Riprendendo

l'equazione di proiezione generale:

$$s\,O_{image} = \begin{bmatrix} f & 0 & c_x & 0 \\ 0 & f & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \\ 0 & 0 & 0 & 1 \end{bmatrix} O_{world}$$

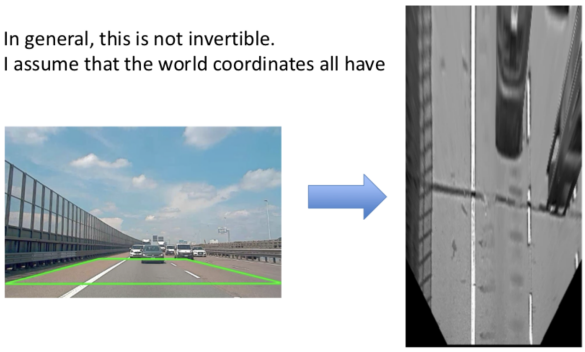
Dato che sappiamo di star cercando le corsie, e le corsie sono per definizione sulla strada, possiamo assumere che la coordinata z nel mondo sia $z_{world} = 0$ (livello del suolo). Con questa assunzione aggiuntiva, la matrice diventa **invertibile**, e possiamo risolvere per x e y (posizione sul piano stradale), noto il punto immagine, ottenendo tipicamente un sistema con **3 equazioni e 3 incognite** (x, y e il fattore di scala s).

Considerazioni sulla bird-eye view.

- Le corsie sono **parallele**: questo ha senso, dato che nella realtà lo sono — nell'immagine originale (prospettica) non appaiono parallele, e la loro "apertura" cambia lungo l'immagine, ma dopo la trasformazione in bird-eye view devono risultare effettivamente parallele.

Svantaggi della bird-eye view.

1. Compaiono degli strani "blob" deformati, corrispondenti tipicamente ad altri veicoli: dato che stiamo forzando l'assunzione che tutti i pixel si trovino a livello del suolo, tutto ciò che *non* è effettivamente a livello del suolo (come un'altra auto) risulta **deformato** nella trasformazione.
2. Guardando vicino alla base dell'immagine, la ricostruzione risulta più **nitida**: l'immagine ha una risoluzione costante nel piano immagine, ma qui stiamo proiettando nel mondo reale — più siamo lontani, più larga è l'area di terreno che viene "compressa" in un singolo pixel dell'immagine originale, per via degli angoli di vista.



Ci stiamo affidando al fatto di **conoscere dove si trova il terreno** rispetto alla telecamera. C'è un aspetto importante da considerare: l'**orientazione** della telecamera cambia durante l'utilizzo normale del veicolo. La matrice di rotazione che assumiamo nota non è in realtà sempre la stessa: il veicolo tende a **beccheggiare** (pitch) e **rollare** (roll) in base all'accelerazione — quando freniamo, il veicolo tende a "abbassare il muso" (pitch down), quando accelera tende a "sollevarlo" (pitch up). Approssimativamente:

$$\varphi \approx -\frac{Mh}{K^s L_\varphi^2}\, a_y = \gamma_\varphi\, a_y \qquad \theta \approx -\frac{Mh}{K^s L_\theta^2}\, a_x = \gamma_\theta\, a_x$$

dove φ è l'angolo di roll (quanto rolla l'auto) e θ è l'angolo di pitch (quanto beccheggia). Questa **calibrazione dinamica** è importante, perché se non teniamo conto di questi cambiamenti di orientazione, la matrice di rotazione usata per la trasformazione in bird-eye view non sarà più corretta.

Step 3: Image Segmentation.

L'image segmentation si basa sul fatto che il colore delle linee è diverso da quello della strada.

Prima idea: se dobbiamo cercare qualcosa che cambia colore, perché non guardare direttamente all'**intensità**? L'idea è cercare un valore di intensità specifico.

Svantaggio: troppa variabilità dovuta alle condizioni ambientali — a seconda delle condizioni di illuminazione, l'immagine può risultare più chiara o più scura, rendendo poco affidabile la ricerca per un valore assoluto di intensità.

Seconda idea: cercare la **variazione** di intensità, non un valore dato, ma un **cambiamento** di intensità. A seconda di quanto sia luminosa la giornata, si può avere una soglia (threshold) diversa, in modo che tutto ciò che sta sopra sia considerato "chiaro" e tutto ciò che sta sotto sia considerato "scuro". Applicare la soglia sul **gradiente** dell'intensità, invece che sull'intensità stessa, rende tutto più robusto.

2.6.1.2 Pipeline dettagliata della segmentazione

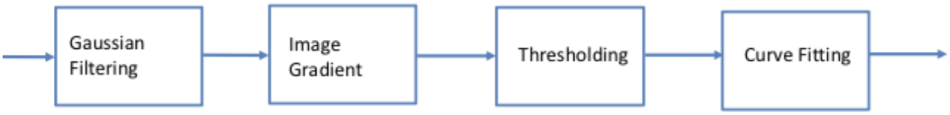


Figure 2.7: Image Segmentation Pipeline

Gaussian filtering.

Lavorando su $\mathbb{R}^2$ (immagine come funzione di due variabili), estendiamo il concetto di filtro (visto per funzioni a una variabile — high pass filtering) a due variabili. Si genera un'immagine in cui il valore di ciascun pixel è la **media pesata** dei pixel vicini (nel template), usando uno schema gaussiano: molto peso al punto centrale, peso che decresce esponenzialmente per gli altri punti. Il filtraggio si realizza **convolvendo** l'immagine con un template gaussiano:

$$I_f(x,y) = \frac{4}{9}I(x,y) + \frac{1}{9}I(x-1,y-1) + \frac{1}{9}I(x-1,y) + \frac{1}{9}I(x-1,y+1) + \dots$$

(pesi puramente indicativi dell'idea, l'importante è la struttura: un nucleo gaussiano che dà più peso ai pixel centrali). La media gaussiana è meno soggetta a picchi (spike) dovuti al rumore rispetto a una media uniforme.

Image Gradient.

Il calcolo del gradiente si effettua convolvendo l'immagine con due template (uno per ciascuna direzione):

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \qquad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Il gradiente complessivo si calcola come:

$$G = \sqrt{G_x^2 + G_y^2}$$

G_y è il gradiente lungo l'asse y; si comporta in modo analogo a G_x. Non usiamo solo una singola riga di pixel, ma consideriamo anche la riga sopra e sotto, con peso inferiore (di qui il pattern $[-1, 0, 1]$ esteso su più righe).

Thresholding.

Si calcola il quadrato del gradiente e si applica una soglia:

$$I_2 = G > \bar{I}$$

ottenendo un'**immagine binaria**.

Curve Fitting.

Una volta ottenuto un insieme di punti "candidati" (i pixel sopra soglia), l'ultimo step usa l'informazione su cosa stiamo cercando (linee dritte) per interpretare questi punti. Quando abbiamo un insieme di punti che sappiamo giacere su una linea, la prima cosa che viene in mente è l'algoritmo dell'**errore quadratico** (square error / least squares):

$$\min_{a,b} \sum_{i=1}^{N} (y_{data} - a\, x_{data} - b)^2$$

per una retta del tipo $y = ax + b$.

2.6.2 RANSAC (Random Sample Consensus)

Il minimo quadratico classico funziona bene se **tutti** i punti appartengono effettivamente alla linea, ma non è robusto in presenza di **outlier** (punti che non appartengono alla struttura cercata). Il **RANSAC** è un algoritmo pensato proprio per questo scopo.

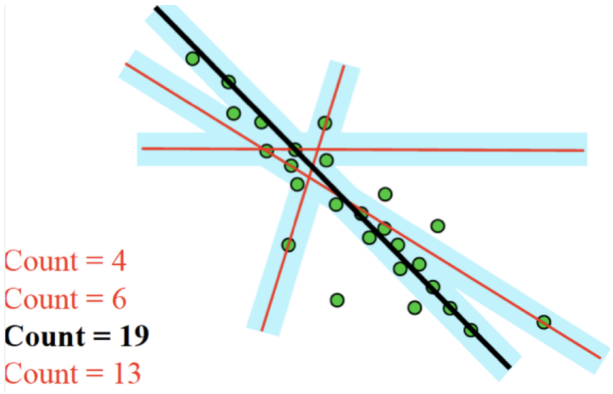
2.6.2.1 Caratteristiche del RANSAC

- Può essere usato con **diversi modelli** (non solo rette);
- è **computazionalmente efficiente**;
- è un algoritmo **non deterministico**;
- risolve il problema di fitting **molte volte**, usando ogni volta un sottoinsieme minimale di dati selezionato casualmente, e scegliendo alla fine il miglior fit trovato.

2.6.2.2 Funzionamento

Stiamo cercando una linea retta: si scelgono **casualmente 2 punti**, e si calcola la retta che passa esattamente per questi due punti. Si classificano poi gli altri punti come **inlier** o **outlier**: gli inlier sono tutti i punti che ricadono entro una certa distanza (regione, tipicamente rappresentata come una fascia) dalla retta stimata.

Si ripete il procedimento scegliendo un'altra coppia di punti casuale, e così via per un certo numero di iterazioni. Alla fine si sceglie la retta che ha ottenuto **più inlier**.



2.6.2.3 Quante iterazioni servono

La scelta di quando fermarsi è un aspetto interessante. Per essere sicuri di trovare la retta corretta, in linea di principio servirebbe **esplorare tutte le possibili coppie** di punti — ma questo renderebbe l'algoritmo rapidamente **computazionalmente infattibile**. Ad esempio, per un dataset di N punti, il numero di combinazioni possibili di 2 punti è $\binom{N}{2} = \frac{N(N-1)}{2}$: per 1000 punti, servirebbero circa $1000 \times 999/2 = 500{,}000$ possibilità — troppe per essere tutte testate.

Invece di testare tutte le possibili opzioni, si può **stimare** il numero di iterazioni necessarie, dato un'idea approssimata della frazione di inlier nel dataset.

Derivazione.

Sia:

- N il numero totale di punti nel dataset;
- w la frazione di inlier nel dataset, cioè $w = \dfrac{\text{numero di inlier}}{N}$ — interpretabile anche come la probabilità di selezionare un punto inlier dal dataset.

Assumendo che i 2 punti necessari per stimare una retta vengano selezionati in modo **indipendente**:

$w^2 = P(\text{entrambi i punti selezionati sono inlier})$ $\quad$ $1-w^2 = P(\text{almeno uno dei due è un outlier})$

Sia k il numero di iterazioni RANSAC eseguite finora:

$(1-w^2)^k = P(\text{RANSAC non ha mai selezionato due punti entrambi inlier dopo } k \text{ iterazioni})$

Sia p la **probabilità di successo**, cioè la probabilità di aver selezionato almeno una volta

due punti entrambi inlier dopo k iterazioni:

$$1 - p = (1 - w^2)^k \quad \Longrightarrow \quad k = \frac{\log(1-p)}{\log(1-w^2)}$$

Una volta che un'iterazione sceglie due punti entrambi inlier, "abbiamo finito": facendo girare l'algoritmo abbastanza a lungo da essere ragionevolmente sicuri di aver scelto almeno una volta due inlier, possiamo fermarci.

Esempio numerico.

Se vogliamo una probabilità di successo $p = 99\%$ e sappiamo che $w = 50\%$, si ottiene $k \approx 16$ iterazioni — un numero **significativamente inferiore** rispetto al numero di tutte le combinazioni possibili (nell'esempio precedente, 500,000).

Osservazione importante: il **numero di punti** nel dataset *non* influenza il numero minimo di iterazioni k necessarie — a determinarlo è solo w, la frazione di inlier.

Un approccio alternativo al RANSAC, che funziona solo con linee rette, è la **trasformata di Hough** (Hough transform).

2.7 Neural Network Image Processing

2.7.1 Model based vs neural network

Nel **model based image processing** dobbiamo trovare strumenti matematici per descrivere una feature. Sappiamo cosa aspettarci dall'algoritmo, perché è sviluppato sotto assunzioni ben comprese, e in alcuni casi è possibile garantire matematicamente che funzioni. Tuttavia, in molti casi, la conoscenza a priori necessaria è **difficile da modellare**: possiamo facilmente definire la figura geometrica di un triangolo, ma descrivere matematicamente cosa si trovi *all'interno* di un cartello triangolare (ad esempio, il pittogramma di pericolo) diventa molto più complicato.

Questo motiva il passaggio al **supervised learning** — le reti neurali.

2.7.2 Supervised learning

L'idea del **supervised learning** è insegnare all'algoritmo cosa riconoscere, tramite esempi. Concretamente, si usa un insieme di dati **etichettati** (labeled data) per addestrare una rete neurale a riconoscere quegli oggetti. Addestrare la rete significa determinare la

"forza" (i pesi) dei collegamenti tra i neuroni attraverso gli esempi forniti.

Esistono centinaia di strutture di rete neurale diverse, che si sono dimostrate efficaci in determinate condizioni. Esempi citati nel corso: **YOLO** e **RetinaNet**.

Vantaggi:

- funzionano bene con oggetti **difficili da modellare** matematicamente;
- sono computazionalmente **efficienti** nella fase di inferenza.

Svantaggi:

- sono **black-box** — questo approccio funziona molto bene quando le cose vanno bene, ma se qualcosa va storto è difficile capire *perché* sia andato storto;
- richiedono **migliaia di immagini etichettate** per essere addestrate.

2.7.3 Training vs inference

- **Training phase** — quando "insegniamo" alla rete, fornendo migliaia di immagini etichettate (ad esempio per addestrare YOLO a riconoscere e etichettare oggetti nell'immagine);
- **Inference phase** — quando usiamo la rete già addestrata "online", cioè in fase operativa.

Nella guida autonoma, questo tipo di approccio è particolarmente rilevante per compiti come il riconoscimento di altri veicoli, il rilevamento di segnali stradali, ecc. Esistono anche reti neurali capaci di ricostruire, a partire dalle immagini, **bounding box 3D** attorno agli oggetti rilevati.

2.7.4 Altre tecnologie di visione

Oltre a stereo camere e reti neurali, esistono anche tecnologie ibride basate su vision, come le **Time of Flight Cameras**, che combinano il principio di funzionamento dei sensori time-of-flight (misura diretta della distanza) con l'acquisizione di un'immagine, fornendo direttamente una mappa di profondità in millimetri per ogni pixel.

3 | Localization

3.1 Il quadro generale: perché la localizzazione è un problema a sé

Per un veicolo autonomo, sapere *dove si trova* è un prerequisito per tutto il resto: senza una stima affidabile di posizione e stato del veicolo, sia il planning (che lavora su un riferimento globale/locale) sia il control (che deve inseguire una traiettoria) non hanno un punto di partenza consistente. Il modulo di Localizzazione + Maps si colloca quindi a monte della pipeline, insieme alla Perception, e alimenta sia il Planning che il Control.

Il problema si scompone in tre sotto-problemi concettualmente distinti:

- **Localization problem** — stimare/misurare *pose* e *stato* del veicolo rispetto a un riferimento (assoluto o relativo);
- **State estimation** — stimare in particolare le grandezze cinematiche/dinamiche interne del veicolo (velocità longitudinale, laterale, attrito stradale);
- **Exteroceptive-based relative localization** — localizzarsi rispetto a una struttura locale dell'ambiente (una corsia, un filare) quando non si dispone di una mappa globale.

3.1.1 Pose e Stato: due concetti diversi

È fondamentale non confondere **pose** e **stato**, perché rispondono a domande diverse e vengono stimati con strumenti diversi.

Pose — è l'informazione di *alto livello* su posizione e orientamento del veicolo:

- posizione globale X, Y;
- heading ψ.

Per un veicolo su strada (lane vehicle), per descrivere in modo univoco la pose serve sapere sia la posizione sul piano sia l'assetto (verso il quale il veicolo sta puntando). La pose può

essere:

- **assoluta** — rispetto a un sistema di riferimento fisso (es. coordinate globali di una mappa);
- **relativa** — rispetto a una feature locale (es. la corsia): un navigatore dà una posizione assoluta, ma se si processa l'immagine per rilevare la corsia si può ottenere anche una posizione relativa ad essa.

Stato — è l'informazione di *basso livello* sul moto e sulle condizioni interne del veicolo, necessaria perché la sola pose non basta a localizzare correttamente il veicolo nel tempo:

- velocità longitudinale v_x;
- velocità laterale v_y — quanto il veicolo sta scivolando; da qui l'angolo di deriva (side-slip), rilevante ad esempio nella guida rally;
- attrito stradale (road friction);
- assetto del veicolo: pitch, roll, corse delle sospensioni.

Entrambe le informazioni sono mandatorie sia per il local planning sia per gli algoritmi di controllo.

3.2 SLAM: posizionamento assoluto

SLAM (Simultaneous Localization And Mapping) è l'approccio classico per ottenere una *pose assoluta*. L'idea è costruire, mentre ci si muove, una **mappa di localizzazione** (eventualmente georeferenziata, cioè associata a coordinate di latitudine/longitudine) usando due categorie di sensori:

- **sensori propriocettivi** — misurano lo stato interno del veicolo, senza informazioni sull'esterno. Esempi: **IMU** (Inertial Measurement Unit, misura accelerazioni e velocità angolare) ed **encoder** (misurano la velocità delle ruote). Non sono strettamente necessari per fare SLAM ma sono estremamente utili;
- **sensori esteroceptivi** — misurano l'ambiente esterno: LiDAR e camere, entrambi in grado di fornire dati sotto forma di point cloud 3D $\mathcal{P} = \{(x_1, y_1, z_1) \dots (x_n, y_n, z_n)\}$ nel riferimento del sensore.

L'uscita del modulo SLAM è una stima di pose: posizione e orientamento (2D o 3D) rispetto a un riferimento fissato alla mappa (world-fixed, se georeferenziato).

L'approccio più efficace in pratica è il **graph-based SLAM**, organizzato in due macro-componenti concettualmente separate:

- **front-end** (aka Local SLAM) — è una sorta di odometria: stima quanto il veicolo si è mosso tra un istante e il successivo, confrontando scansioni successive. È soggetto a *drift* (l'errore si accumula nel tempo);
- **back-end** (aka Global SLAM) — mette insieme le informazioni prodotte dal front-end e corregge la deriva accumulata, riconoscendo luoghi già visitati.

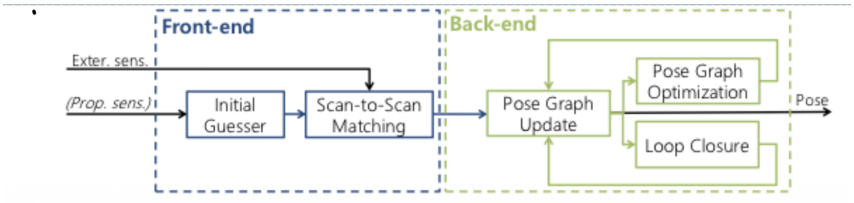
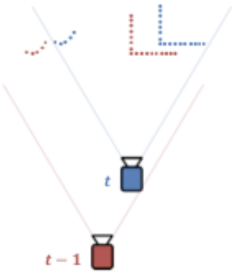


Figure 3.1: SLAM Algorithm

3.2.1 Front-end: scan-to-scan matching

Obiettivo: dati due istanti temporali consecutivi $t-1$ e t, stimare la posa relativa tra i due, cioè di quanto si è mosso il sensore.



Si dispone di due point cloud, $\mathcal{P}_{t-1}$ e $\mathcal{P}_t$. L'idea è confrontarle direttamente: se l'ambiente è statico (ipotesi fondamentale del metodo — tutto ciò che cambia tra le due scansioni è dovuto al movimento del veicolo), allora la trasformazione rigida (*rototraslazione*) che sovrappone al meglio le due nuvole di punti corrisponde esattamente al movimento del sensore tra $t-1$ e t.

L'algoritmo procede quindi così:

1. si parte dalle due point cloud;
2. si osservano da una prospettiva relativa;
3. si calcola la rototraslazione che massimizza la sovrapposizione (overlap) tra le due, ottenendo così di quanti metri/gradi si è spostato il sensore.

Initial guesser.

Molti algoritmi di scan-matching richiedono un'inizializzazione, cioè una stima approssimativa del movimento da cui partire la ricerca dell'allineamento ottimo. Questa inizializzazione può derivare da:

- ipotesi semplificative (piccolo spostamento, velocità costante);
- sensori propriocettivi: se si conosce la velocità delle ruote, integrandola si stima di quanto ci si è mossi; integrando lo yaw rate si stima di quanto si è ruotato.

Odometria e il problema del drift.

Concatenando (cumulando via moltiplicazione di matrici di rototraslazione) le uscite successive dello scan-to-scan matcher si ottiene una traiettoria odometrica. In linea di principio il front-end da solo basterebbe a sapere dove ci si trova; il problema pratico è che **ogni stima è affetta da un errore**, e integrando molti passi successivi questi errori si accumulano — è il fenomeno del **drift**. Da qui la necessità del back-end.

3.2.2 Back-end: pose graph e loop closure

Il back-end è progettato per correggere il drift. L'unica informazione aggiuntiva realmente disponibile, muovendosi in un ambiente sconosciuto, è accorgersi di essere ripassati in un punto già visitato: questa è l'idea di fondo della **loop closure**.

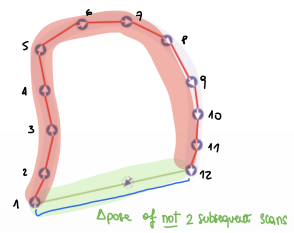
Pose graph update.

Il front-end fornisce, ad ogni step, la posa relativa tra istanti successivi (Δpose). Questa informazione viene organizzata in un **grafo**:

- ogni **nodo** rappresenta la posa del sensore (x, y, θ) a un certo istante di acquisizione — una posa che *non conosciamo* e vogliamo stimare;
- ogni **arco** rappresenta una misura, cioè uno scostamento di posa (Δpose) tra due nodi — una misura che *conosciamo*.

Se il veicolo è equipaggiato con più sensori esteroceptivi (es. LiDAR e stereo-camera), lo scan matching di ciascun sensore produce una propria misura indipendente, quindi si possono avere più archi paralleli tra gli stessi nodi.

Loop closure.



Ad ogni nuova scansione, questa non viene confrontata solo con la precedente (come nel front-end), ma anche con *tutte* le scansioni precedenti già acquisite. Se si trova una corrispondenza, significa che si è tornati in un luogo già visitato in precedenza: questo produce un nuovo arco di misura che collega il nodo attuale a un nodo "vecchio" del grafo (non necessariamente il predecessore immediato).

Pose Graph Optimization (PGO).

A questo punto si ottimizzano i valori (x, y, θ) associati a ciascun nodo — si noti: si aggiornano i *nodi*, non le misure, che restano fisse in quanto osservazioni. Per un nodo coinvolto in una loop closure (ad esempio il nodo 12 nell'esempio delle note) si hanno contemporaneamente:

- misure che vincolano la sua posizione lungo il "percorso rosso" (la sequenza odometrica normale);
- misure aggiuntive dalla loop closure che lo vincolano rispetto a un nodo lontano nel tempo ma vicino nello spazio.

L'ottimizzazione consiste nel muovere le pose dei nodi in modo da **minimizzare la somma degli errori su tutte le misure** (tutti gli archi del grafo). Questo è ciò che permette, di fatto, di "richiudere" l'anello e ridistribuire l'errore accumulato lungo tutto il tragitto, correggendo il drift.

Se il veicolo procede sempre dritto, senza mai richiudere un anello, il drift non può essere corretto solo con la loop closure — a meno di non introdurre un'altra fonte di misura assoluta. Un caso tipico: un GNSS/GPS. Ogni volta che si dispone di una buona ricezione GNSS, si può aggiungere un nodo di riferimento GNSS al grafo, fornendo un'ulteriore misura assoluta che ancora la soluzione e limita la deriva.

3.2.3 SLAM vs. Autonomous Driving: uso asincrono

Il nome SLAM sottolinea la natura *simultanea* di localizzazione e mappatura: si sta guidando in un ambiente completamente sconosciuto. In pratica, lo stesso algoritmo viene usato in due modalità, in modo **asincrono**:

- **fase di mappatura (SLAM alg)** — il primo passaggio, in cui si costruisce effettivamente la mappa;

- **fase di guida autonoma (AUT Driving)** — si riusa lo stesso tipo di algoritmo, ma partendo da un database già costruito (la mappa): non si stanno più visitando posti nuovi per salvarli, ma si stanno rivisitando posti già memorizzati per localizzarsi rispetto ad essi.

Il pregio dell'architettura front-end/back-end è la sua **modularità**: permette di affrontare separatamente il problema locale (quanto mi sono mosso) e quello globale (dove sono rispetto a tutto il resto).

3.3 State Estimation *

3.3.1 Idea generale: closed-loop estimation

Lo state estimation affronta un problema diverso dalla pose: non determinare *dove* si è, ma *come* ci si sta muovendo (velocità, road friction). L'idea di fondo è quella dello **state observer**: si vuole stimare lo stato di un sistema dinamico che non è direttamente misurabile, sfruttando la misura di *altre* grandezze correlate.

Il principio è: si tiene un modello del sistema in parallelo al sistema reale; alimentando modello e sistema reale con lo stesso ingresso u, se lo stato stimato $\hat{x}$ è corretto, anche l'uscita stimata $\hat{y}$ (calcolata dal modello) coinciderà con l'uscita realmente misurata y. Se invece $\hat{x}$ è sbagliato, ci sarà uno scarto $\hat{y} - y$: questo scarto viene retroazionato per correggere la stima dello stato. Più piccolo è lo scarto tra uscita stimata e misurata, meno bisogna correggere; se l'errore è grande, la stima è chiaramente fuori strada.

Formalmente, dato il sistema reale (lineare, per semplicità)

$$\dot{x} = Ax + Bu, \qquad y = Cx + Du, \tag{3.1}$$

dove x non è accessibile ma u e y sì, si costruisce un **osservatore**:

$$\dot{\hat{x}} = A\hat{x} + Bu + K(\hat{y} - y), \qquad \hat{y} = C\hat{x} + Du. \tag{3.2}$$

Perché non basta usare solo $\dot{\hat{x}} = A\hat{x} + Bu$ (cioè simulare il modello "in open loop")? Per due motivi pratici:

1. lo stato iniziale $\hat{x}(0)$ va comunque assunto arbitrariamente, se non lo si conosce con precisione;
2. se il sistema reale è soggetto a disturbi non modellati, la simulazione open-loop

diverge dal comportamento reale senza alcun meccanismo di correzione.

Definendo l'errore di stima $e = x - \hat{x}$, si può ricavarne la dinamica:

$$\begin{aligned} \dot{e} &= \dot{x} - \dot{\hat{x}} = Ax + Bu - A\hat{x} - Bu - K(y - \hat{y}) \\ &= Ae - K(y - \hat{y}) = Ae - KCe \\ &= (A - KC)e. \end{aligned} \tag{3.3}$$

Questa è la cosiddetta **error dynamics**: sotto opportune ipotesi (osservabilità del sistema), è possibile scegliere il guadagno K in modo da garantire che l'errore converga a zero, con la velocità di convergenza desiderata. Il bello di questa idea è che si applica, concettualmente, a qualsiasi sistema dinamico che soddisfi le ipotesi necessarie, e permette di fondere informazioni da più sensori che misurano indirettamente la stessa grandezza incognita.

Considerando poi il sistema reale con disturbi di processo e di misura,

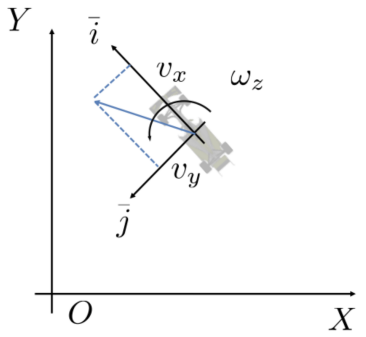
$$\dot{x} = Ax + Bu + Qd, \qquad y = Cx + Du + Rn, \tag{3.4}$$

dove d è un disturbo incognito e n un rumore di misura, si arriva naturalmente all'idea di filtro di Kalman come caso particolare (ottimo, sotto ipotesi di rumore gaussiano) di questo schema di osservatore.

3.3.2 Modelli del veicolo *

Un problema di stima richiede sempre un modello che leghi *stati incogniti*, *ingressi noti* e *misure*. È importante ricordare che "tutti i modelli sono sbagliati, alcuni sono utili": l'utilità di un modello dipende da cosa se ne deve fare, non dalla sua fedeltà assoluta alla realtà.

3.3.2.1 Rigid Body Kinematic Model *



Il modello più semplice possibile non assume nulla sul veicolo, se non che si muove come corpo rigido su un piano (2D). Considerando la velocità vettoriale $\bar{V} = v_x\bar{i} + v_y\bar{j}$ nel riferimento body (non inerziale, perché ruota insieme al veicolo), la sua derivata temporale non è semplicemente la derivata delle componenti, perché anche i versori $\bar{i}, \bar{j}$ cam-

biano direzione nel tempo:

$$\frac{d\bar{V}}{dt} = \frac{d(v_x\bar{i} + v_y\bar{j})}{dt} = \dot{v}_x\bar{i} + v_x\omega_z\bar{j} + \dot{v}_y\bar{j} - v_y\omega_z\bar{i} = a_x\bar{i} + a_y\bar{j}, \tag{3.5}$$

da cui, uguagliando le componenti,

$$\boxed{\dot{v}_x = a_x + v_y\omega_z, \qquad \dot{v}_y = a_y - v_x\omega_z.} \tag{3.6}$$

Il termine $v\omega$ è il ponte tra derivata "ingenua" e derivata reale nel riferimento body: nasce dal fatto che il sistema di riferimento ruota.

Le equazioni cinematiche che legano velocità body e posizione globale sono poi

$$\dot{X} = v_x\cos\psi - v_y\sin\psi, \qquad \dot{Y} = v_x\sin\psi + v_y\cos\psi. \tag{3.7}$$

Questo modello mette in relazione:

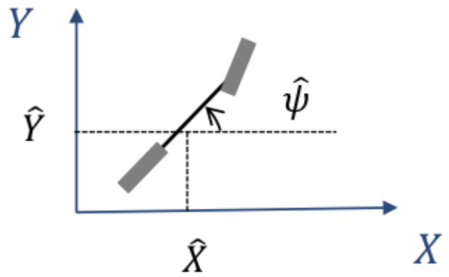
- **ingressi** — le misure fornite dall'IMU (a_x, a_y, ω_z);
- **uscite** — posizione X, Y.

Vantaggio: derivazione semplicissima, nessuna informazione specifica sul veicolo richiesta — è *parametric independent*, funziona per qualunque veicolo che si muova su un piano.

Svantaggio: il modello è essenzialmente un insieme di **integratori** (per ottenere ψ si integra ω_z, per ottenere X, Y si integrano le velocità). Ogni piccolo errore/bias sull'IMU, integrato nel tempo, diverge senza limite: il modello è *extremely sensitive to IMU noise and biases*. Inoltre, si noti che $\dot{v}_x = a_x + v_y\omega_z$ non è un puro integratore di a_x, a causa del termine di accoppiamento $v_y\omega_z$ — se ω_z è piccolo, ci si avvicina però al comportamento di un integratore puro.

3.3.2.2 Vehicle Bicycle Model

Il modello a corpo rigido non sfrutta nessuna informazione specifica sul veicolo. Per fare di meglio, si passa al **bicycle model**: l'idea è forzare un veicolo a 4 ruote in un **single track**, cioè rappresentare ruota sinistra e destra dello stesso asse come un'unica ruota centrale. Questo introduce un'approssimazione: in curve strette (raggio pic-

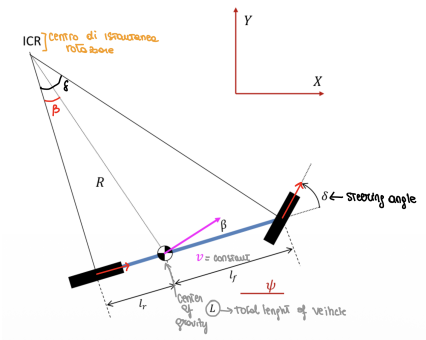


colo rispetto alla carreggiata) ruota interna ed esterna seguirebbero in realtà percorsi diversi, cosa che il modello single-track non cattura.

Kinematic Bicycle Model.

Ipotesi:

- la velocità V (al baricentro, COG) è costante;
- le ruote **non sono soggette a slip**: puntano esattamente nella direzione in cui il veicolo si sta muovendo (nessuno slittamento).



Ricordando che, in un triangolo rettangolo, vale la relazione $tan(\alpha) = \frac{cateto_opposto}{cateto_adiacente}$. Allora usiamo questa relazione con β e δ scrivendo

$$tan(\beta) = \frac{l_r}{H} \qquad tan(\delta) = \frac{L}{H}$$

da cui ricaviamo che $H = \frac{L}{tan(\delta)}$ e sostituiamo nella prima espressione ottenendo

$$tan(\beta) = \frac{l_r}{H} = \frac{l_r}{\frac{L}{tan(\delta)}} = \frac{l_r}{L} \cdot tan(\delta)$$

Con queste ipotesi si ha una relazione *statica* tra angolo di sterzo δ e side-slip angle β:

$$\boxed{\hat{\beta} = \arctan\left(\frac{l_r}{L}\tan\delta\right)} \tag{3.8}$$

dove L è il passo (lunghezza) del veicolo e l_r la distanza tra asse posteriore e baricentro. Le equazioni del modello sono:

$$\dot{X} = V\cos(\hat{\psi}+\hat{\beta}), \qquad \dot{Y} = V\sin(\hat{\psi}+\hat{\beta}), \qquad r = \frac{V}{L}\tan\delta\cos\beta, \tag{3.9}$$

dove r è lo yaw rate — anch'esso legato geometricamente allo sterzo, essendo V noto/costante e input al modello.

Dynamic Bicycle Model.

Si rimuove l'ipotesi di assenza di slip: adesso servono le equazioni di equilibrio di forze e momenti (perché è un modello *dinamico*, non solo cinematico):

$$Ma_y = F_{yf} + F_{yr} \qquad \text{(equilibrio forze laterali)} \tag{3.10}$$

$$J_z\dot{r} = l_f F_{yf} - l_r F_{yr} \qquad \text{(equilibrio momento di imbardata)} \tag{3.11}$$

$$\dot{x} = V\cos(\psi + \beta) \tag{3.12}$$

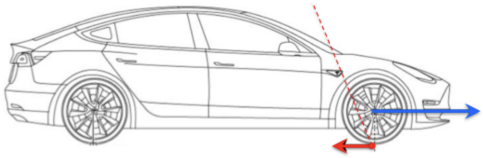
$$\dot{y} = V\sin(\psi + \beta) \tag{3.13}$$

$$\dot{\psi} = r \tag{3.14}$$

Qui β non è più determinato da una relazione statica, ma dipende dalle forze laterali F_{yf}, F_{yr}, che a loro volta dipendono dallo stato interno del sistema (slip angles). Il problema si sposta quindi a: come esprimere le forze F_y in funzione delle grandezze note/stimate.

3.3.2.3 Modelli di attrito pneumatico

Slip longitudinale λ.



È definito come differenza normalizzata tra velocità di rotolamento della ruota e velocità del corpo del veicolo. Se v è la velocità del corpo, ω la velocità angolare della ruota e r il raggio di rotolamento, il punto di contatto ha velocità nulla solo in condizione di puro rotolamento ($v = \omega r$). Si definisce

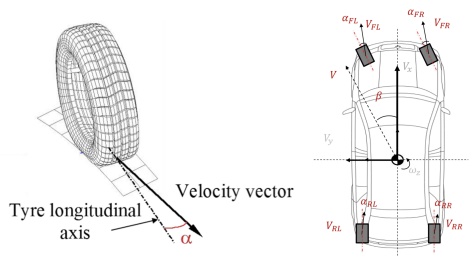
$$\lambda = \frac{v - \omega r}{\max\{v, \omega r\}}, \tag{3.15}$$

che, distinguendo i due casi (frenata/trazione) per garantire $0 \leq \lambda \leq 1$, diventa:

$$\text{frenata: } \lambda = \frac{v - \omega r}{v}, \qquad \text{trazione: } \lambda = \frac{-(v - \omega r)}{\omega r}. \tag{3.16}$$

$\lambda = 0$ corrisponde a rotolamento libero (free rolling); $\lambda = 1$ in frenata corrisponde a ruota bloccata (locked wheel); $\lambda = 1$ in trazione corrisponde a pattinamento puro (burnout).

Slip laterale α.



È l'angolo tra l'asse longitudinale dello pneumatico e il vettore velocità della ruota.

Le forze pneumatico-strada, sia longitudinale che laterale, sono funzioni (approssimativamente) del carico verticale, di λ, di α (e dell'angolo di camber, spesso trascurato):

$$F_x \approx F_z \cdot \mu_x(\lambda, \alpha), \qquad F_y \approx F_z \cdot \mu_y(\lambda, \alpha). \tag{3.17}$$

Qualitativamente:

- **forza longitudinale vs** λ — inizialmente cresce in modo circa lineare con λ; oltre un certo valore (tipicamente $\lambda \approx 0.1$–0.15) si raggiunge un massimo, oltre il quale la pendenza cambia segno (la forza cala). All'aumentare dell'angolo di side-slip α, la forza longitudinale massima disponibile diminuisce e il punto di massimo si sposta verso λ più piccoli;
- **forza laterale vs** α — è nulla se $\alpha = 0$; cresce linearmente con α per poi saturare; è massima quando $\lambda = 0$ (nessuna forza longitudinale); se le ruote sono bloccate ($\lambda = 1$) la forza laterale si annulla del tutto (perdita totale di direzionalità).

Poiché μ_x e μ_y dipendono entrambi da λ e α, esiste un compromesso tra le due: questa è esattamente la **friction ellipse** già incontrata nel planning, qui vista "dal basso", a livello di pneumatico: combinando tutte le coppie (λ, α) possibili si ottiene un inviluppo (circa un cerchio/ellisse) che rappresenta la massima forza totale disponibile in ogni direzione. Se si vuole accelerare/frenare al massimo possibile, la forza laterale disponibile tende a zero, e viceversa.

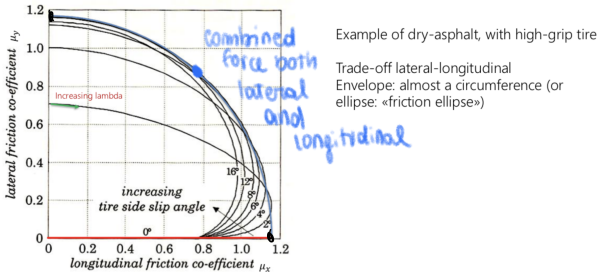


Figure 3.2: Friction Ellipse

3.4 Stima combinata cinematica-dinamica dello stato (algoritmo di state estimation)

3.4.1 Il problema: nessuno dei due modelli, da solo, basta

Riassumendo i pro/contro visti finora:

- **modello cinematico** — sempre valido, pochi/nessun parametro del veicolo richiesto, ma **diverge per integrazione** (drift) quando ω_z è piccolo (marcia rettilinea);
- **modello dinamico** — accurato sia a basse che alte velocità, ma richiede **molti parametri** del veicolo (in particolare il coefficiente di attrito, spesso non noto con precisione) e la conoscenza di a_x.

L'idea vincente è **fondere** i due modelli, non sceglierne uno.

3.4.2 L'osservatore ibrido cinematico-dinamico

Si parte dal modello cinematico riscritto in variabili di stato (velocità longitudinale/laterale) come sistema lineare tempo-variante (LTV):

$$\begin{bmatrix}\dot{\hat{v}}_x\\ \dot{\hat{v}}_y\end{bmatrix} = \begin{bmatrix}0 & \omega_z\\ -\omega_z & 0\end{bmatrix}\begin{bmatrix}v_x\\ v_y\end{bmatrix} + \begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}\begin{bmatrix}a_x\\ a_y\end{bmatrix}, \qquad \beta = \arctan\frac{v_y}{v_x}. \tag{3.18}$$

Qui a_x, a_y (misurate dall'IMU) sono gli ingressi dell'osservatore.

Il problema, come detto, è che se $\omega_z \approx 0$ il sistema diventa sostanzialmente due integratori indipendenti — non osservabile in modo utile — e un osservatore progettato su questo solo modello **diverge** durante la marcia rettilinea. Viceversa, per ω_z non piccolo l'osservatore funziona bene.

Soluzione: aggiungere al modello cinematico un **termine di correzione a feedback**, costruito con l'approccio di closed-loop estimation già visto, che sfrutta l'accelerazione laterale come misura di confronto:

$$\begin{bmatrix}\hat{v}_x\\ \hat{v}_y\end{bmatrix}\Bigg|^{\cdot} = \begin{bmatrix}0 & \omega_z\\ -\omega_z & 0\end{bmatrix}\begin{bmatrix}\hat{v}_x\\ \hat{v}_y\end{bmatrix} + \begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}\begin{bmatrix}a_x\\ a_y\end{bmatrix} + \begin{bmatrix}K_x & 0\\ 0 & K_y(\dot{\beta})\end{bmatrix}\begin{bmatrix}V_x - \hat{V}_x\\ A_y - \hat{A}_y\end{bmatrix} \tag{3.19}$$

Il termine di correzione ha due componenti concettualmente distinte:

- $V_x - \hat{V}_x$: dove V_x è una **stima diretta** (non basata su un modello, ma su un segnale) della velocità longitudinale, ottenuta dalle ruote — vedi sotto;

- $A_y - \hat{A}_y$: dove A_y è misurata dall'IMU e $\hat{A}_y$ è ricostruita con un **modello dinamico semplificato** (single-track).

3.4.2.1 Stima diretta di V_x: min-max sulle velocità ruota

Per la correzione longitudinale, l'idea è sfruttare la velocità delle 4 ruote (FL, FR, RL, RR) per ottenere una prima stima grezza di v_x. Il ragionamento è: quando si accelera, la ruota motrice tende a slittare positivamente (pattinare in avanti) più delle altre, quindi la ruota *più lenta* è la più rappresentativa della reale velocità del veicolo; quando si frena, vale l'opposto — la ruota che si blocca tende a scivolare, e la ruota *più veloce* è la più rappresentativa. Da qui la logica **min-max**:

$$V_{x,\text{min-max}} = \begin{cases} \min(V_{\text{ruote}}) & \text{se } a_x > 0 \text{ (accelerazione: le ruote possono slittare positivamente)} \\ \max(V_{\text{ruote}}) & \text{se } a_x < 0 \text{ (frenata: le ruote possono slittare negativamente)} \end{cases} \tag{3.20}$$

3.4.2.2 Stima di A_y tramite modello dinamico semplificato

Per calcolare $\hat{A}_y$ si usa una versione **semplificata** del modello dinamico single-track, con le seguenti ipotesi aggiuntive rispetto al modello "pieno":

- nessuna differenziazione sinistra/destra;
- nessuna dinamica verticale;
- nessun accoppiamento con le forze longitudinali;
- yaw rate trascurato nell'equazione di bilancio delle forze.

Il bilancio di forze laterali dà:

$$\hat{A}_y = f_2(\omega_z, \delta_f, \delta_r, \hat{V}_x, \hat{V}_y) = m^{-1} \left(F_{sf} \cos \delta_f + F_{sr} \cos \delta_r \right), \tag{3.21}$$

dove le forze pneumatico sono modellate con una funzione a saturazione (tipo tangente iperbolica) dello slip angle:

$$F_{sx} = -\frac{C_{r_x}}{K_x} \tanh(K_x \cdot \alpha_x), \qquad x = r, f, \tag{3.22}$$

con gli slip angle anteriore/posteriore ottenuti dallo stato dell'osservatore stesso:

$$\alpha_f = -\delta_f + \beta + l_f \frac{\omega_z}{V_x/\cos\beta}, \qquad \alpha_r = \beta - l_r \frac{\omega_z}{V_x/\cos\beta}. \tag{3.23}$$

Si tratta quindi di un modello volutamente semplificato, il cui scopo **non** è essere il modello fisico più accurato possibile, ma essere adatto a scopi di stima in un osservatore closed-loop.

3.4.3 Identificazione dei coefficienti C_r, K_x

I parametri $C_{r_{f/r}}$ (cornering stiffness) e $K_{f/r}$ (slope coefficient) non sono noti a priori e vanno **calibrati** sperimentalmente. La procedura è:

1. **caricamento e selezione dei dati**: filtraggio passa-basso per rimuovere rumore; si selezionano solo i tratti con slip longitudinale piccolo ($<\bar{\lambda}$) — cioè quelli in cui l'ipotesi di trascurare l'accoppiamento longitudinale è più valida — e con yaw jerk piccolo ($<\bar{\dot{\omega}}_z$), cioè manovre non troppo brusche;
2. **costruzione dei segnali di identificazione**: calcolo di $\alpha_{r/f}$ usando il β misurato dal veicolo strumentato, e calcolo delle forze laterali stimate a partire dall'accelerazione laterale misurata (es. $F_{sf} = \frac{mA_y l_r}{L_r+L_f}$, $F_{sr} = \frac{F_{sf}L_f}{L_r}$ nel caso semplificato);
3. **ottimizzazione**: si risolve un problema ai minimi quadrati che minimizza lo scarto tra forza laterale "osservata" e forza laterale modellata:

$$J_x = \left| F_x - \frac{C_x}{K_x} \tanh(K_x \cdot \alpha_x) \right|_2^2. \tag{3.24}$$

Il modello risultante, essendo semplificato (nessuno slip longitudinale, nessun camber, nessun trasferimento di carico), non si adatta perfettamente a tutti i punti sperimentali — ma è comunque sufficientemente accurato per l'uso in un osservatore.

3.4.4 Quando fidarsi di quale modello: scheduling di K_y

Resta il problema di decidere, istante per istante, *quanto* pesare la correzione dinamica rispetto al comportamento puramente cinematico. Il guadagno $K_y(\dot{\beta})$ non è quindi costante, ma viene fatto variare in funzione di quanto la manovra è "eccitante" dinamicamente:

- se la dinamica del side-slip è **poco eccitata** ($|\dot{\beta}|$ piccolo, sotto una soglia) $\Rightarrow K_y =$

$K_{y,\min}$: ci si affida di più al modello cinematico;

- se la dinamica del side-slip è **molto eccitata** ($|\dot{\beta}|$ grande, sopra soglia) $\Rightarrow K_y = K_{y,\max}$: ci si affida di più alla correzione dinamica.

Nella pratica implementativa si adottano due accorgimenti:

- **isteresi**: la soglia usata per passare da $K_{y,\min}$ a $K_{y,\max}$ è diversa da quella usata per il passaggio inverso, in modo da evitare oscillazioni continue tra i due regimi (*chattering*);
- **filtraggio dell'inviluppo**: si stima l'inviluppo (envelope) del segnale $|\dot{\beta}|$ filtrato, in modo da classificare correttamente una manovra come "dinamicamente eccitante" anche se il segnale oscilla rapidamente attorno a un valore medio elevato. Un filtro di inviluppo troppo lento introduce però un ritardo nel rilevare che la manovra è terminata.

3.4.5 Dalla velocità alla pose completa: fusione con GNSS

Una volta ottenuta una stima affidabile di v_x, v_y (e quindi β), si può risalire alla pose completa integrando le equazioni cinematiche già viste:

$$\dot{X} = V\cos(\hat{\psi}+\hat{\beta}), \qquad \dot{Y} = V\sin(\hat{\psi}+\hat{\beta}), \qquad \dot{\psi} = \omega_z. \tag{3.25}$$

Anche qui, integrare "in open loop" porta a deriva. Si costruisce quindi un ulteriore osservatore in closed-loop, dove la correzione stavolta arriva da un **GNSS**:

$$\begin{bmatrix}\hat{X}\\ \hat{Y}\\ \hat{\psi}\end{bmatrix}^{\cdot} = \underbrace{\begin{bmatrix}V\cos(\hat{\psi}+\hat{\beta})\\ V\sin(\hat{\psi}+\hat{\beta})\\ \omega_z\end{bmatrix}}_{\text{modello}} + K\underbrace{\begin{bmatrix}\hat{X}-X\\ \hat{Y}-Y\\ \hat{\psi}-\psi\end{bmatrix}}_{\text{correzione GNSS}}. \tag{3.26}$$

La ragione per cui questa fusione è necessaria (non basta usare il GPS da solo) è duplice:

- il GNSS può avere **bassa frequenza** rispetto alla dinamica del veicolo (le sorgenti assolute sono tipicamente più lente dei sensori propriocettivi);
- il GNSS non è sempre disponibile (perdita di segnale, aree urbane, gallerie).

La soluzione è usare la misura GNSS quando disponibile, e nel frattempo affidarsi al modello per propagare la stima.

3.4.6 Sensor fusion generale: il filtro EKF

Generalizzando, un modulo di localizzazione realistico riceve informazioni da più sorgenti eterogenee (IMU, encoder ruote, GPS, localizzazione basata su mappa) e le fonde tramite un **Extended Kalman Filter (EKF)**. Le difficoltà pratiche da gestire sono:

- **misure multi-rate**: le sorgenti assolute sono tipicamente più lente di quelle propriocettive;
- **misure asincrone**: i sensori non sono sincronizzati tra loro. A meno di un modo centralizzato di misurare il tempo, il clock di ciascun sensore può essere leggermente diverso da quello degli altri, introducendo ritardi relativi. La soluzione "ideale" (riprogettare tutto in modo sincrono) spesso non è praticabile: si preferisce progettare un ritardo di fusione dei sensori (sensor fusion delay) tollerante a questa asincronia;
- **perdita di misure**: alcuni sensori possono diventare temporaneamente non disponibili;
- **qualità delle misure variabile nel tempo**: ad esempio la perdita di precisione RTK o un algoritmo map-based che perde il tracciamento richiedono uno *scheduling* dei guadagni del filtro.

3.5 Localizzazione relativa (mapless)**(warehouse)

3.5.1 Perché "senza mappa"

Quando non è disponibile (o non conviene costruire) una mappa globale, si può comunque localizzarsi **rispetto a una struttura locale** dell'ambiente: questo è l'approccio **mapless**. Rispetto alla localizzazione globale su mappa presenta alcuni vantaggi:

- più economico (non serve costruire/mantenere una mappa);
- più efficiente;
- più robusto in certi contesti;
- più "umano" (un guidatore, in fondo, si orienta rispetto a ciò che vede attorno a sé, non rispetto a coordinate assolute).

È molto efficace in ambienti **strutturati** (es. strade). Il caso di studio discusso nel corso è la **guida in filari** (vineyard/orchard row keeping) con un trattore agricolo: rispetto al semplice lane-keeping stradale, è un problema più impegnativo perché l'aspetto dei filari

cambia con la stagione (inverno vs. estate), a differenza di una corsia stradale che ha sempre lo stesso aspetto — è quindi un buon banco di prova per la robustezza.

3.5.2 Setup sensoristico

Il veicolo (trattore) è equipaggiato con:

- steering encoder, steering actuator, speed encoder — sensori/attuatori proprioceptivi classici;
- IMU;
- **sensori ultrasonici** — usati sia per misurare tempo di volo che distanza; il caso di studio ne usa 12 (6 per lato). Sono sensori economici ma **poco risolutivi**: forniscono la distanza dell'oggetto più vicino nel loro campo visivo (FOV conico), ma non la direzione precisa in cui si trova l'oggetto all'interno di quel cono — si assume, per ipotesi, che l'oggetto rilevato sia al centro del FOV;
- un LiDAR 3D usato come ground truth per validazione, non come sensore operativo primario.

3.5.3 Impostazione del problema

L'architettura generale è quella classica: veicolo — sensori — perception system — regolatore — attuatore. Il punto chiave è che, se il perception system è ben progettato, il resto del problema (controllo) diventa relativamente semplice.

Alcune osservazioni preliminari, importanti per capire come si è impostato il problema:

- Non si può considerare il veicolo come un punto: interessa la sua **direzione** (per capire l'azione di controllo appropriata) e il suo **ingombro** non è trascurabile rispetto alla larghezza del filare.
- **Non si può risolvere il problema di pose "assoluta"**: non essendoci una mappa, non c'è modo di sapere *a che punto* del filare (longitudinalmente) ci si trova.
- **Non serve nemmeno la pose completa**: l'obiettivo (lane/row keeping, cioè restare al centro del filare) richiede in linea di principio solo la *posizione* laterale. Tuttavia l'orientamento resta rilevante: la direzione verso cui si punta condiziona l'azione di sterzo da compiere (due veicoli nella stessa posizione ma con orientamento diverso richiedono un comando di sterzo diverso).

Si sceglie quindi di stimare solo due grandezze, invece dell'intera pose:

- **distanza dal filare** d (rispetto, per convenzione, alla fila di sinistra o destra, scelta arbitrariamente);
- **angolo di incidenza** γ — al posto dello heading assoluto, si usa l'angolo relativo rispetto alla direzione del filare.

Statico o dinamico?

Ci si può chiedere se il problema sia statico (risolvibile con le sole misure all'istante t) o dinamico (serve l'informazione temporale). In realtà, con una singola "fotografia" — le 12 misure ultrasoniche all'istante t — si potrebbe già stimare approssimativamente la posizione, interpolando le rette dei due filari. Tuttavia, per avere una stima più robusta e continua nel tempo (e per fondere le informazioni propriocettive), si adotta comunque una stima dinamica.

3.5.4 Modello cinematico per il row-following

Si riadatta il kinematic bicycle model al problema, esprimendo la dinamica nel riferimento **relativo al filare** anziché in coordinate assolute.

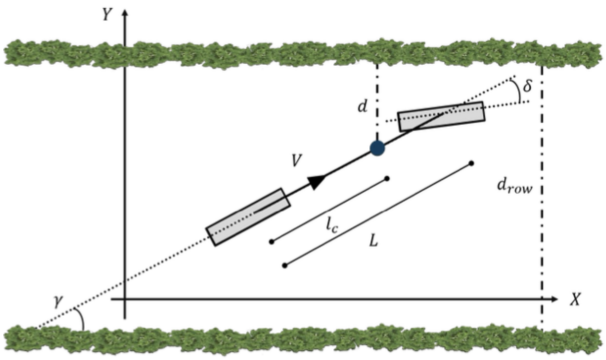


Figure 3.3: Vineyard

Variabili di stato: distanza dalla fila sinistra d, angolo di incidenza γ.

Ingressi: angolo di sterzo δ (positivo verso sinistra), velocità del veicolo all'asse posteriore V.

Parametri: lunghezza del veicolo L, distanza l_c tra asse posteriore e un punto arbitrario del veicolo (tipicamente il baricentro).

Ipotesi:

- filari (crop rows) rettilinei, o quasi;
- velocità longitudinale costante;
- V sufficientemente piccola da poter trascurare lo slip degli pneumatici (si resta quindi nell'ambito del modello cinematico, non dinamico).

Sotto queste ipotesi, il modello cinematico del veicolo, espresso in coordinate assolute allineate al filare, è:

$$\dot{X} = V\cos\gamma - l_c\, r\sin\gamma, \qquad \dot{Y} = V\sin\gamma + l_c\, r\cos\gamma, \qquad \dot{\gamma} = r, \tag{3.27}$$

con lo yaw rate calcolato dalla versione linearizzata del modello di sterzo cinematico:

$$r = \frac{V}{L}\delta. \tag{3.28}$$

(si assume, in questa versione semplificata, nessuno slip laterale all'asse posteriore).

Riduzione dello stato utile.

Scegliendo il sistema di riferimento allineato al filare, l'equazione per $\dot{X}$ (avanzamento lungo il filare) può essere trascurata perché non è utile ai fini del row-keeping (non interessa "quanto" si è avanzati, solo restare centrati), mentre $\dot{Y}$ può essere direttamente ricondotta a $\dot{d}$, dato che d è per costruzione la distanza laterale dal filare misurata lungo Y:

$$\dot{d} = -\dot{Y}. \tag{3.29}$$

Il modello si semplifica quindi in due sole equazioni di stato rilevanti per d e γ.

3.5.5 Modello di misura degli ultrasonici

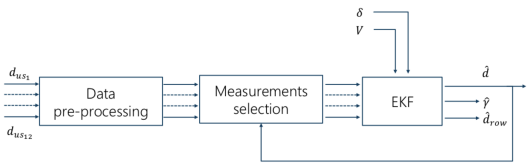


Figure 3.4: Closed Loop Estimation

Il modulo di stima è un **Extended Kalman Filter**, con ingressi δ, V e le misure ultrasoniche $d_{us_1}, \ldots, d_{us_{12}}$; le uscite sono la distanza $\hat{d}$ e l'angolo di incidenza $\hat{\gamma}$ (oltre alla distanza tra i due filari, vedi sotto).

Conoscendo d e γ, e la posizione geometrica di ciascun sensore lungo il veicolo, si può **predire** la distanza che ciascun sensore dovrebbe misurare — questo è il modello di misura (output equation) richiesto dal filtro. Per i sensori sul lato sinistro (lo stesso lato usato per definire d), con semplice trigonometria:

$$\tilde{d}_{us_i}(k) = \frac{d(k)}{\cos(\gamma(k))} + l_i \cdot \tan(\gamma(k)) - d_{T_i}, \qquad i \in [1, 6], \tag{3.30}$$

dove l_i è la posizione longitudinale del sensore lungo il veicolo e d_{T_i} tiene conto dello

scostamento laterale del sensore dal centro del veicolo.

Per i sensori sul lato destro, servirebbe conoscere la distanza tra i due filari d_{row}:

$$\tilde{d}_{us_i}(k) = \frac{d_{row}(k) - d(k)}{\cos(\gamma(k))} - l_i \cdot \tan(\gamma(k)) + d_{T_i}, \qquad i \in [7, 12]. \tag{3.31}$$

Il problema di d_{row}: uno "stato fittizio".

La larghezza del filare, d_{row}, non è nota a priori. Due strade sono possibili:

1. chiederla all'operatore (il conducente/agricoltore la conosce);
2. trattarla come un parametro incognito ma **non costante nel tempo** (in generale può variare, seppur lentamente, e non ha effetto diretto sulla dinamica del veicolo).

Nel framework del filtro di Kalman, un parametro incognito che si vuole comunque stimare va incluso come uno **stato aggiuntivo**, la cui equazione dinamica è semplicemente costante:

$$d_{row}(k+1) = d_{row}(k). \tag{3.32}$$

Questo è un trucco molto usato in generale: ogni volta che si ha un parametro incognito ma rilevante per il modello di misura, lo si può "promuovere" a stato con dinamica banale (costante), lasciando che sia il filtro stesso a stimarlo insieme al resto.

3.5.6 Modello di predizione a tempo discreto

Discretizzando il modello continuo con il metodo di Eulero (passo T_s) si ottiene:

$$\begin{cases} d(k+1) = d(k) - T_s V(k) \left(\sin(\gamma(k)) + \dfrac{l_c}{L} \cos(\gamma(k))\, \delta \right) \\ \gamma(k+1) = \gamma(k) + T_s \dfrac{V(k)}{L} \delta \\ d_{row}(k+1) = d_{row}(k) \end{cases} \tag{3.33}$$

3.5.7 EKF: perché "extended"

Il modello di misura contiene una non linearità evidente (i termini in $\cos(\gamma)$, $\tan(\gamma)$), quindi non si può applicare il filtro di Kalman lineare standard: serve una **linearizzazione locale** ad ogni passo — da qui **Extended** Kalman Filter. Lo schema concettuale resta quello classico a due fasi:

- **Time Update (Predict)** — si proietta lo stato in avanti col modello, e si propaga la covarianza dell'errore;
- **Measurement Update (Correct)** — si calcola il guadagno di Kalman $K_k = P_k^- H^T (H P_k^- H^T + R)^{-1}$, si aggiorna la stima con la misura reale, si aggiorna la covarianza dell'errore.

A differenza del caso "a guadagno costante" (come nell'osservatore cinematico-dinamico), qui il guadagno viene ricalcolato ad ogni istante risolvendo l'equazione di Kalman — questo garantisce ottimalità sotto le ipotesi del filtro (in particolare, rumore di processo e di misura gaussiani a media nulla).

3.5.8 Il problema pratico: outlier e rumore non gaussiano

L'ipotesi di rumore gaussiano, cruciale per l'ottimalità del filtro di Kalman, in pratica **non è verificata** dai sensori ultrasonici in questo scenario. Osservando l'istogramma delle misure raccolte guidando in linea perfettamente retta (quindi a distanza reale costante dal filare):

- in **inverno**: la maggior parte delle misure forma una distribuzione simile a una gaussiana, ma esiste anche un gruppo consistente di misure molto lontane da essa — dovute al sensore che, in corrispondenza di un "buco" nel filare (rado di vegetazione in inverno), rileva erroneamente il filare successivo (più lontano);
- in **estate**: la distribuzione principale è più gaussiana, ma restano misure anomale dovute sia a buchi sia a "rami sporgenti" più vicini del filare vero e proprio.

Applicando il filtro di Kalman "ingenuamente" su questi dati, la stima risulta **polarizzata** (bias): in inverno verso distanze sovrastimate, in estate in modo diverso a seconda della situazione.

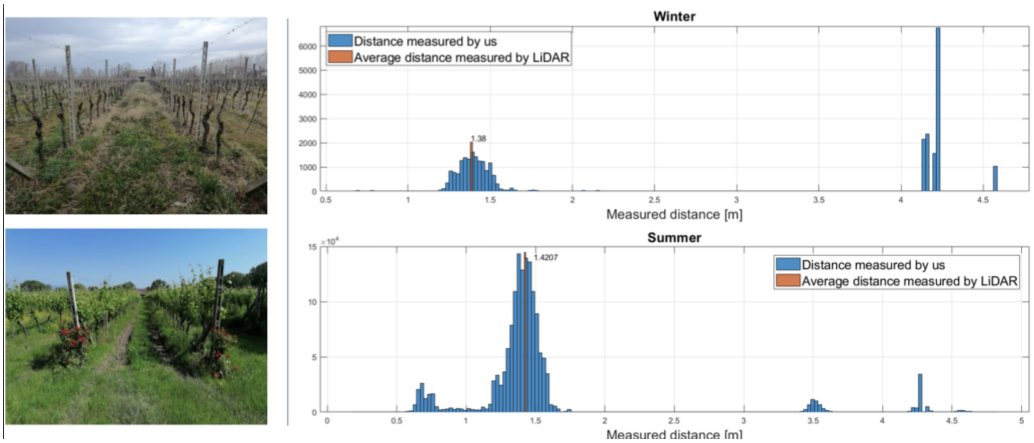


Figure 3.5: Vineyard appearance during seasons

3.5.9 Measurement selection: finestra centrata sulla stima

La soluzione adottata è un pre-processing **indipendente dal filtro di Kalman**, che seleziona solo le misure "gaussian-like" prima di darle in pasto all'EKF. L'idea è usare la stima corrente di posizione per centrare una **finestra** attorno ad essa, ed escludere le misure fuori da tale finestra (interpretabili come outlier/rumore anomalo).

Questo introduce un nuovo parametro di progetto: la **larghezza della finestra**. Il compromesso è delicato:

- finestra troppo **larga** — si perde il beneficio della selezione (gli outlier rientrano comunque nella finestra);
- finestra troppo **stretta** — si rischia di scartare misure valide, specialmente in condizioni dove la distribuzione reale della misura è più larga.

Sperimentalmente, la larghezza ottimale varia con la stagione: in inverno e primavera (filare composto perlopiù da tronchi sottili) conviene una finestra più piccola; in estate (filare più folto) una finestra troppo piccola considera solo la punta dei rami sporgenti anziché l'intero filare, e serve quindi una finestra più larga. Non esiste quindi un'unica taratura valida tutto l'anno (*difficult to find an all-season tuning*).

3.5.10 Window scheduling automatico tramite covarianza del filtro

Per evitare di dover ritarare manualmente la larghezza della finestra stagione per stagione, si sfrutta un'informazione che l'EKF fornisce "gratuitamente": la stima della **covarianza dell'errore** P. Si osserva che la distribuzione della covarianza dell'errore sull'angolo

(P_{22}) differisce sistematicamente tra le stagioni (più bassa/concentrata in inverno, più alta/dispersa in estate).

Si costruisce quindi, offline, una **curva di scheduling**: larghezza della finestra in funzione di P_{22}. Online, ad ogni istante si legge la covarianza corrente restituita dal filtro e si seleziona automaticamente la larghezza di finestra più opportuna, senza dover sapere esplicitamente in che stagione ci si trova.

I risultati sperimentali confermano il beneficio:

- senza alcuna selezione, in **inverno** il filtro tende sistematicamente a **sovrastimare** la distanza (a causa degli outlier verso il filare successivo);
- con finestra fissa la stima migliora ma non è ottimale in tutte le stagioni;
- con finestra **scheduled** (adattiva) si evita sia il problema invernale sia un problema analogo osservato in estate, ottenendo una stima coerente con il ground truth LiDAR in tutte le stagioni testate.

3.5.11 Osservazioni conclusive

- Il problema di localizzazione relativa nasce dall'assenza di mappa: si rinuncia alla pose assoluta e ci si concentra sulla posizione/orientamento relativi a una struttura locale (filare, corsia).
- Introdurre una **stima dinamica** (closed-loop, EKF) rispetto a una stima puramente istantanea migliora l'accuratezza, permettendo di fondere l'informazione esteroceptiva (ultrasonici) con quella propriocettiva (sterzo, velocità).
- Il filtro di Kalman è un modo naturale per fare closed-loop estimation, ma la sua ottimalità dipende da un'ipotesi (rumore gaussiano a media nulla) che va verificata criticamente: quando non è soddisfatta, va corretta con un pre-processing dedicato (measurement selection) piuttosto che ignorata.
- Lo stesso approccio concettuale (EKF + selezione/filtraggio delle misure) si può applicare, con un modello di misura diverso, alla localizzazione relativa rispetto a una corsia stradale, sostituendo gli ultrasonici con una **camera** e l'estrazione della corsia tramite bird's-eye view — resta da definire la relazione ingresso-uscita specifica per quel sensore.

4 | Target Tracking

4.1 Il problema: perché non basta la detection

Siamo nell'area **percettiva** del sistema autonomo. L'obiettivo non è semplicemente "vedere" un ostacolo, ma capire due cose contemporaneamente: **dove si trova** l'ostacolo (posizione) e **dove sta andando** (velocità, intesa come direzione e modulo). Questa seconda informazione è cruciale per il local planning: per pianificare la traiettoria del veicolo non basta sapere che "c'è un ostacolo qui adesso", serve prevedere dove si troverà tra qualche istante.

Il motivo per cui la velocità è la parte difficile del problema è il seguente: con un sensore **esterocettivo** generico (tipicamente un LiDAR), da una singola scansione si possono stimare posizione e orientamento di un oggetto, ma *non* la sua velocità — la velocità è per definizione una derivata temporale, e da un solo istante non si ha informazione sul tempo. Esistono due strade per ottenerla:

- **RADAR**: sfrutta l'**effetto Doppler** e può quindi stimare direttamente, in un singolo istante, la componente radiale della velocità dell'oggetto;
- **LiDAR** (o in generale sensori a nuvola di punti): non hanno questa capacità diretta. Per stimare la velocità è necessario **tenere traccia (track)** dello stesso oggetto in istanti successivi e osservarne lo spostamento — da qui il nome *tracking*.

C'è però un secondo problema, più sottile: anche avendo posizioni misurate in istanti diversi, come si fa a sapere che si tratta *dello stesso oggetto* e non di due oggetti diversi apparsi in istanti differenti? Non basta quindi conoscere le posizioni nel tempo: serve anche una forma di **identificazione/associazione** tra le misure, un problema che tornerà centrale più avanti sotto il nome di *data association*.

4.2 Due filosofie di approccio

4.2.1 Tracking by detection

L'idea è separare il problema in due moduli in cascata:

$$[\text{detection}] \longrightarrow [\text{tracking module}] \longrightarrow \text{tracks (pose + velocity)}$$

Prima si rileva (detection) l'ostacolo in ogni singolo frame, poi un modulo di tracking, separato, prende in ingresso queste rilevazioni nel tempo e produce, per ciascun *track* (oggetto seguito), una stima di posa e velocità. I modelli usabili per detection/tracking possono essere model-based, feature-based, oppure basati su machine learning.

4.2.2 End-to-end tracking

Approccio più recente, che salta lo step esplicito di detection: un unico modulo (quasi sempre basato su ML) produce direttamente posa e velocità a partire dai dati grezzi del sensore.

4.2.3 Confronto

Nonostante l'end-to-end sia l'approccio più moderno, **il tracking-by-detection fornisce tuttora stime migliori**. È questo il motivo per cui l'intero capitolo si concentra sulla costruzione, pezzo per pezzo, di una pipeline tracking-by-detection.

4.3 Dalla point cloud 3D alla griglia 2D

Il caso di studio è un robot mobile (*Yape*) equipaggiato con un LiDAR meccanico 3D (Quanergy M8: 8 strati di scansione, campo visivo orizzontale di 360°, campo verticale di 20°).

Due considerazioni motivano la riduzione da 3D a 2D:

1. **l'applicazione è essenzialmente planare**: il robot si muove su una superficie piatta;
2. **la coordinata z non è molto informativa**: il moto avviene sul piano e non c'è una variazione utile in altezza tra i vari target.

L'obiettivo è quindi **convertire l'input per ridurre la complessità**, passando da una point cloud 3D a una rappresentazione 2D bird's-eye-view (vista dall'alto). È importante sottolineare che non si "buttano via" dati arbitrariamente: si usano tutti gli strati del LiDAR, ma si comprime l'informazione lungo z, poiché l'altezza dell'ostacolo non è rilevante

per il tracking planare.

4.4 Detection

La pipeline completa che si costruisce è la seguente:

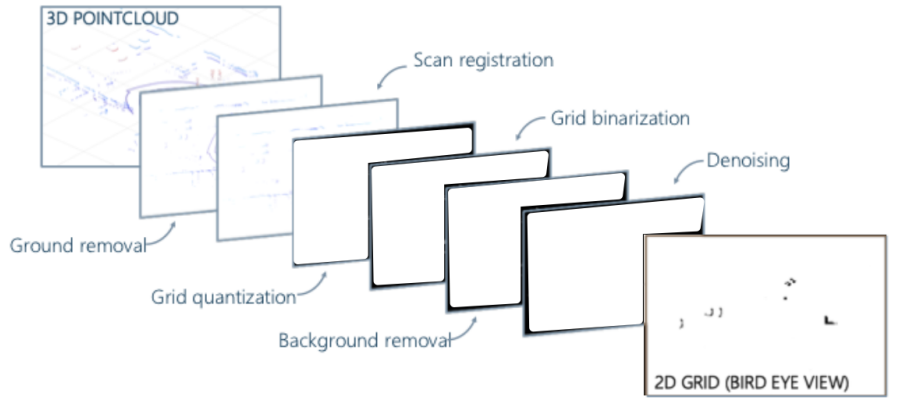


Figure 4.1: Pipeline

4.4.1 Ground Removal

I LiDAR a scansione, ogni volta che un raggio "punta verso il basso" (verso il terreno), registrano punti che sembrano ostacoli ma sono in realtà rumore (il terreno stesso). Vanno eliminati prima di ogni altra elaborazione. L'idea generale è:

1. segmentare la nuvola di punti;
2. etichettare ogni punto come "terreno" o "non terreno".

4.4.1.1 RANSAC

Lo strumento usato è **RANSAC** (RANdom SAmple Consensus), un algoritmo generale e robusto agli outlier per il fitting di modelli. Normalmente lo si conosce applicato al fitting di rette; qui viene usato per il fitting di **piani** (bastano 3 punti per definirne uno). L'algoritmo procede così:

- si estrae casualmente un piccolo campione di punti (qui: 3, il minimo per un piano);
- si fitta il modello (il piano) su questo campione;
- si contano quanti altri punti sono "vicini" al piano, entro una soglia d;
- se il numero di inlier supera una soglia T, il modello è considerato valido;

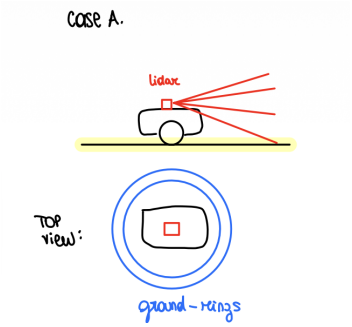
- si ripete per N iterazioni e si tiene il fit migliore (più inlier / minore errore).

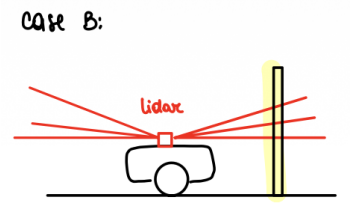
L'assunzione (debole, ma ragionevole) è che in un contesto di guida urbana il terreno sia sufficientemente piatto, quindi un singolo piano è un buon modello.

4.4.1.2 Il caso critico: pareti verticali

C'è un dettaglio importante, legato a due scenari opposti:

- **Caso A**: il LiDAR vede principalmente il terreno intorno a sé (anelli concentrici, i cosiddetti *ground rings*, nella vista dall'alto) ⇒ RANSAC trova correttamente il piano orizzontale come miglior fit.
- **Caso B**: il robot è vicino a una parete verticale molto estesa ⇒ il "miglior piano" che RANSAC trova può essere il piano *verticale* della parete stessa! Se si rimuove tutto ciò che appartiene a questo piano pensando sia terreno, si rischia di **rimuovere l'ostacolo reale** (la parete), che è proprio ciò che si vorrebbe rilevare.





Soluzione: si introduce un vincolo aggiuntivo — un piano verticale non può essere considerato terreno. RANSAC si presta bene a questo tipo di aggiustamenti (basta scartare i fit la cui normale è troppo lontana dalla verticale).

4.4.2 Scan Registration

4.4.2.1 Il problema

Il veicolo si muove nel tempo, quindi tra uno scan e l'altro occorre capire se ciò che si è spostato è l'ostacolo, il veicolo ego, o entrambi. Per confrontare correttamente scan successivi, bisogna riportarli in un sistema di riferimento comune.

Una prima opzione è usare la **localizzazione**: se si conosce con precisione lo spostamento del veicolo, si può "riallineare" la nuvola di punti di conseguenza. Il problema è che questo approccio dipende fortemente dalla qualità della localizzazione, che in genere stima bene

la posizione sul piano ma molto meno bene l'assetto (roll e pitch). Un errore di pochi gradi nell'assetto del sensore, proiettato su un punto lontano, può tradursi in errori di diversi metri nella nuvola di punti — troppo rischioso per affidarsi solo a questo.

4.4.2.2 ICP — Iterative Closest Point

La seconda opzione, più robusta, è la **scan registration tramite ICP**. Dati due insiemi di punti

$$X = \{x_1, \dots, x_{N_x}\}, \qquad P = \{p_1, \dots, p_{N_p}\},$$

si vogliono trovare rotazione R e traslazione t che minimizzano

$$E(R,t) = \frac{1}{N_p} \sum_{i=1}^{N_p} \|x_i - Rp_i - t\|^2. \tag{4.1}$$

Il problema si scompone in due sotto-problemi, risolti **iterativamente**:

1. trovare la **corrispondenza** — quale punto di P corrisponde a quale punto di X?
2. trovare la **rototraslazione** — dato l'accoppiamento, quali R e t minimizzano l'errore?

Calcolo della rototraslazione (nota la corrispondenza).

Il primo passo è sottrarre i centroidi:

$$\mu_x = \frac{1}{N_x} \sum_i x_i, \qquad \mu_p = \frac{1}{N_p} \sum_i p_i, \qquad X' = \{x_i - \mu_x\}, \qquad P' = \{p_i - \mu_p\}. \tag{4.2}$$

L'idea è traslare tutto sul centroide, in modo da disaccoppiare il problema di rotazione da quello di traslazione. Derivando $E(R,t)$ rispetto a t (assumendo R fissato) e ponendo il risultato uguale a zero, si ottiene la traslazione ottima:

$$t = \mu_x - R\mu_p. \tag{4.3}$$

Sostituendo questa espressione nell'errore originale e ottimizzando rispetto a R, il problema si riscrive — dopo aver espanso il quadrato della norma — come una **massimizzazione di una traccia**:

$$\arg\max_R \; 2\,\mathrm{trace}\left(P'^T R^T X'\right). \tag{4.4}$$

Definendo

$$W = \sum_i x'_i p'^T_i, \qquad [U, \Sigma, V] = \mathrm{SVD}(W), \tag{4.5}$$

si dimostra che la traccia è massimizzata quando $V^T R^T U = I$, da cui:

$$\boxed{R = UV^T} \qquad \boxed{t = \mu_x - R\mu_p}. \tag{4.6}$$

L'intuizione: U e V sono ortonormali per costruzione della SVD, quindi anche $R = UV^T$ è ortonormale (una vera rotazione). La matrice Σ ha elementi ≥ 0, e il prodotto trace(ΣM) con M ortonormale è massimizzato quando $M = I$ — da cui $V^T R^T U = I \Rightarrow R = UV^T$. I valori singolari σ_1, σ_2 descrivono come i punti si distribuiscono lungo le nuove direzioni principali della rototraslazione.

Calcolo della corrispondenza.

L'euristica più semplice è, per ogni punto, trovare il punto più vicino nell'altra nuvola (*nearest neighbor*). Il difetto è evidente: il nearest neighbor non dà necessariamente la corrispondenza fisicamente corretta, specialmente se le nuvole sono inizialmente molto distanti tra loro.

Algoritmo ICP (vanilla).

1. trova, per ogni punto, il più vicino nell'altra nuvola (corrispondenza);
2. calcola la rototraslazione ottima (R, t) tramite SVD;
3. applica la rototraslazione alla nuvola;
4. ripeti da 1 con le nuove posizioni.

Il difetto principale è la necessità di una **buona inizializzazione**: se le due nuvole partono troppo distanti, il primo step di corrispondenza sbaglia grossolanamente e l'algoritmo può convergere a un minimo locale sbagliato.

Varianti che migliorano ICP.

1. usare solo sottoinsiemi di punti (feature significative, es. angoli/corner);
2. pesare diversamente le corrispondenze (es. meno peso ai punti lontani, più soggetti a rumore);
3. usare algoritmi di associazione più sofisticati;
4. rigettare le coppie di punti troppo distanti (outlier rejection).

L'assunzione implicita di ICP è che le due nuvole rappresentino *la stessa scena*. Nel contesto del tracking questa assunzione è violata per costruzione (gli oggetti si muovono!). ICP resta comunque utile a patto che valgano due condizioni pratiche:

- **A)** l'inizializzazione sia ragionevole (es. usare la stima di localizzazione come punto di partenza);
- **B)** la parte fissa della scena (background statico) sia molto più grande della parte in movimento — il rapporto tra ciò che si muove e ciò che resta fermo deve essere < 1.

4.4.3 Grid Quantization e Binarization

4.4.3.1 Da point cloud a griglia

Si converte la rappresentazione da **lista di punti** a **matrice/griglia** (rappresentazione topologica dell'ambiente). Si sovrappone una griglia allo spazio; ogni cella riceve un valore pari al numero di punti che vi cadono.

Il vantaggio della griglia è l'accesso "statico"/diretto: per sapere cosa c'è in una posizione basta guardare le coordinate della cella, mentre con una point cloud si dovrebbe scorrere l'intera lista. Lo svantaggio è che l'occupazione di memoria non migliora necessariamente rispetto alla lista di punti.

4.4.3.2 Binarizzazione

Occorre decidere se ogni cella è occupata (1) o libera (0). L'idea semplice è porre una soglia sul numero di punti nella cella — in un mondo ideale basterebbe un solo punto, ma i sensori reali sono rumorosi, quindi la soglia funge da filtro di rumore.

Una soglia **costante non va bene**, perché il LiDAR è un sensore polare (singolo emettitore, risoluzione angolare fissa): un ostacolo vicino riceve molti più punti di uno lontano, a parità di dimensione fisica. Con una soglia fissa si rischia di "perdere" oggetti lontani o di essere troppo permissivi su quelli vicini.

La soluzione è normalizzare rispetto alla **copertura angolare** della cella:

$$\bar{b}_{n,m} = \alpha(n, m)\, \delta, \tag{4.7}$$

dove $\alpha(n, m)$ è la copertura angolare della cella (n, m) vista dal sensore e δ è la densità angolare di punti del LiDAR. In pratica: per celle vicine serve una soglia maggiore (più punti attesi), per celle lontane la soglia si abbassa automaticamente.

4.4.4 Background Removal

Prima di fare tracking bisogna capire *cosa* tracciare; l'approccio più semplice è rilevare ciò che si muove, rimuovendo lo sfondo (le parti della griglia che non cambiano nel tempo). Si usa una **moving window average**:

$$G(t) = \frac{\sum_{\tau=0}^{W} \tilde{B}(t-\tau)}{W}, \tag{4.8}$$

seguita da binarizzazione:

$$\tilde{g}_{n,m} = \begin{cases} 1 & \text{se } g_{n,m} > \tau_{bck} \\ 0 & \text{altrimenti} \end{cases} \tag{4.9}$$

Interpretazione visiva a tre livelli: **nero** — nessun ostacolo; **grigio** — background (qualcosa sempre presente); **bianco** — oggetti in movimento (quello che si vuole tracciare).

Un problema residuo è che oggetti al confine tra "movimento" e "stazionario" possono sopravvivere erroneamente al filtro (es. un'auto parcheggiata che si è appena mossa lascia residui) — da qui la necessità di un ulteriore step di denoising.

4.4.5 Denoising

Due sotto-fasi: *image processing* e *shadow management*.

4.4.5.1 Image processing (morfologia matematica)

L'obiettivo è duale: rimuovere pixel rumorosi isolati e aggregare pixel sparsi appartenenti allo stesso oggetto.

1. **Background dilation**: si applica un template (es. 3×3) e, ogni volta che il suo centro coincide con un 1, si riempiono di 1 tutti i punti coperti — questo "gonfia" gli oggetti, riempiendo buchi e unendo punti vicini in un unico blob.
2. **Closure** (chiusura) = dilation + erosion: la dilation da sola tende a far crescere troppo gli oggetti; la chiusura aggiunge un'erosione per ripulire il blob, mantenendo l'aggregazione ottenuta.
3. **Area thresholding**: se un blob sopravvissuto è troppo piccolo (probabilmente rumore residuo), viene eliminato.

4.4.5.2 Shadow management

Quando un oggetto in movimento passa davanti a qualcosa, lascia dietro di sé un'area che il LiDAR non riesce più a vedere (un'"ombra" nella scansione). L'idea è:

1. per ogni oggetto dinamico già tracciato, si stima e proietta un cono d'ombra;
2. i pixel che cadono in quest'ombra vengono tenuti costanti finché non tornano visibili;
3. si assume che lo sfondo ombreggiato non cambi durante il passaggio dell'ombra.

È un processo **iterativo**: per sapere quale ombra proiettare serve che l'oggetto sia già in movimento/già tracciato.

4.5 Tracking*

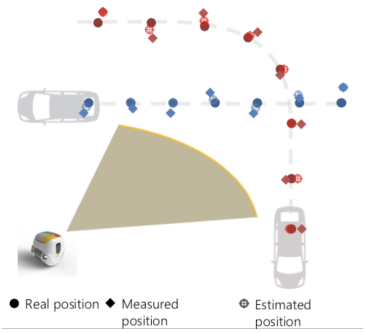
The tracking module uses the Two-Level-Algorithm

4.5.1 Multi-target Tracking *

Il Multi-Target Tracking (MTT) è il processo di **rilevare, identificare e seguire contemporaneamente più oggetti dinamici presenti nell'ambiente circostante** il veicolo nel tempo. In pratica, il veicolo autonomo non deve solo sapere che cosa c'è intorno a sé (pedoni, auto, biciclette, motocicli, ecc.), ma deve anche capire dove si trovano, come si stanno muovendo e come evolveranno le loro traiettorie.

4.5.1.1 Le due sotto-sfide

1. **Track**: stimare lo stato (posizione, velocità) di ogni oggetto nel tempo;
2. **Riconoscere/associare**: capire se due blob rilevati in istanti diversi appartengono allo stesso oggetto fisico.



4.5.1.2 Setup del problema base

Ipotesi di partenza (caso "facile"):

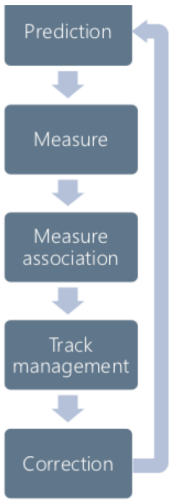
1. il numero di target è **sconosciuto**;
2. ogni target genera **al massimo una misura** per istante (assunzione semplificativa, rimossa più avanti nell'Extended Target Tracking in Sec 4.5.2).

Problemi pratici: non si ha accesso alla posizione reale, solo a quella **misurata** (soggetta a rumore); inoltre le misure **non hanno un ID/colore** — il LiDAR non dice a quale oggetto appartiene un dato punto, va dedotto seguendo la traiettoria nel tempo.

Il caso critico è quello delle **intersezioni**: quando due traiettorie si incrociano, la sola distanza spaziale non basta più a distinguere quale misura appartenga a quale track. Serve un'informazione aggiuntiva — la velocità stimata che ciascun track aveva prima di entrare nella zona critica — per predire dove ci si aspetta che ciascun oggetto sia, e scegliere l'associazione più compatibile con questa predizione.

4.5.1.3 Il ciclo del Kalman Filter

Lo schema concettuale centrale di tutta la parte di tracking è il seguente ciclo.



- **Prediction**: si usa un modello dinamico per prevedere dove sarà ciascun target al prossimo istante;
- **Measure**: arrivano le nuove misure (blob) dal sensore;
- **Measure Association**: si decide quale misura appartiene a quale track, confrontando predizione e misura;
- **Track management**: si gestiscono nascita e morte dei track (è entrato un nuovo oggetto? un track è sparito?);
- **Correction**: si aggiorna la stima del track con la misura associata (classico update del Kalman Filter).

Questo è un approccio *model-based*: richiede un modello dinamico per fare la predizione (verrà trattato nella sezione sul Low Level Tracking).

4.5.2 Extended Target Tracking *

Il tracker appena descritto assume che ogni oggetto generi al massimo una misura. In realtà un singolo oggetto fisico (es. un'auto) può generare **più blob/misure** contem-

poraneamente. Bisogna quindi anche capire quali blob appartengano allo stesso oggetto fisico — problema noto come **Extended Target Tracking**.

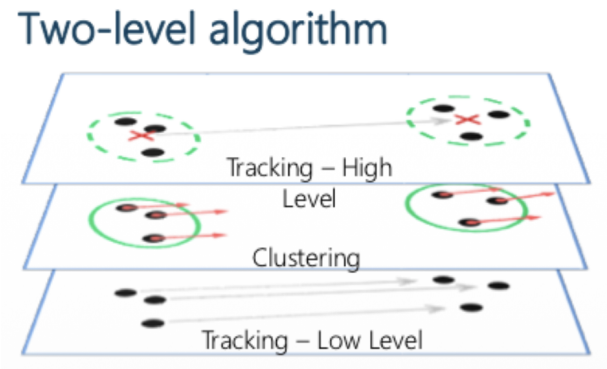
- **Idea 1 — solo distanza**: se due blob sono vicini, probabilmente sono lo stesso oggetto. Il problema è che se due oggetti diversi si muovono vicini ma in direzioni diverse, la sola distanza spaziale non li distingue e si rischia di fonderli erroneamente in un unico cluster.
- **Idea 2 — posizione e velocità**: si fa clustering basandosi anche sulla velocità. Due blob vicini ma con velocità molto diverse restano separati.

Il problema è che, per clusterizzare in base alla velocità, serve prima avere una stima di velocità per ogni singolo punto/blob — da qui la necessità di un'architettura a due livelli.

4.6 Architettura a due livelli

Come detto precedentemente, l'architettura a due livelli utilizzata dal tracking predilige il multi-target tracking per il primo e ultimo livello, mentre l'extended-target tracking per il livello centrale:

- **Low Level**: tratta ogni punto/blob come se fosse un oggetto a sé, applicando il ciclo Kalman completo. L'output include, per ogni blob, una stima di posizione *e* velocità;
- **Clustering**: usa posizione e velocità stimate per raggruppare i blob dello stesso oggetto fisico (double-threshold clustering, vedi oltre);
- **High Level**: applica di nuovo lo stesso schema Kalman, questa volta sui cluster, fornendo la stima finale (una posizione + velocità per ogni oggetto fisico).



4.6.1 Low Level Tracking — la Prediction

4.6.1.1 Scelta del modello dinamico

Per fare prediction serve un modello dello stato (posizione + velocità). I modelli "classici" non vanno bene per questo contesto:

- **Modello cinematico del corpo rigido**: richiede in ingresso le accelerazioni, che non sono disponibili per un veicolo esterno ⇒ non utilizzabile;
- **Bicycle model**: assume velocità costante e richiede in ingresso l'angolo di sterzo, non disponibile per un veicolo esterno ⇒ non utilizzabile;
- **Bicycle model dinamico**: richiederebbe anche i parametri fisici del veicolo altrui (massa, inerzie) — impossibile da conoscere in pratica.

4.6.1.2 Coordinated Turn Motion Model

Si sceglie il modello più generale possibile nel piano: il **coordinated turn model**, un modello cinematico che assume **yaw rate costante** (ω costante).

È utile sottolineare un punto filosofico tipico dell'approccio Kalman: si scrive l'equazione dinamica *assumendo* ω costante ($\dot{\omega} = 0$), ma allo stesso tempo si tratta ω come **incognito** — fa parte dello stato da stimare, non è un parametro noto a priori. La "costanza" è solo un'assunzione di modello (con rumore), non un valore fissato. Analogamente si assume velocità costante in modulo ($d|v|/dt = 0$).

Lo stato è

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ v_x \\ v_y \\ \omega \end{bmatrix}, \tag{4.10}$$

con equazioni continue

$$\dot{x} = v_x, \quad \dot{y} = v_y, \quad \dot{v}_x = -\omega v_y, \quad \dot{v}_y = \omega v_x, \quad \dot{\omega} = 0. \tag{4.11}$$

Derivazione di $\dot{v}_x$, $\dot{v}_y$.

Nel riferimento inerziale, scrivendo $v_x = v\cos(\psi)$, $v_y = v\sin(\psi)$ (dove ψ è l'angolo di heading e v il modulo della velocità), derivando rispetto al tempo e usando l'assunzione

di modulo costante ($\dot{v} = 0$):

$$\dot{v}_x = -v\sin(\psi)\dot{\psi} \;\Rightarrow\; \dot{v}_x = -v_y\omega, \qquad \dot{v}_y = v\cos(\psi)\dot{\psi} \;\Rightarrow\; \dot{v}_y = v_x\omega, \tag{4.12}$$

dove $\dot{\psi} = \omega$ è proprio lo yaw rate. Il termine "costante" non si perde nella derivazione: resta solo il termine di rotazione, perché il modulo non cambia.

4.6.1.3 Discretizzazione esatta

Integrando il sistema da k a $k+1$ (durata T) si ottiene una discretizzazione **esatta** (non un'approssimazione ai primi ordini come farebbe Eulero).

Righe 3 e 4 (velocità).

L'integrazione di $\dot{v}_y = -v_x\omega$, $\dot{v}_x = -v_y\omega$ è formalmente una rotazione:

$$\begin{bmatrix} v_{x,k+1} \\ v_{y,k+1} \end{bmatrix} = \begin{bmatrix} \cos(\omega_k T) & -\sin(\omega_k T) \\ \sin(\omega_k T) & \cos(\omega_k T) \end{bmatrix} \begin{bmatrix} v_{x,k} \\ v_{y,k} \end{bmatrix}. \tag{4.13}$$

Ha senso: se il modulo della velocità è costante e sta solo "ruotando" con tasso ω, il vettore velocità dopo un tempo T è semplicemente il vettore iniziale ruotato di ωT.

Righe 1 e 2 (posizione).

Integrando la velocità nel tempo:

$$x_{k+1} = x_k + \frac{\sin(\omega_k T)}{\omega_k} v_{x,k} + \frac{\cos(\omega_k T) - 1}{\omega_k} v_{y,k}, \tag{4.14}$$

$$y_{k+1} = y_k + \frac{1 - \cos(\omega_k T)}{\omega_k} v_{x,k} + \frac{\sin(\omega_k T)}{\omega_k} v_{y,k}. \tag{4.15}$$

Riga 5 (yaw rate).

Banalmente $\omega_{k+1} = \omega_k$ (costante per assunzione).

La matrice di rumore

La matrice G_k rappresenta come una piccola accelerazione non modellata (rumore di processo) influenzi lo stato. Si deriva assumendo un modello *diverso e più semplice*: che il rumore sia un'accelerazione costante applicata durante l'intervallo T (modello standard

ad accelerazione costante):

$$\mathbf{w}_k = [a_x, a_y, \alpha]^T. \tag{4.16}$$

Per la componente x (analogamente per y): con $\dot{x} = v_x$, $\dot{v}_x = a_x$ costante, integrando:

$$v_{k+1} = v_k + a_x T, \qquad x_{k+1} = x_k + v_k T + \frac{1}{2} a_x T^2. \tag{4.17}$$

Stessa logica per ω con accelerazione angolare α: $\omega_{k+1} = \omega_k + \alpha T$.

4.6.1.4 Equazione di output

Ciò che si **misura** realmente è solo la **posizione** (l'output della detection restituisce solo il centro/posizione di ogni blob, non la velocità!). Dato che x e y sono già componenti dello stato:

$$\mathbf{y}_k = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} \mathbf{x}_k + \mathbf{v}_k, \qquad \mathbf{v}_k \sim \mathcal{N}(0, \mathbf{R}). \tag{4.18}$$

Questo è il punto chiave del Kalman filter applicato al tracking: si stima una velocità che non viene mai misurata direttamente, solo grazie al modello dinamico che lega posizione e velocità nel tempo.

4.6.1.5 Measure Association*

Dopo la prediction si hanno, per ogni track, una stima predetta; arrivano poi le nuove misure (blob). Bisogna capire quale misura corrisponde a quale track. Il processo si articola in due fasi: **gating** e **association**.

Gating

L'idea è che, se due misure sono troppo distanti da una predizione, è impossibile che siano dello stesso track: si scartano a priori, riducendo il problema combinatorio.

Perché non si usa semplicemente la distanza euclidea? Perché non tiene conto dell'**incertezza**. La matrice di covarianza dell'errore indica quanto ci si fida della propria stima: se la si visualizza come un'ellisse, l'incertezza lungo x e y (i semiassi) spesso non è uguale nelle due direzioni. Due punti alla stessa distanza euclidea da una predizione possono avere probabilità molto diverse di essere la misura corretta, a seconda di dove cade l'ellisse di incertezza.

Distanza di Mahalanobis.

È la distanza euclidea scalata e ruotata secondo la matrice di covarianza:

$$D_{n,m} = \sqrt{(\mathbf{y}_m - \hat{\mathbf{y}}_n)^T S^{-1} (\mathbf{y}_m - \hat{\mathbf{y}}_n)}, \tag{4.19}$$

dove $\mathbf{y}_m$ è la posizione misurata, $\hat{\mathbf{y}}_n$ la posizione predetta (a priori, prima della correzione) e

$$S = HP'H^T + R \tag{4.20}$$

è la covarianza dell'innovazione (combina l'incertezza sulla predizione P' e quella della misura R). Invece di misurare distanza su uno spazio dove le curve di livello sono cerchi (euclidea), si misura distanza rispetto a un'ellisse — quindi è possibile usare un'unica soglia τ_G valida in tutti i casi, perché la scala è già "normalizzata" dall'incertezza:

$$d_{n,m} = \begin{cases} D_{n,m} & \text{se } D_{n,m} \leq \tau_G \\ \infty & \text{altrimenti} \end{cases} \tag{4.21}$$

Se la distanza supera la soglia, il peso dell'associazione diventa infinito e l'associazione viene scartata.

Association

Dopo il gating restano possibili associazioni multiple: più di una misura può soddisfare il gating per lo stesso track, e viceversa. Bisogna risolvere un problema di **assegnazione** su un grafo bipartito (misure ↔ track), dove ogni arco ha un peso pari alla distanza di Mahalanobis.

Approccio greedy.
Si prende l'associazione con costo minimo, la si "congela", si passa alla coppia disponibile successiva con costo minimo, e così via. Il problema è che il greedy **non è garantito ottimo**: bloccando presto un'associazione a basso costo locale, può costringere le associazioni rimanenti a costi molto alti (un esempio tipico mostra un costo totale di 311 con il greedy contro un ottimo di 90).

Algoritmo Ungherese (Hungarian) / Munkres.
Risolve il problema di assegnazione in **tempo polinomiale**, minimizzando il costo totale su tutte le associazioni contemporaneamente (non solo localmente). Lo stesso schema, noto come **GNN — Global Nearest Neighbor**, verrà riusato più avanti per la fusione camera-LiDAR.

4.6.1.6 Track Management

Dopo l'associazione, una misura non associata a nessun track esistente genera un **nuovo track tentativo**, tenuto in una sorta di limbo. Le regole (a stati finiti) sono:

- **Tentative**: $N_{LiDAR} = 1$ AND $N_{Cam} = 1$ (track appena creato);
- **Accepted**: $N_{LiDAR} > M_{accept}$ OR $N_{Cam} > M_{accept}$;
- **Confirmed**: $N_{LiDAR} > M_{conf}$ AND $N_{Cam} > M_{conf}$;
- **Dead**: $N_{LiDAR} < M_{elim}$ OR $N_{Cam} < M_{elim}$ (oppure numero massimo di iterazioni raggiunto senza conferma).

In sintesi: se in N istanti un track tentativo accumula almeno M misure, viene **confermato** ed entra a far parte dell'output ufficiale; se un track (anche confermato) accumula meno di M misure nelle ultime N istanti, viene **eliminato**. Alla fine ogni track ha un ID persistente e un vettore che ne rappresenta la velocità stimata.

4.6.2 Double-Threshold Clustering

Una volta che il Low Level tracker fornisce, per ogni blob, posizione e velocità stimate, si può fare clustering (per l'Extended Target Tracking) usando due soglie:

$$\delta_d(p, C) = \min_{q \in C} \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2} \quad \text{(soglia di posizione)}, \tag{4.22}$$

$$\delta_v(p, C) = \max_{q \in C} \sqrt{(v_{x,p} - v_{x,q})^2 + (v_{y,p} - v_{y,q})^2} \quad \text{(soglia di velocità)}. \tag{4.23}$$

Algoritmo:

1. si seleziona casualmente un track come "seme" del primo cluster C_1;
2. per ogni track non ancora clusterizzato, si calcolano δ_d e δ_v rispetto al cluster: se entrambe le distanze sono sotto le rispettive soglie, il punto viene incluso nel cluster;
3. se restano track non clusterizzati, se ne sceglie uno nuovo come seme del cluster successivo e si ripete.

Usando sia posizione che velocità, oggetti vicini ma con moto diverso vengono correttamente separati.

4.6.3 High Level Tracking

Si applica lo stesso identico schema del Kalman filter (coordinated turn model, gating con Mahalanobis, GNN/Hungarian per l'associazione, gestione M/N dei track), ma ai **cluster** invece che ai singoli blob: la "misura" è ora la posizione media del cluster, e la velocità considerata è quella aggregata dal Low Level.

4.6.3.1 Drawback residuo.

Resta un problema: il punto misurato dal LiDAR si sposta rispetto all'auto man mano che cambia la prospettiva (il LiDAR misura solo la superficie più vicina dell'oggetto, non il suo "vero" centro geometrico fisso). Di conseguenza, la velocità stimata non è mai perfettamente accurata.

4.6.3.2 Soluzione parziale.

Dato che la forma dell'oggetto (es. un'auto) è approssimabile a un rettangolo, si stima il **bounding box** e si usa il suo centroide invece del punto grezzo più vicino. La pipeline aggiornata è:

1. Ground removal (grid-based, guardando la differenza di altezza massima in ogni cella);
2. Bounds removal (map-based, usando la stima corrente di localizzazione);
3. Measures extraction: chiusura morfologica per connettere i pixel dell'oggetto ed estrarre un bounding box $\Rightarrow$ centroide più stabile.

4.7 Sensor Fusion con la Camera

Fin qui si è usato solo il LiDAR. Vediamo ora come integrare anche la camera nello stesso paradigma tracking-by-detection.

4.7.1 Perché la camera aiuta

Con solo LiDAR, decidere cosa tracciare richiedeva affidarsi al background removal (rimozione dello sfondo statico) — approccio che **non funziona bene se il veicolo ego si muove** (perché "sfondo" e "movimento" diventano relativi al moto del sensore stesso, e tutto sembra muoversi). Con la camera si può invece decidere cosa tracciare basandosi sulla detection visiva ("questo è un veicolo"), indipendentemente dal fatto che si stia muovendo o meno rispetto allo sfondo.

4.7.2 Euclidean Clustering

Con la camera a disposizione, la pipeline LiDAR si semplifica: non serve più tarare finestre di background removal. Basta:

1. rimuovere il terreno (ground removal);
2. fare **clustering euclideo**: connettere punti vicini tra loro in gruppi (cluster) — questo raggruppa naturalmente guardrail, oggetti, elementi fuori pista, ecc., in base alla vicinanza spaziale.

L'algoritmo di Euclidean Clustering prende in input i punti non-ground e restituisce in output una lista di oggetti rilevati, ciascuno con centroide (X, Y) e convex hull (nel riferimento del veicolo). Questi cluster sono **candidati**: sarà poi la camera a decidere cosa vale effettivamente la pena tracciare.

4.7.3 DD3D

Per la parte camera si usa **DD3D**, una rete neurale complessa (usata come black-box, pre-trainata su KITTI) capace di stimare direttamente la posa 3D a partire da immagini monoculari.

Perché DD3D.

Stima la posizione 3D da camera monoculare e facilita l'associazione con i dati LiDAR (entrambi in "spazio 3D").

Parametri di calibrazione necessari.

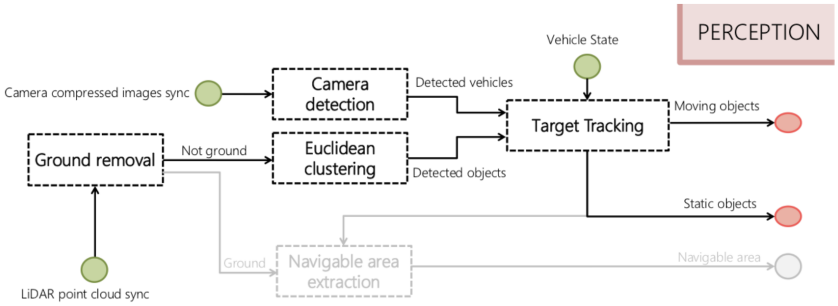
- **Intrinseci camera**: per raddrizzare (rectify) l'immagine e rendere DD3D "camera-aware";
- **Estrinseci camera**: per trasformare i bounding box dal riferimento camera al riferimento veicolo (LiDAR e camera vivono in riferimenti diversi, vanno riportati a un riferimento comune);
- **Image ROI**: per processare solo la porzione utile dell'immagine.

Limite di DD3D.

L'orientamento stimato non è perfetto (auto e bounding box non sempre perfettamente allineati). Non è quindi affidabile da solo per il tracking, ma resta un ottimo strumento

per selezionare cosa tracciare e fare fusione con LiDAR.

4.7.4 Architettura di fusione



Se la camera identifica direttamente "questo è un veicolo" con un bounding box, si può usare questa informazione per raggruppare tutti i punti LiDAR appartenenti allo stesso oggetto fisico — non serve più il clustering "cieco" basato solo su posizione/velocità dei singoli punti, e quindi non serve più l'Extended Target Tracking descritto in precedenza.

4.7.5 Struttura del track finale

Misure (input, poco strutturate):

- non ordinate, lista non consistente (una misura n non corrisponde sempre allo stesso oggetto fisico da un frame all'altro);
- osservabilità parziale (LiDAR e camera non misurano direttamente la velocità);
- possibili falsi positivi;
- rilevazioni mancate (occlusioni, misure asincrone).

Track (output, strutturato):

- ordinati e consistenti — ogni track corrisponde sempre allo stesso oggetto fisico;
- stima completa dello stato (inclusa la velocità, mai misurata direttamente);
- rigetto di falsi positivi e falsi negativi ("l'oggetto c'è anche se non lo vedi in questo istante", grazie alla predizione).

Per ogni track si usa un EKF con

$$X = [x, y, \psi, v, \omega] \quad \text{(riferimento locale)}, \tag{4.24}$$

usando come misura i centroidi dei cluster: $Y = [x, y]$ (riferimento locale). Il sistema realizzato è quindi un **EKF Multi-Target Tracking** che effettua **track-level fusion** tra camera e LiDAR, con gestione M/N dei track e una **macchina a stati finiti (FSM)** che determina nascita e morte dei track.

4.8 Sistema di riferimento: globale vs locale

Un ultimo punto concettuale, spesso trascurato ma cruciale quando il veicolo ego si muove.

4.8.1 Definizioni

- **Riferimento globale**: fisso rispetto al mondo. $\dot{x}, \dot{y}, \dot{v}_x, \dot{v}_y$ si riferiscono alle coordinate globali dell'oggetto.
- **Riferimento locale**: solidale al veicolo ego. $\dot{x}, \dot{y}, \dot{v}_x, \dot{v}_y, \omega$ rappresentano la posizione *relativa* tra target ed ego. Se il moto è espresso in locale, il modello deve includere anche informazioni sul moto del veicolo ego stesso.

4.8.2 Perché con ego fermo non importava

Nel caso iniziale (veicolo ego fermo, es. per decidere quando attraversare), locale e globale coincidono e non c'era alcuna ambiguità.

4.8.3 Perché con ego in movimento importa

Il coordinated turn model derivato in precedenza è scritto nel **riferimento globale** — non contiene alcuna informazione sulla velocità dell'ego veicolo (è "cieco" rispetto al moto proprio).

- **Vantaggio del riferimento globale**: il modello è semplice, scritto direttamente nel riferimento inerziale.
- **Svantaggio**: ogni volta che arriva una misura (naturalmente espressa nel riferimento *locale* del sensore), va trasformata in globale prima di poterla usare — serve quindi conoscere con precisione la trasformazione (posa dell'ego veicolo) in ogni istante.

4.8.4 Lavorare in locale

Si preferisce riscrivere tutto in coordinate locali (relative all'ego veicolo). Le equazioni EKF diventano:

$$T_{(k)}^{(k+1)} = \begin{bmatrix} \cos(\Delta T \omega_{ego}) & -\sin(\Delta T \omega_{ego}) & \Delta T v_{x,ego} \\ \sin(\Delta T \omega_{ego}) & \cos(\Delta T \omega_{ego}) & \Delta T v_{y,ego} \\ 0 & 0 & 1 \end{bmatrix}. \tag{4.25}$$

Questa matrice di trasformazione (dipendente dal moto proprio dell'ego veicolo) viene applicata per riportare lo stato predetto dal riferimento locale al tempo k al riferimento locale al tempo $k+1$ (che si è spostato insieme all'ego).

Equazioni EKF standard.

$$\hat{X}(t) = f(X(t-1)) \qquad \text{(prediction del modello)} \tag{4.26}$$

$$\hat{Y}(t) = H\hat{X}(t) \qquad \text{(misure predette)} \tag{4.27}$$

$$P(t) = A(t)P(t-1)A(t)^T + Q \qquad \text{(covarianza predetta)} \tag{4.28}$$

$$S(t) = HP(t)H^T + R \qquad \text{(covarianza dell'innovazione)} \tag{4.29}$$

dove $A(t)$ è lo jacobiano delle equazioni di stato (linearizzazione, tipico dell'Extended Kalman Filter):

$$A_{i,j} = \frac{dX_i}{dX_j}. \tag{4.30}$$

4.8.5 Frequenza di esecuzione

- Esecuzione dell'algoritmo: 33 Hz (fissa);
- LiDAR (dopo rimozione ground): 10 Hz;
- Camera (DD3D): $\approx$ 10 Hz.

Dato che l'esecuzione è più veloce dei sensori, non tutti i cicli hanno nuove misure: in quei cicli si esegue solo lo step di **EKF Prediction** in modalità *open loop* (senza correzione), aggiornando comunque la stima in base al solo modello dinamico, fino a quando non arriva una nuova misura da fondere.

5 | Control

5.1 Path e traiettoria

Prima di affrontare il controllo vero e proprio, è necessario chiarire un punto concettuale importante: la differenza fra *path* e *traiettoria*. Non è un dettaglio terminologico, perché a seconda di quale dei due oggetti si sta inseguendo cambia il tipo di errore che il controllore deve annullare.

- **Path**: una sequenza di punti o di pose *senza* associazione temporale. È un oggetto puramente geometrico: descrive *dove* deve passare il veicolo, non *quando*.
- **Traiettoria**: una sequenza di pose *con* associazione temporale (o, equivalentemente, un path a cui è stato aggiunto un profilo di velocità).

Questa distinzione si traduce direttamente in due problemi di controllo diversi:

- **Path tracking**: l'obiettivo è che il veicolo (EGO) non si allontani mai troppo dal path. Non importa *quando* il veicolo si trova in un certo punto, importa solo che stia vicino alla curva di riferimento.
- **Trajectory tracking**: l'obiettivo è che il veicolo si trovi in una posizione precisa in un istante preciso. Qui il riferimento temporale è vincolante.

Dal punto di vista matematico, un path è descritto da una terna di stati $[x\ y\ \theta]$ campionata N volte (una matrice $N \times 3$), mentre una traiettoria aggiunge la dimensione del tempo o, equivalentemente, della velocità: $[x\ y\ \theta\ t]$ oppure $[x\ y\ \theta\ v]$. Nella maggior parte dei casi pratici non si lavora con un'espressione analitica continua del percorso ($f(x, y) = 0$), ma con un *path campionato*: una sequenza di punti presi a intervalli regolari lungo il tracciato di interesse.

5.1.1 Curvatura

La curvatura è una proprietà del *path*, non della traiettoria (non ha senso parlare di "curvatura nel tempo": la curvatura descrive quanto una curva si piega nello spazio,

indipendentemente da quando il veicolo la percorre). Geometricamente, la curvatura in un punto è definita a partire dal raggio del cerchio osculatore (il cerchio tangente alla curva in quel punto): più piccolo è il raggio, maggiore è la curvatura.

Data una parametrizzazione del path rispetto all'ascissa curvilinea s,

$$\gamma(s): \quad \begin{cases} x = x(s) \\ y = y(s) \end{cases}$$

la curvatura si calcola come

$$k = \frac{x'y'' - y'x''}{(x'^2 + y'^2)^{3/2}}$$

dove le derivate sono fatte rispetto all'ascissa curvilinea s.

Perché la curvatura è così importante dal punto di vista del controllo? Perché lega direttamente la geometria del percorso alla dinamica del veicolo: una curvatura elevata significa che il veicolo deve cambiare rapidamente orientamento per rimanere sul path, e se questo cambiamento è anche *rapido nello spazio* (curvatura che varia velocemente da un punto all'altro), il comfort di guida ne risente. Da questa osservazione nascono due criteri di valutazione di un percorso:

- **Comfort**: curvature piccole e che variano lentamente sono più confortevoli, perché comportano cambi di orientamento del veicolo più dolci.
- **Prestazioni**: un percorso a bassa curvatura può essere percorso a velocità più alta senza superare i limiti di accelerazione laterale del veicolo.

5.2 Lateral Control

A questo punto ci mettiamo nei panni del progettista del controllo: assumiamo che percezione, localizzazione e planning siano già risolti (cioè che il path/traiettoria di riferimento sia disponibile), e ci concentriamo sul modulo che decide sterzo, freno e acceleratore per seguire quel riferimento. In particolare, in questo capitolo ci occupiamo del **controllo laterale**, cioè di come calcolare **l'angolo di sterzo** δ.

Esistono tre grandi famiglie di approcci al path/trajectory tracking:

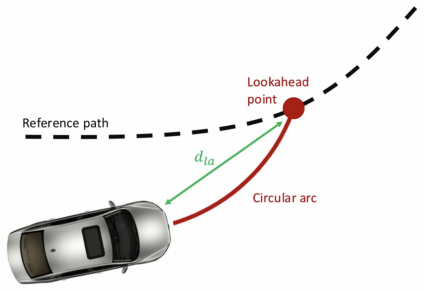
- **Geometrico**: molto comune in robotica, meno adatto ai veicoli stradali. Funziona bene quando il veicolo si muove lentamente. L'angolo di sterzo viene calcolato con considerazioni puramente geometriche sul percorso e sulla posizione del veicolo.
- **Cinematico**: più comune nella guida autonoma. Si basa su un modello cinematico

del veicolo (senza slittamento laterale delle ruote) ed è adatto a guida a velocità contenuta e pressoché costante.

- **Dinamico**: si basa su un modello dinamico del veicolo (che tiene conto delle forze di interazione pneumatico-strada), ed è quindi in grado di gestire manovre più aggressive e ad alta velocità.

5.2.1 L'approccio geometrico: Pure Pursuit

L'idea del **Pure Pursuit** è semplice e intuitiva: si sceglie un punto sul path di riferimento posto a una certa distanza in avanti rispetto al veicolo (il *lookahead point*, a distanza d_{la}), e si calcola l'arco di cerchio che collega la posizione attuale del veicolo a quel punto. L'angolo di sterzo viene poi determinato in modo da far percorrere al veicolo esattamente quell'arco.



1. Si collega la posizione del veicolo al lookahead point tramite un arco di cerchio.
2. Si calcola l'angolo di sterzo geometrico necessario per seguire quell'arco.

Il pregio principale di questo metodo è che sfrutta la conoscenza *a priori* del percorso (il lookahead point è un'informazione futura rispetto alla posizione attuale). Il limite, però, è altrettanto importante: è un metodo puramente **euristico**, che non tiene in alcun conto le dinamiche del veicolo né la stabilità ad anello chiuso. In altre parole, l'azione di controllo non nasce da un ragionamento di retroazione formale: non c'è alcuna garanzia teorica di stabilità. Il Pure Pursuit **funziona bene solo per veicoli molto lenti**, dove il moto può essere approssimato come una successione di configurazioni statiche. Per un veicolo autonomo che si muove a velocità apprezzabili, questa ipotesi non è più accettabile, ed è per questo che ci si sposta verso approcci basati su modelli cinematici o dinamici.

5.2.2 L'approccio cinematico

5.2.2.1 Il modello cinematico a bicicletta

Il punto di partenza dell'approccio cinematico è il **modello cinematico a bicicletta**, che si basa su due ipotesi fondamentali:

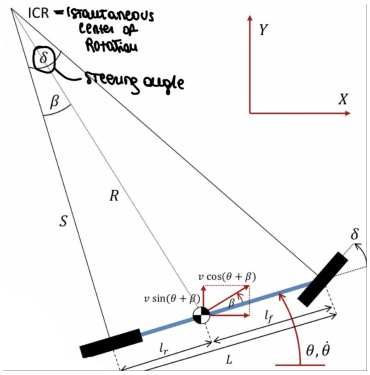
- la velocità v (misurata al baricentro, COG) è costante;

- non c'è angolo di slittamento laterale (*side slip angle*) alle ruote: le ruote puntano localmente nella stessa direzione in cui si muove il veicolo.

Con queste ipotesi, l'equazione del moto in coordinate globali (X, Y, θ) diventa una funzione delle sole variabili di stato $[X\ Y\ \theta]^T$ e dell'ingresso di controllo, l'angolo di sterzo δ:

$$\dot{x} = f\left(\begin{bmatrix} X \\ Y \\ \theta \end{bmatrix}, \delta\right)$$

Dal punto di vista geometrico, il centro di istantanea rotazione (ICR) del veicolo permette di ricavare la relazione tra l'angolo di sterzo e l'angolo di assetto β (attitude angle), grazie a considerazioni puramente geometriche sul triangolo formato da assale anteriore, posteriore e centro di rotazione:



$$S = \frac{L}{\tan(\delta)}, \qquad R = \frac{S}{\cos\beta} = \frac{L}{\tan\delta\cos\beta}$$

$$\tan\beta = \frac{l_r \tan\delta}{L} \quad \Rightarrow \quad \beta = \operatorname{atan}\left(\frac{l_r}{L}\tan\delta\right)$$

dove L è il passo del veicolo e l_r la distanza fra l'assale posteriore e il centro di gravità.

5.2.2.2 Perché questo modello, da solo, non basta

Il modello appena scritto restituisce $\dot{X}, \dot{Y}, \dot{\theta}$, cioè descrive dove si trova il veicolo in un sistema di riferimento globale. Ma il nostro obiettivo non è "sapere dove si trova il veicolo in assoluto": è **seguire un path**, cioè conoscere quanto il veicolo si discosta da quel path. Se provassimo a impostare il problema di controllo direttamente in coordinate globali, ci troveremmo davanti a un problema mal posto: vorremmo inseguire una coppia di riferimenti (x_0, y_0) con un solo ingresso di controllo (δ). È un problema **SIMO** (Single Input, Multiple Output): un solo ingresso δ contro (almeno) due uscite $\{x, y\}$.

Il punto chiave è che x e y non sono in realtà due gradi di libertà indipendenti: sono *correlati* dal fatto che il veicolo deve muoversi lungo un percorso ben preciso. Non è quindi un vero problema a due gradi di libertà. Conviene allora riformulare il problema: invece di inseguire x e y separatamente, conviene controllare la **distanza del veicolo dal path**. In questo modo il problema torna a essere SISO (Single Input, Single Output): un solo ingresso (δ) e una sola uscita (l'errore rispetto al path).

5.2.2.3 Cambio di coordinate: dal riferimento globale al riferimento del path

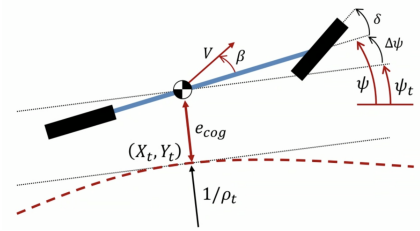
Per formalizzare questa idea, si passa da una descrizione in coordinate globali a una descrizione in **coordinate del path**, definendo due nuove variabili di stato:

- e_{cog}: errore laterale al baricentro, cioè la distanza (con segno) fra il baricentro del veicolo e il punto più vicino sul path;
- $\Delta\psi$: errore di orientamento, cioè la differenza fra l'orientamento del veicolo e la direzione tangente al path nel punto più vicino.

Per ogni punto del path si definiscono le variabili ausiliarie X_t, Y_t (coordinate globali del punto più vicino sul path), ψ_t (orientamento del path in quel punto, cioè direzione della tangente) e ρ_t (curvatura del path in quel punto). Curvatura e orientamento del path di riferimento vengono presi *nel punto più vicino al baricentro del veicolo*, lavorando nell'ipotesi di spazio euclideo, per cui il vettore lungo cui si misura la distanza è perpendicolare al path nel punto più vicino.

Questo cambio di variabili si basa su due assunzioni di validità del modello:

$$L \ll \frac{1}{\rho_t}, \qquad e_{cog} \ll \frac{1}{\rho_t}$$



cioè: la lunghezza del veicolo deve essere piccola rispetto al raggio di curvatura locale del path, e l'errore laterale (che vogliamo comunque portare a zero) deve essere piccolo rispetto allo stesso raggio.

5.2.2.4 Derivazione delle equazioni in coordinate del path

L'orientamento del path cambia, lungo il percorso, con una velocità angolare pari a $\rho_t \cdot v$: se il veicolo seguisse perfettamente il path a velocità v, il suo yaw rate sarebbe proprio $\rho_t v$. Di conseguenza, la derivata dell'errore di orientamento è:

$$\Delta\dot{\psi} = r - \dot{\psi}_t \cong r - \rho_t V$$

dove r è lo yaw rate del veicolo.

Analogamente, la derivata dell'errore laterale al baricentro è data dalla componente della velocità proiettata lungo la direzione perpendicolare al path:

$$\dot{e}_{cog} \cong V \sin(\beta + \Delta\psi) \cong V(\beta + \Delta\psi)$$

dove nell'ultimo passaggio si è usata l'approssimazione per piccoli angoli.

A questo punto si sostituiscono le relazioni del modello cinematico a bicicletta (anch'esse linearizzate per piccoli angoli di sterzo):

$$\beta = \operatorname{atan}\left(\frac{l_r}{L}\tan\delta\right) \cong \frac{l_r}{L}\delta, \qquad r = \frac{V}{L}\tan\delta\cos\beta \cong \frac{V}{L}\delta$$

e si ottiene il **modello cinematico linearizzato in coordinate del path**:

$$\Delta\dot{\psi} = \frac{V}{L}\delta - \rho_t V, \qquad \dot{e}_{cog} = V\left(\frac{l_r}{L}\delta + \Delta\psi\right)$$

che, in forma di stato, diventa:

$$\begin{bmatrix}\Delta\dot{\psi}\\ \dot{e}_{cog}\end{bmatrix} = \begin{bmatrix}0 & 0\\ V & 0\end{bmatrix}\begin{bmatrix}\Delta\psi\\ e_{cog}\end{bmatrix} + \begin{bmatrix}V/L\\ V\,l_r/L\end{bmatrix}\delta + \begin{bmatrix}-V\\ 0\end{bmatrix}\rho_t, \qquad e_{cog} = \begin{bmatrix}0 & 1\end{bmatrix}\begin{bmatrix}\Delta\psi\\ e_{cog}\end{bmatrix}$$

Il termine in δ è la variabile di controllo vera e propria; il termine in ρ_t va invece interpretato come un **disturbo**: non è sotto il controllo diretto del sistema, ma è comunque un disturbo *misurabile e predicibile*, perché la curvatura del path è nota esplicitamente dal planner.

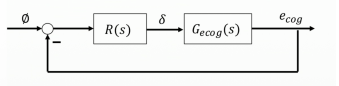
La funzione di trasferimento da δ a e_{cog} risulta:

$$G_{ecog}(s) = \frac{V^2}{L}\left(\frac{1 + (l_r/V)s}{s^2}\right)$$

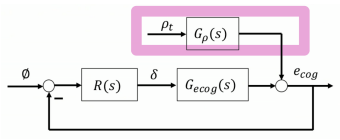
cioè un **doppio integratore più uno zero**, in cui il guadagno generalizzato è proporzionale a V^2 e la posizione dello zero dipende dalla velocità.

5.2.2.5 Lateral error feedback

L'obiettivo del controllo è **mantenere l'errore laterale il più piccolo possibile**: idealmente, un tracking perfetto del path corrisponde a $e_{cog} = 0$ in ogni istante. Il riferimento del sistema è quindi sempre nullo, e il disturbo da rigettare è la curvatura del percorso. L'approccio più naturale è utilizzare un semplice **controllore proporzionale**. Su carta funziona: essendo un controllore proporzionale, viene già garantito che l'errore tenda a zero a fronte di variazioni a gradino e a rampa del disturbo. Nella pratica, però, le prestazioni non sono altrettanto buone: il diagramma di



Bode di $G_{ecog}(s)$ parte da -180 di fase (a causa del doppio integratore), e sebbene lo zero introdotto dal termine l_r/V recuperi un po' di margine di fase, non è sufficiente per garantire un buon comportamento in anello chiuso.



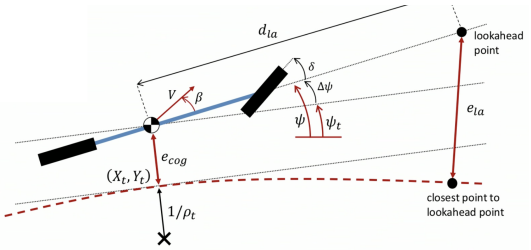
Per questo motivo, al posto di un semplice controllore P, si preferisce un **controllore PD** (o, equivalentemente, uno zero a bassa frequenza in un compensatore lead), che aumenta il margine di fase disponibile e quindi migliora le prestazioni. È inoltre possibile introdurre un termine di **feedforward**: poiché il disturbo dovuto alla curvatura ρ_t è misurabile, si può usare direttamente questa informazione per compensarlo anticipatamente, piuttosto che aspettare che l'errore si manifesti e venga corretto solo in retroazione.

Il limite del lateral error feedback: nessuna anticipazione

Anche introducendo il feedforward, resta un limite strutturale: la compensazione avviene sulla curvatura *attuale* nel punto in cui si trova il veicolo. Ma quando si guida, non si guarda solo il punto esatto in cui ci si trova: si guarda *avanti*. Se l'errore viene calcolato al baricentro, il controllore è "cieco" rispetto ai cambiamenti del percorso finché il veicolo non li raggiunge fisicamente.

5.2.2.6 Lookahead error feedback

L'idea del **controllo con errore lookahead** è di controllare la distanza fra un punto virtuale posto in avanti rispetto al veicolo e il path, anziché la distanza al baricentro. In questo modo si tiene conto del fatto che l'errore si accumulerà *prima* di raggiungere la curva, anticipando quindi l'azione di sterzo.



L'errore lookahead può essere approssimato come:

$$e_{la} = e_{cog} + d_{la}\,\Delta\psi \qquad (d_{la} \ll 1/\rho_t)$$

Questa espressione mostra un fatto interessante: e_{la} è la somma dell'errore laterale attuale più un termine proporzionale a $\Delta\psi$, cioè all'orientamento del veicolo rispetto al path.

Guardando la relazione già trovata in precedenza,

$$\dot{e}_{cog} = V\left(\frac{l_r}{L}\delta + \Delta\psi\right),$$

si nota che $\Delta\psi$ è strettamente legato alla *derivata* dell'errore laterale. Usare e_{la} come variabile controllata equivale quindi ad aggiungere, in modo intrinseco, un'**azione derivativa** al controllo, senza bisogno di derivare esplicitamente il segnale di errore (operazione sempre delicata dal punto di vista del rumore).

La funzione di trasferimento da δ a e_{la} ha la stessa struttura di prima, ma lo zero si sposta:

$$H_{ela}(s) = \frac{V^2}{L}\left(\frac{1 + \left(\frac{l_r + d_{la}}{V}\right)s}{s^2}\right)$$

cioè ancora un doppio integratore con uno zero, ma posizionato a frequenza più bassa rispetto al caso precedente, tanto più bassa quanto maggiore è d_{la}.

5.2.2.7 Lookahead feedback vs PD: sono davvero equivalenti?

Viene naturale chiedersi: se il PD applicato a e_{cog} e il P applicato a e_{la} producono, dal punto di vista lineare, esattamente la stessa struttura di funzione di trasferimento (doppio integratore più zero), qual è la differenza pratica fra i due approcci?

Dal punto di vista strettamente lineare sono equivalenti: in entrambi i casi si introduce uno zero che si può posizionare a piacimento. Ma **nella realtà il sistema non è lineare**: il modello linearizzato è stato derivato assumendo che la curvatura del percorso sia *costante* (assunzione implicita nel processo di linearizzazione, non solo un'approssimazione locale). Se la curvatura cambia lungo il percorso — come accade sempre in un percorso reale — usare l'errore calcolato alla distanza di lookahead permette di includere un'informazione di **preview** sul futuro del percorso, superando l'ipotesi (troppo forte) che la curvatura di riferimento non cambi mai. Con l'errore al baricentro, invece, il controllore "non vede" il cambiamento del percorso finché non lo incontra fisicamente.

In altre parole: il quadro lineare è utile per progettare il controllore (scegliere guadagni, margini di fase, banda passante), ma bisogna sempre ricordare che il comportamento reale del sistema, in un mondo non lineare e con curvatura variabile, dipende in modo sostanziale dalla scelta della distanza di lookahead — che non è affatto una semplice caratteristica "cosmetica" del sistema, ma parte integrante del design del controllore.

5.2.2.8 Scelta della distanza di lookahead

Il controllore lavora nel dominio del tempo, ma una curva che si presenta a una certa distanza (in metri) ha conseguenze diverse a seconda della velocità con cui la si affronta: alla stessa distanza d_{la}, un veicolo che viaggia a 10 m/s ha un secondo di tempo per reagire, mentre uno a 20 m/s ne ha solo mezzo. La distanza di lookahead determina quindi *quanto in anticipo* viene avviata l'azione di sterzo:

- lookahead troppo lungo ⇒ si anticipa troppo presto (comportamento poco sensato);
- lookahead troppo corto ⇒ si reagisce troppo tardi.

La distanza ottima dipende da tre fattori:

- la **banda del controllore**: più velocemente il controllore riesce ad agire (e più velocemente il veicolo risponde al cambio di sterzo), minore è il tempo di reazione necessario, e quindi minore può essere la distanza di lookahead;
- la **velocità del veicolo**: a parità di tempo di reazione, andando più veloci si copre più spazio nello stesso tempo, quindi serve guardare più lontano;
- la **forma del percorso**: su questo aspetto non c'è molto margine di intervento.

L'intuizione conclusiva è semplice: più veloce si va, più lontano bisogna guardare, perché si ha meno tempo per anticipare. Il modo più semplice per tenerne conto è rendere la distanza di lookahead proporzionale alla velocità:

$$d_{la} = k_d^* V$$

Questa strategia si chiama **lookahead distance scheduling**.

5.2.2.9 Gain scheduling

Anche il guadagno del controllore proporzionale dipende dalla velocità (dato che il guadagno generalizzato della funzione di trasferimento è proporzionale a V^2). Anche in questo caso la funzione di trasferimento suggerisce direttamente come modulare il guadagno: scegliendo

$$k_p = k_p^* \frac{L}{V^2}$$

si cancella esattamente la dipendenza dalla velocità nel guadagno generalizzato del sistema in anello aperto.

Combinando lookahead distance scheduling e gain scheduling si ottiene una funzione di

trasferimento in anello aperto

$$L(s) = k_p H_{ela}(s) = k_p^* \frac{1 + \left(\frac{l_r}{V} + k_d^*\right) s}{s^2}$$

in cui la dipendenza residua dalla velocità (nella posizione dello zero) è resa trascurabile dal termine k_d^*, che tende a dominare rispetto a l_r/V. Il risultato pratico è che il diagramma di Bode della funzione di trasferimento in anello aperto — e, di conseguenza, anche la funzione di sensitività complementare $F(s)$ che lega il riferimento all'errore in anello chiuso — risulta pressoché **indipendente dalla velocità**: stesso margine di fase, stessa banda passante, indipendentemente da quanto veloce si sta guidando. Questo è esattamente l'obiettivo dello scheduling: gestire un sistema intrinsecamente tempo-variante (perché V cambia nel tempo) progettando un controllore il cui comportamento sia il più possibile costante.

5.2.2.10 Robustezza rispetto al cambiamento di curvatura

Fin qui si è studiata la funzione di trasferimento dal riferimento (nullo) all'errore. Ma è altrettanto importante studiare cosa succede quando cambia la *curvatura* del percorso, perché in fondo il "cambio di riferimento" nel nostro schema di controllo è proprio un cambio di curvatura ρ_t (il riferimento sull'errore lookahead è sempre zero: non ha senso pensare di "cambiare il riferimento per e_{la}").

Calcolando la funzione di trasferimento ad anello chiuso da ρ_t a e_{cog} quando il controllo è basato sull'errore lookahead, si osservano tre fenomeni interessanti dal grafico di Bode:

- un **errore laterale a regime non nullo** in curva (se $\rho_t \neq 0$, resta un errore statico);
- la dipendenza dell'errore **dalla velocità**: a velocità più alta l'errore cresce, anche con lo scheduling attivo;
- un **cambio di segno** dell'errore a seconda della velocità: la fase a bassa frequenza passa da 0 a 180, il che significa che esiste una velocità limite oltre la quale il veicolo comincia a "tagliare" la curva anziché seguirla correttamente.

A basse velocità l'errore a regime è piccolo e di segno positivo; man mano che si aumenta la velocità, l'errore cresce rapidamente. In sintesi: più veloce si va, più il veicolo taglia la curva, e maggiore è l'errore. Questa osservazione motiva il passaggio a un modello più accurato — il modello dinamico — quando si vogliono gestire velocità elevate.

5.2.3 Il modello dinamico

5.2.3.1 Perché non basta più il modello cinematico

Il modello cinematico è adeguato per basse velocità e manovre poco aggressive, ma diventa inaccurato quando le forze di interazione pneumatico-strada diventano rilevanti — cioè quando si guida più velocemente o si effettuano manovre più dinamiche. Per questi scenari si ricorre al **modello dinamico a bicicletta**, basato su equilibrio delle forze laterali, equilibrio del momento di imbardata, ed equazioni cinematiche per la posizione:

$$Ma_y = F_{yf} + F_{yr} \quad \text{(equilibrio forze laterali)}$$

$$J_z\dot{r} = l_f F_{yf} - l_r F_{yr} \quad \text{(equilibrio momento di imbardata)}$$

$$\dot{x} = V\cos(\psi+\beta), \qquad \dot{y} = V\sin(\psi+\beta), \qquad \dot{\psi} = r$$

Nell'ipotesi di pneumatici lineari, la forza laterale generata da ciascun pneumatico è proporzionale all'angolo di deriva (side slip angle) secondo la rigidezza di deriva (cornering stiffness):

$$F_{yf} = C_f\alpha_f, \qquad F_{yr} = C_r\alpha_r$$

Anche in questo caso, però, il modello così scritto fornisce (x, y) in coordinate globali, mentre a noi interessa il comportamento del veicolo *rispetto al path*. Si applica quindi la stessa procedura vista per il modello cinematico: sotto le medesime ipotesi ($L \ll 1/\rho_t$, $e_{cog} \ll 1/\rho_t$, piccoli angoli), si aggiungono le due variabili di stato $\Delta\psi$ ed e_{cog}, ottenendo un **modello linearizzato in coordinate del path** con quattro stati:

$$\begin{bmatrix} \dot{\beta} \\ \dot{r} \\ \Delta\dot{\psi} \\ \dot{e}_{cog} \end{bmatrix} = \begin{bmatrix} -\dfrac{C_f+C_r}{MV} & \dfrac{C_r l_r - C_f l_f}{MV^2} - 1 & 0 & 0 \\ \dfrac{C_r l_r - C_f l_f}{J_z} & -\dfrac{C_f l_f^2 + C_r l_r^2}{J_z V} & 0 & 0 \\ 0 & 1 & 0 & 0 \\ V & 0 & V & 0 \end{bmatrix} \begin{bmatrix} \beta \\ r \\ \Delta\psi \\ e_{cog} \end{bmatrix} + \begin{bmatrix} C_f/MV \\ l_f C_f/J_z \\ 0 \\ 0 \end{bmatrix} \delta + \begin{bmatrix} 0 \\ 0 \\ -V \\ 0 \end{bmatrix} \rho_t$$

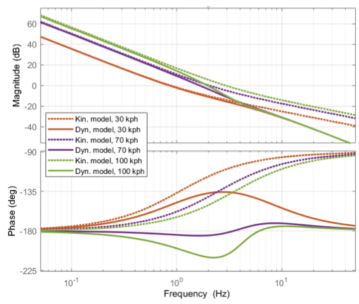
Rispetto al modello cinematico, che aveva solo due stati ($\Delta\psi$ ed e_{cog}), il modello dinamico ne ha quattro: si aggiungono l'angolo di assetto β e lo yaw rate r come variabili dinamiche vere e proprie (non più semplici funzioni algebriche di δ). Questo modulo (le due equazioni aggiuntive $\Delta\dot{\psi}$ ed $\dot{e}_{cog}$) può essere "agganciato" a qualsiasi modello di veicolo si abbia a disposizione, cinematico o dinamico: è essenzialmente un cambio di rappresentazione, non una proprietà specifica del modello.

5.2.3.2 Confronto fra modello cinematico e dinamico

Confrontando le funzioni di trasferimento da δ a e_{cog} dei due modelli — entrambe strutturate come due zeri su due poli, ma derivate in modo diverso — emergono considerazioni importanti a partire dai diagrammi di Bode a diverse velocità:

1. **A bassa frequenza** i due modelli sono molto simili. Questo è atteso: a bassa frequenza corrisponde un ingresso di sterzo pressoché costante, in cui la dinamica del veicolo (transitori, inerzie) non gioca un ruolo rilevante.
2. **Ad alta frequenza** la differenza cresce sensibilmente. In particolare:
   - il modello cinematico tende a *sovrastimare* la fase;
   - il modello dinamico resta ancorato a -180.

Questa discrepanza ha una conseguenza pratica molto concreta: se si progetta un controllore basandosi sul modello cinematico, e poi lo si applica al veicolo reale (che si comporta più vicino al modello dinamico), attorno a 1 Hz il margine di fase reale sarà molto inferiore a quello previsto in fase di progetto. Il rischio è instabilità (quando il margine di fase reale scende sotto zero) oppure oscillazioni scarsamente smorzate.



Inoltre, la differenza fra i due modelli cresce con la velocità: a 30 km/h la discrepanza diventa significativa solo intorno ai 3 Hz, mentre a 100 km/h è già rilevante a 1 Hz. La regola pratica che ne consegue è:

- **controllo per parcheggio assistito** → il modello cinematico è sufficientemente accurato;
- **controllo per guida autostradale** (alte velocità, alte frequenze di eccitazione) → è necessario il modello dinamico.

5.2.4 Curvature Preview Control****

5.2.4.1 L'idea di base: controllare l'orientamento, non solo l'errore

Le tecniche viste finora (lateral error feedback, lookahead error feedback) sono applicabili anche al modello dinamico. Il **curvature preview control** è un approccio alternativo, pensato per un tracking ad alte prestazioni e robusto, che parte da una premessa diversa: fino a questo punto si è utilizzato solo l'errore laterale come variabile controllata, trascurando esplicitamente l'orientamento. **Nel curvature preview control, invece,**

l'informazione sull'orientamento diventa centrale.

La catena causale della dinamica veicolo è: quando si cambia l'angolo di sterzo, cambia l'angolo delle ruote e, di conseguenza, lo yaw rate; lo yaw rate cambia la direzione del moto, e questo, con il tempo, genera un errore laterale rispetto al path. L'idea è quindi intervenire "a monte" di questa catena, controllando direttamente lo **yaw rate** in modo da tenere piccolo l'errore di orientamento.

Se si definisce $\Delta\psi$ come l'angolo fra la direzione in cui punta il veicolo e la direzione del path, e se ne considera la derivata,

$$\Delta\dot{\psi} = r - \dot{\psi}_t \cong r - \rho_t V_x = 0$$

si ottiene la condizione ideale

$$\dot{\psi}_t \approx \rho_t V_x \approx r \quad \Longleftrightarrow \quad \text{errore di orientamento piccolo (buon tracking)}$$

L'idea è quindi progettare un controllore che insegua un riferimento di yaw rate calcolato direttamente dalla curvatura desiderata, $r_{ref} = V_x\,\rho_t$, tramite l'angolo di sterzo δ. Sotto due ipotesi ideali:

1. banda infinita del controllore (irrealistica)
2. veicolo già esattamente sul path all'istante iniziale

questo semplice anello di controllo dello yaw rate sarebbe già sufficiente. Nella pratica nessuna delle due ipotesi è verificata, e questo motiva le due estensioni descritte di seguito.

Prima criticità: banda finita del controllore

In realtà **la banda passante non può essere infinita**: l'anello di controllo dello yaw rate attraversa la dinamica dell'attuatore e la dinamica del veicolo, quindi ha una banda limitata. Bisogna allora studiare il sistema completo, includendo la conversione fra il riferimento di curvatura del path ρ_{ref} e l'errore risultante e_{cog}.

Va inoltre tenuto conto che la velocità con cui il veicolo avanza non è allineata esattamente con il suo asse longitudinale, ma forma un **angolo di rotta** (course angle) $\varphi = \psi + \beta$ con il sistema di riferimento, a causa dell'angolo di deriva β. La curvatura del percorso

effettivamente seguito dal veicolo dipende quindi dalla derivata di φ:

$$\dot{\varphi} = \dot{\psi} + \dot{\beta} = r + \dot{\beta}$$

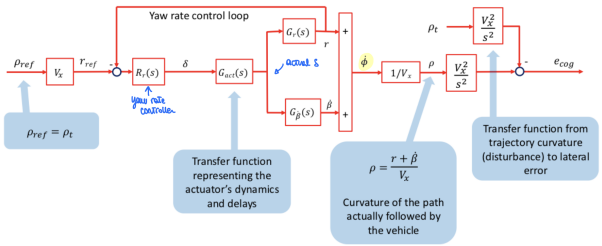
da cui, dividendo per V_x e integrando due volte, si ottiene e_{cog}.

L'anello di controllo dello yaw rate (Yaw rate control loop) è tipicamente progettato con un controllore **PI**, a partire dalla funzione di trasferimento in anello aperto fra δ e r, e tenendo conto esplicitamente della dinamica dell'attuatore $G_{act}(s)$. La funzione di trasferimento ad anello chiuso da ρ_t a e_{cog} risultante non è, per i motivi appena visti, esattamente uguale a quella "ideale" (che darebbe errore dinamico nullo): a bassa frequenza (fino a circa 1–4 Hz) si comporta come V_x^2/s^2, ma la fase non resta ancorata esattamente a -180.

La soluzione pratica è approssimare la funzione di trasferimento ad anello chiuso come

$$F(s) \approx \frac{V_x^2}{s^2}\, e^{-s\tau}$$

cioè un doppio integratore ideale più un semplice **ritardo** τ. Questo ritardo può essere facilmente compensato: dato che si conosce a priori l'intero percorso da seguire (fornito dal planner), è sufficiente usare la curvatura del path calcolata non nel punto in cui ci si trova ora, ma nel punto in cui ci si troverà fra τ secondi. Questo concetto è molto simile, nello spirito, alla distanza di lookahead già vista nell'approccio cinematico: lì l'anticipazione era di natura euristica, qui invece tutte le dipendenze (velocità, forma della curvatura, banda del controllore) vengono catturate e quantificate rigorosamente dal parametro τ, calcolabile a partire dalla funzione di trasferimento del sistema.

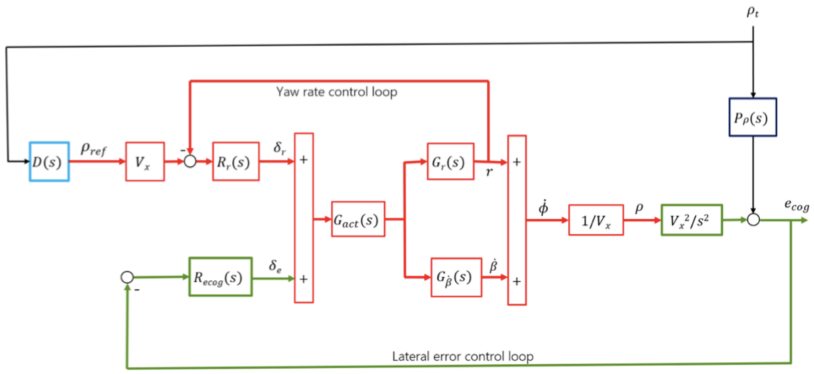


Seconda criticità: il drift dal path

La seconda ipotesi ideale (veicolo già sul path all'istante iniziale) non è in generale verificata: se applicassimo il solo anello sullo yaw rate senza correzioni, un piccolo errore iniziale — o anche solo piccoli errori di tracking che si accumulano nel tempo — porterebbe il veicolo ad allontanarsi progressivamente dal path. Il motivo è strutturale: nella relazione fra yaw rate ed errore laterale è presente un **integratore**, quindi anche piccoli errori nel

seguire il riferimento di yaw rate si accumulano nel tempo, generando un drift laterale crescente.

La soluzione è aggiungere un **secondo anello di controllo**, a bassa banda, che prende in ingresso e_{cog} e restituisce un termine correttivo aggiuntivo sull'angolo di sterzo δ_e. Questo secondo anello ha lo scopo esclusivo di evitare il drift, non di guidare la dinamica principale: per questo può essere progettato con una banda molto più bassa rispetto all'anello sullo yaw rate (tipicamente con un controllore PID).



5.2.4.2 Architettura cascata

Un primo modo, semplice e intuitivo, di combinare i due anelli è progettarli separatamente e sommare i due contributi: $\delta = \delta_r + \delta_e$ (dove δ_r è generato dall'anello sullo yaw rate e δ_e dall'anello sull'errore laterale). L'idea funziona, ma presenta due problemi seri in fase di implementazione:

1. **Antiwindup difficile da gestire**: l'azione di controllo finale è la somma di due azioni indipendenti. Se l'attuatore satura (esiste un angolo di sterzo massimo), serve un meccanismo di antiwindup — ma con due controllori indipendenti, ciascuno con la propria integrazione, non è chiaro chi debba "farsi carico" dell'antiwindup: sono due master, o uno slave dell'altro?

2. **Cancellazione critica polo/zero**: analizzando in dettaglio le funzioni di trasferimento, si scopre che nel percorso da δ_e a e_{cog} compare una cancellazione zero/polo. In simulazione tutto sembra funzionare correttamente (l'azione complessiva $\delta = \delta_e + \delta_i$ appare regolare), ma le singole funzioni di trasferimento delle due variabili di controllo, prese separatamente, risultano **instabili**. Prima o poi, per via di questa instabilità nascosta, si osserveranno problemi numerici (overflow) nell'implementazione reale.

La soluzione a entrambi i problemi è l'**architettura a cascata** (cascade approach): al

posto di sommare due segnali di sterzo indipendenti, si progetta l'anello sull'errore laterale in modo che generi un termine *aggiuntivo al riferimento di yaw rate*, anziché un contributo di sterzo separato. Il controllore sull'errore laterale, cioè, non calcola più direttamente δ_e, ma modifica r_{ref}, che poi passa attraverso l'unico anello sullo yaw rate per generare un unico δ.

Per garantire che l'architettura a cascata produca esattamente lo stesso comportamento ingresso-uscita di quella "parallela" già tarata (block-diagram equivalence), si impone l'uguaglianza delle due funzioni di trasferimento d'anello dell'errore:

$$L_e(s) = R_{ecg}^{baseline}(s)\, G_{\delta e_{cg}}(s) = R_{ecg}^{cascaded}(s)\, R_r(s)\, G_{\delta e_{cg}}(s)$$

da cui si ricava il nuovo controllore, che risulta

$$R_{ecg}^{cascaded}(s) = R_{ecg}^{baseline}(s)\, \frac{1}{R_r(s)}$$

Svolgendo i calcoli, si ottiene un controllore equivalente in cui la cancellazione critica zero/polo *sparisce*: la cancellazione che appare nella derivazione a questo punto non è più critica, perché nasce dal rapporto fra due termini appartenenti alla stessa funzione di trasferimento, non da poli/zeri instabili "nascosti" fra due sottosistemi indipendenti.

I vantaggi dell'architettura a cascata sono quindi:

- nessuna cancellazione critica zero/polo;
- un solo controllore genera l'angolo di sterzo finale, quindi l'antiwindup si implementa in modo naturale su un'unica variabile di controllo;
- è facile introdurre una saturazione sul riferimento di yaw rate: dato che yaw rate × velocità = accelerazione laterale, saturare il riferimento di yaw rate equivale direttamente a limitare l'accelerazione laterale del veicolo, un vincolo fisico molto naturale da imporre.

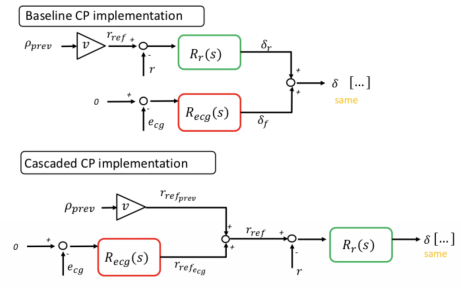


Figure 5.1: Cascade Implementation

5.3 Longitudinal Control *

5.3.1 Collocazione del problema

Dopo aver affrontato il controllo laterale, ci occupiamo ora del **controllo longitudinale**, cioè di come gestire velocità e accelerazione del veicolo lungo la direzione di marcia. Le due variabili di interesse sono:

- la **velocità** del veicolo;
- l'**accelerazione longitudinale**.

Si assume che il sistema disponga già di un ABS (Anti-lock Braking System), il che permette di **trascurare la dinamica delle ruote**: l'ABS garantisce che le ruote non slittino, quindi possiamo ragionare direttamente in termini di forze e accelerazioni del veicolo nel suo complesso, senza dover modellare esplicitamente lo slittamento longitudinale.

Il controllo longitudinale di cui si parla in questo capitolo appartiene alla categoria dei **controlli di basso livello** (low level control), nel senso che si occupa di tradurre una richiesta di accelerazione in comandi fisici (acceleratore/freno). Al di sopra di questo livello si colloca il caso applicativo su cui ci si concentra maggiormente: l'**Adaptive Cruise Control (ACC)**, che opera in scenari dinamicamente evolventi (ad esempio la presenza di un veicolo che precede).

		Domain	
SAE Level		Longitudinal	Lateral
	Level 0	ABS, TC, Cruise Control	ESC, ESP, Torque Vectoring
	Level > 0	Adaptive Cruise Control, Platooning	Lane Centering, Path Tracking

Figure 5.2: SAE Level table

L'architettura generale del controllo longitudinale a basso livello si basa su algoritmi già esistenti nel veicolo:

- ABS;
- controllo motore (engine control);
- controllo del cambio (gear control).

Lo schema concettuale di ingresso/uscita è semplice: dalla coppia motore (o posizione del pedale dell'acceleratore) e dalla coppia frenante si ottiene, tramite la dinamica longitudi-

nale del veicolo, il comportamento del veicolo stesso. Se si tratta di un veicolo elettrico, si ha accesso diretto all'interfaccia di coppia (quanta coppia applicare alla ruota); discorso analogo vale per la coppia frenante.

5.3.2 Il modello di dinamica longitudinale

L'equazione fondamentale che governa il moto longitudinale del veicolo è:

$$M\dot{v} = F(throttle) - F(brake) + F_{eng} - F_g - F_r - F_a$$

dove il termine $M\dot{v}$ rappresenta la forza di inerzia, e a destra dell'uguale compaiono tutti i contributi di forza che agiscono sul veicolo. Analizziamoli uno per uno.

- **Forza di inerzia**: $F_i = M_v\dot{v}$, dove M_v è la massa del veicolo (nota) e $\dot{v}$ l'accelerazione (variabile di controllo).
- **Forza di gravità (grade force)**: $F_g = M_v g \sin(\theta)$, dove θ è la pendenza della strada. È un termine noto, se si conosce la pendenza del tracciato.
- **Resistenza al rotolamento (rolling resistance)**: $F_r = c_r(v)M_v g\cos(\delta)$, dove c_r è il coefficiente di resistenza al rotolamento (dipendente dalla velocità).
- **Resistenza aerodinamica**: $F_a = \frac{1}{2}\rho_a A_f C_d v^2$, un termine quadratico dipendente dalla velocità, dove ρ_a è la densità dell'aria, A_f la sezione frontale del veicolo e C_d il coefficiente di resistenza aerodinamica.
- **Forza di trazione** $F(throttle)$, **forza frenante** $F(brake)$ ed **engine brake** F_{eng} (freno motore).

Il punto cruciale, evidenziato esplicitamente, è che alcuni di questi termini sono **noti** (massa, accelerazione, pendenza), mentre altri — in particolare i coefficienti legati alla resistenza al rotolamento e all'aerodinamica — devono essere **identificati sperimentalmente**, perché non sono generalmente disponibili con precisione a priori (o comunque conviene verificarli sul veicolo reale).

5.3.3 Identificazione dei parametri: la prova di coasting down

5.3.3.1 Impostazione del problema

Per identificare i coefficienti incogniti si utilizza una prova sperimentale chiamata **coasting down**: il veicolo, su una superficie piana, viene lasciato decelerare liberamente sotto il solo effetto dell'attrito, senza applicare né freno né coppia motore.

In queste condizioni, l'equazione generale

$$M\dot{v} = F(throttle) - F(brake) + F_{eng} - F_g - F_r - F_a$$

si semplifica notevolmente: non applicando né coppia motore né freno, e trovandosi su superficie piana (quindi $F_g = 0$), rimane

$$\underbrace{m\dot{v}}_{\text{misurato}} = F_r + F_a$$

Il termine a sinistra, $m\dot{v}$, è direttamente **misurabile** tramite gli strumenti di bordo (accelerometro), mentre a destra compaiono esattamente i due termini che vogliamo identificare. In pratica si può quindi scrivere, in forma compatta,

$$m\dot{v} = f(v)$$

dove la funzione f è nota nella sua *struttura*, ma con coefficienti da determinare. Il fatto di poter esprimere tutto in funzione della sola velocità (e non del tempo) permette di trattare il problema come un problema di regressione statica.

5.3.3.2 Identificazione tramite regressione lineare

Poiché sia la resistenza al rotolamento sia la resistenza aerodinamica sono, per costruzione fisica, funzioni della velocità (la prima linearmente, la seconda quadraticamente), è naturale postulare una struttura del tipo

$$m\dot{v} = F_r + F_a = Av^2 + Bv + C$$

dove A, B, C sono le incognite del problema. Si tratta quindi di un classico problema di **regressione lineare** (lineare nei parametri, anche se la relazione fisica in v non lo è), che si può scrivere in forma matriciale come

$$m\dot{v} = \begin{bmatrix} A & B & C \end{bmatrix} \begin{bmatrix} v^2 \\ v \\ 1 \end{bmatrix}, \qquad y = \theta x$$

dove $y = m\dot{v}$ è misurato, $x = [v^2 \ v \ 1]^T$ è misurato (essendo funzione della sola velocità, anch'essa misurata), e $\theta = [A \ B \ C]$ è il vettore di parametri incogniti da stimare risolvendo un problema ai minimi quadrati sui dati sperimentali raccolti durante la prova di coasting down.

Il risultato tipico è una nuvola di punti sperimentali (accelerazione longitudinale in funzione della velocità del veicolo) su cui si sovrappone la curva di regressione stimata: il fit è generalmente buono proprio perché la struttura funzionale ($Av^2 + Bv + C$) è coerente con la fisica del problema (aerodinamica quadratica, rotolamento approssimativamente lineare).

5.3.3.3 La forza alla ruota

Oltre ai termini resistivi, è utile introdurre esplicitamente il concetto di **forza alla ruota** F_w (wheel force): è la forza generata dalla ruota per interazione fra pneumatico e terreno, e dipende dallo scorrimento longitudinale (*slip*). Per le applicazioni di controllo longitudinale, tuttavia, si trascura solitamente lo scorrimento delle ruote (coerentemente con l'ipotesi iniziale sull'ABS), e si scrive semplicemente

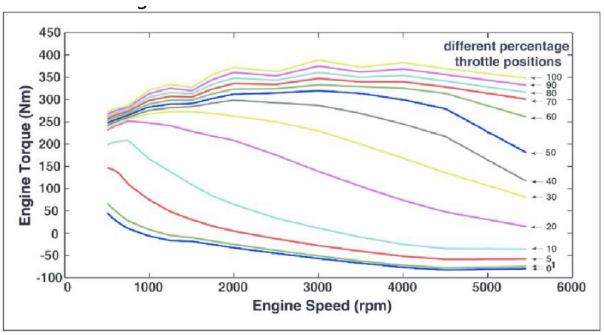
$$F_w = rT_T - rT_B$$

cioè la forza alla ruota come rapporto (tramite il raggio r della ruota) fra la coppia di trazione T_T e la coppia frenante T_B: si passa cioè direttamente da coppia a forza. Due osservazioni pratiche:

- in **trazione**, la coppia di trazione è controllata in modo diretto e molto preciso dalla centralina motore (electronic control unit) — con precisione maggiore nei veicoli elettrici rispetto ai veicoli con motore a combustione interna;
- in **frenata**, la coppia frenante è, in prima approssimazione, **lineare** rispetto alla pressione applicata all'impianto frenante.

5.3.4 Le mappe motore

5.3.4.1 Motore a combustione interna *

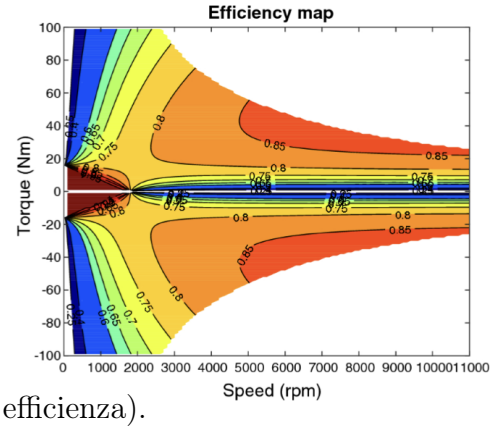


Per un motore a combustione interna, la relazione fra coppia erogata, velocità di rotazione del motore (regime, rpm) e posizione dell'acceleratore non è esprimibile in forma chiusa semplice: viene tipicamente descritta tramite una **mappa** sperimentale, con curve diverse per ogni percentuale di apertura dell'acceleratore. Da questa mappa emerge una dipendenza sia dal regime motore sia dalla percentuale di

apertura del pedale: anche premendo a fondo l'acceleratore, la coppia disponibile non è costante, ma varia (e in genere diminuisce) al crescere del regime oltre un certo punto.

5.3.4.2 Motore elettrico

Per un motore elettrico la situazione è più semplice sotto un aspetto fondamentale: non è necessario passare attraverso la relazione posizione-pedale → coppia, perché è possibile comandare direttamente la coppia desiderata. L'unica informazione che serve conoscere è, per ciascun regime di rotazione, quale sia la **coppia massima disponibile** — un vincolo imposto dai limiti fisici del motore elettrico stesso (tipicamente rappresentato tramite una mappa di efficienza).



Per quanto riguarda il sistema frenante, si osserva sperimentalmente che, per la maggior parte del campo di funzionamento, esiste una relazione **lineare** fra pressione applicata all'impianto frenante e coppia frenante generata (con leggere variazioni al variare della velocità del veicolo). Nel complesso, quindi, la modellazione della dinamica longitudinale risulta relativamente semplice: dipende in gran parte da mappe statiche, determinabili sperimentalmente o note dal costruttore.

5.3.5 Progettazione del controllo longitudinale

5.3.5.1 Architettura generale: alto livello e basso livello

L'architettura complessiva del controllo longitudinale si articola su due livelli:

- **High-Level Control**: a partire da un profilo di velocità desiderata, genera una richiesta di accelerazione desiderata a_{des};
- **Low-Level Control**: a partire da a_{des}, genera i comandi di throttle e brake da inviare al veicolo, ricevendo anche un feedback diretto dal veicolo stesso.

Nello specifico:

- il **low level control** è responsabile di generare throttle e brake per inseguire un'accelerazione longitudinale desiderata assegnata dall'alto;
- il **high level control** si occupa invece di progettare quale accelerazione desiderata richiedere, a partire dal profilo di velocità di riferimento.

5.3.5.2 Il controllo dell'accelerazione

Il controllore longitudinale confronta l'accelerazione desiderata a_{des} con l'accelerazione misurata a_{meas} e calcola un'accelerazione di riferimento a_{stpt} (setpoint). È importante osservare che, anche non applicando alcun comando, il veicolo non resta a velocità costante: è comunque soggetto alla decelerazione dovuta alla resistenza al rotolamento e all'aerodinamica (esattamente la curva di coasting down identificata in precedenza). È quindi necessario **compensare** questa curva di coasting down: il confine fra la zona in cui serve applicare throttle e quella in cui serve applicare freno è determinato proprio dalla curva di coasting down stessa.

Il legame fra coppia e accelerazione è, in prima approssimazione, **statico** e proporzionale (l'accelerazione è proporzionale alla coppia applicata), il che rende concettualmente semplice il progetto del controllore che insegue l'accelerazione desiderata. L'unica vera complessità risiede nelle **non linearità** fra la posizione dell'attuatore (throttle) e la coppia effettivamente generata — esattamente le non linearità descritte dalle mappe motore viste in precedenza.

5.3.5.3 Compensazione delle non linearità: le mappe statiche inverse

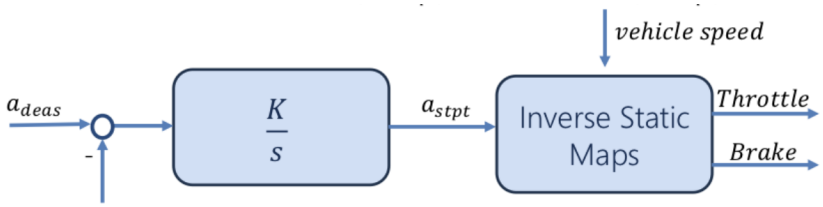
La soluzione proposta per gestire queste non linearità è l'uso delle **Inverse Static Maps**, che rappresentano il comportamento a regime (steady-state) del veicolo. Si assume che la dinamica di basso livello sia descrivibile come

$$a_{stpt} = f_1(v_x, throttle) + f_2(v_x, brake)$$

dove f_1 rappresenta il contributo del motore (inclusa la frenata motore) e f_2 il contributo del sistema frenante. Dal punto di vista del controllore, ciò che serve è l'operazione **inversa**: dato che si conosce l'accelerazione desiderata (e la velocità attuale), si vuole risalire a quale comando di throttle o di brake applicare:

$$throttle = f_1^{-1}(v_x, a_{stpt}), \qquad brake = f_2^{-1}(v_x, a_{stpt})$$

Il blocco delle mappe statiche inverse riceve quindi in ingresso la velocità del veicolo e l'accelerazione di riferimento, e restituisce in uscita il comando (throttle o brake) da applicare. Un dettaglio implementativo importante è che, all'interno di questo blocco, si tiene conto anche del contributo di *coasting down* (chiamato "Equivalent Engine/Coasting Down acceleration"): a seconda del segno della differenza fra l'accelerazione richiesta e quella prodotta naturalmente dal coasting down, si decide se il comando debba essere di



throttle o di brake, garantendo così la compensazione di cui si parlava sopra.

5.3.5.4 Il controllore a cascata

Per chiudere l'anello, l'accelerazione desiderata a_{des} viene generata a partire da un errore fra velocità di riferimento e velocità misurata, elaborato tramite un controllore K/s (un integratore puro, con guadagno K):

$$a_{deas} \longrightarrow \frac{K}{s} \longrightarrow a_{stpt} \longrightarrow \text{Inverse Static Maps} \longrightarrow \text{Throttle, Brake}$$

Analizzando l'intero anello (comprensivo di dinamica dell'attuatore, con ritardo puro T_d e costante di tempo L_a), si può approssimare il comportamento dell'anello interno di inseguimento dell'accelerazione con una funzione di trasferimento del primo ordine:

$$F(s) \simeq \frac{K}{K+s} = \frac{1}{1+sT_1}, \qquad T_1 = K^{-1}$$

dove K è proprio il guadagno del controllore interno. Il controllore complessivo (esterno, sulla velocità) ha quindi la struttura

$$v_{ref} \longrightarrow \frac{k_2(1+sT_1)}{1+sT_2} \longrightarrow a_{des} \longrightarrow \frac{1}{1+sT_1} \longrightarrow a_{meas} \longrightarrow \frac{1}{s} \longrightarrow v_{meas}$$

cioè un controllore esterno di tipo lead/lag (con zero a $1/T_1$, cancellante il polo dell'anello interno) che genera a_{des}, seguito dall'anello interno approssimato come sistema del primo ordine, il cui output viene poi integrato per ottenere la velocità. Questo è un esempio classico di **controllo a cascata**: un anello interno veloce (sulla accelerazione) annidato dentro un anello esterno più lento (sulla velocità).

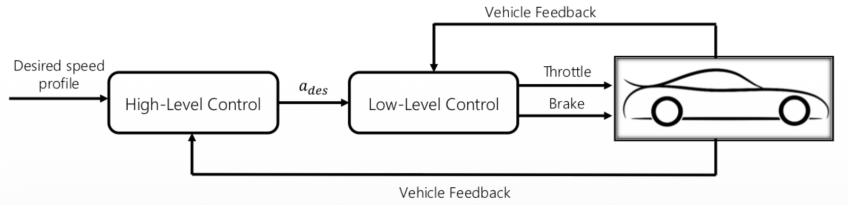


Figure 5.3: Design of a Cascade Longitudinal Controller

5.3.5.5 I vantaggi del controllo a cascata

Progettare il controllo longitudinale con due anelli in cascata, anziché con un unico anello diretto dalla velocità ai comandi attuatore, porta diversi vantaggi pratici:

1. **Compensazione più semplice delle non linearità del motore**: grazie all'anello interno chiuso, è più semplice sfruttare l'informazione contenuta nelle mappe statiche. L'anello interno opera sotto ipotesi statiche, e in quel contesto è possibile invertire e includere direttamente le mappe motore/freno.
2. **Facilità nel limitare l'accelerazione**: è tipico dei controlli a cascata poter introdurre facilmente dei limiti intermedi. Se per qualche motivo si vuole garantire che l'accelerazione longitudinale non superi mai un certo valore massimo, basta inserire una **saturazione** fra il livello alto e il livello basso del controllo, senza dover ridisegnare l'intera architettura.

Un'osservazione pratica ricorrente riguarda proprio la saturazione: in frenata, tipicamente si satura la decelerazione richiesta a un valore pari a circa $1\,g$. È inoltre importante sottolineare che affidarsi in modo eccessivo a un approccio puramente "modulare" (cioè lasciare che sia sempre il livello di basso livello a decidere autonomamente come applicare freno e motore) può essere rischioso: idealmente, dovrebbe essere il **planner** (il livello più alto) a decidere esplicitamente come frenare in una determinata situazione, piuttosto che delegare completamente questa scelta al solo controllore di basso livello.

5.3.5.6 Il sistema visto dall'anello esterno

Dal punto di vista dell'anello *esterno* (quello che chiude su velocità/posizione), l'accelerazione effettiva segue la funzione di trasferimento approssimata vista sopra, e va poi integrata per ottenere la velocità. In altre parole, il sistema visto dall'esterno è composto da un'**integrazione** più un singolo **polo**, la cui posizione dipende dalla banda dell'anello interno e dai limiti dell'attuatore. Si tratta quindi, dal punto di vista dell'anello esterno, di un problema di controllo relativamente "classico", per il quale sono sufficienti tecniche di controllo standard.

Un test sperimentale tipico mostra il confronto fra velocità desiderata (in verde) e velocità effettivamente raggiunta dal veicolo (in blu) su un profilo composto da tratti di accelerazione, velocità costante e frenata: il buon inseguimento del profilo, unito al corretto rispetto dei limiti di accelerazione imposti, conferma la validità di questa architettura.

5.4 Osservazioni conclusive

In sintesi, rispetto al controllo laterale — che richiede modelli via via più sofisticati (geometrico, cinematico, dinamico) a seconda del regime di velocità e delle prestazioni richieste — il controllo longitudinale si presta bene a un approccio *classico*, basato su tecniche di controllo consolidate:

- il problema di controllo longitudinale può essere affrontato con teoria di controllo relativamente "classica";
- il tipo di attuatore disponibile gioca però un ruolo fondamentale: proprio le non linearità dell'attuatore (motore, impianto frenante) determinano la necessità delle mappe statiche inverse, e quindi anche la scelta dell'architettura a cascata.

6 | Planning

6.1 Il quadro generale: perché "planning" e non solo "routing"

Il punto di partenza è distinguere planning da routing. Il routing classico è quello che fa un navigatore: hai una mappa, sai dove sei e dove vuoi andare, e ottieni una sequenza di strade da percorrere. È un problema chiuso, nel senso che una volta definiti mappa, partenza e destinazione, la soluzione è univoca e non richiede altro ragionamento in tempo reale. Il planning va oltre, e si divide in due livelli: **global planning e local planning**.
Il **global planning** parte dal presupposto che il mondo sia statico, cioè lavora solo su informazioni note a priori (la mappa). Rispetto al semplice routing ha però una risoluzione più alta: non ti dà solo la sequenza di strade, ma un **profilo geometrico dettagliato** (il path vero e proprio) e anche un profilo di velocità di riferimento — per esempio, quanto velocemente affrontare una curva. Quindi il global planning risponde sia alla domanda *"dove andare"* sia, in parte, a *"quanto veloce"*.
Il **local planning** invece entra in gioco quando servono **informazioni dinamiche**: altri veicoli, semafori, ostacoli mobili. Parte dal global plan e lo raffina usando il modello di percezione, cioè ciò che i sensori vedono in tempo reale. È qui che **il sistema deve prendere decisioni vere e proprie** (sorpassare o no, fermarsi o no) e non semplicemente seguire una traccia precalcolata. Il caso di studio del corso è deliberatamente semplificato: guida su una strada arteriale urbana, con semafori e traffico, dove il sistema autonomo gestisce accelerazione/frenata e piccole manovre laterali (cambio corsia per sorpassare), ma non gira agli incroci — a quel punto serve il take-over del guidatore. Questo permette di scomporre il problema in due moduli distinti e più trattabili: un planner longitudinale (quanto veloce andare) e un planner laterale (che traiettoria laterale seguire).

6.2 Longitudinal Planner

6.2.1 Cosa deve fare

Il planner longitudinale prende in ingresso il piano laterale, la percezione e la localizzazione, e produce in uscita una **velocità di riferimento che il controllore dovrà inseguire lungo il percorso laterale**. Il profilo di velocità che genera deve soddisfare due requisiti:

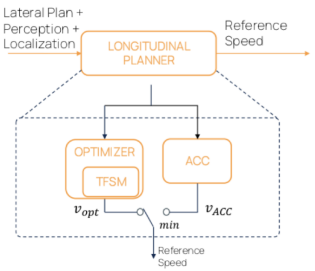


Figure 6.1: Long. Planner

- **sicurezza**: il veicolo deve poter seguire quel profilo senza collisioni;
- **ottimalità**:il profilo deve ottimizzare una funzione obiettivo, che include anche il **comfort** (la guida autonoma deve essere piacevole, non solo sicura).

L'architettura è composta da tre blocchi:

1. **Optimizer** — genera il profilo di velocità ottimale rispettando i limiti fisici del veicolo;
2. **TFSM** (Tactical Finite State Machine) — decide le azioni di alto livello (fermarsi, ripartire);
3. **ACC** (Adaptive Cruise Control) — garantisce la sicurezza mantenendo la distanza dagli ostacoli.

Il punto concettualmente importante è come queste tre parti si combinano:

- l'Optimizer produce una velocità v_{opt} pensata per essere confortevole e ottimale,
- mentre l'ACC produce indipendentemente una velocità v_{ACC} pensata solo per evitare collisioni.

La velocità di riferimento finale è il minimo tra le due. Questa scelta progettuale non è casuale: significa che la sicurezza (ACC) può sempre *"vincere" sull'ottimalità*, e se l'optimizer per qualche motivo fallisce o produce qualcosa di poco sicuro, l'ACC blocca comunque il sistema entro un comportamento sicuro. È un classico pattern di ridondanza per la sicurezza: un modulo "ottimale ma potenzialmente fragile" e un modulo "grezzo ma affidabile" che fa da rete di sicurezza. Un problema, definito come problema di ottimo, è sempre composto da funzione di costo + vincoli. Qui:

- vincoli = non collidere (no crashing);
- funzione di costo = il profilo di velocità ottimale definito sopra (comfort + prestazioni).

Portare la sicurezza dentro un problema di controllo ottimo è delicato — da qui la scelta di tenerla separata nel modulo ACC piuttosto che infilarla come vincolo nell'ottimizzatore.

6.2.2 L'Optimizer

6.2.2.1 Generazione del profilo trapezoidale

Obiettivo: l'obiettivo dell'optimizer è **generare un profilo di velocità ottimo che rispetti i limiti fisici del veicolo** (accelerazione/decelerazione massima, curvatura massima, limiti di velocità stradali) e che sia poi inseguibile dal controllore lungo il percorso locale. Un'osservazione importante: il percorso non è rettilineo. Se il percorso ha una curva, il veicolo è soggetto anche ad accelerazione laterale, non solo longitudinale — e i due tipi di accelerazione condividono lo stesso "budget" fisico (l'aderenza pneumatico-strada). Un punto concettuale chiave: **il profilo di velocità viene generato in funzione dell'ascissa curvilinea (la distanza percorsa lungo il tragitto), non del tempo**. *Perché?* Perché i limiti di velocità (legali, di curvatura, di stop) sono associati a punti dello spazio, non a istanti di tempo. Se lavorassi in dominio temporale, dovresti prima sapere a che velocità arrivi in un certo punto per sapere quando ci arrivi — è circolare. Lavorando in spazio, ogni limite è fissato in un punto preciso indipendentemente dalla velocità con cui ci arrivi.

La friction ellipse

I vincoli di accelerazione sono descritti tramite l'**ellisse di aderenza**:

$$\left(\frac{a^x}{a^x_{max}}\right)^2 + \left(\frac{a^y}{a^y_{max}}\right)^2 = 1$$

Ogni punto dentro l'ellisse rappresenta una combinazione fisicamente raggiungibile di accelerazione longitudinale e laterale. **Guidare sul bordo dell'ellisse significa essere al limite fisico del veicolo**. Il senso pratico: se sto affrontando una curva stretta (a^y grande), mi resta poco budget per accelerare o frenare forte (a^x), e viceversa.

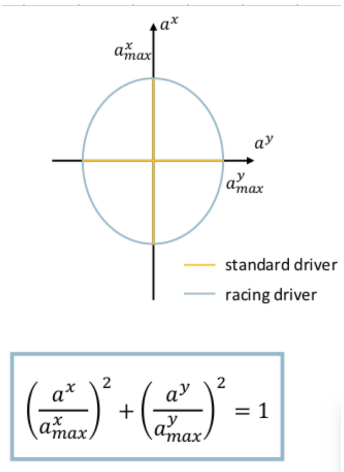


Figure 6.2: Friction Ellipse

L'algoritmo *

L'algoritmo procede in tre passate (da cui il nome "three-pass speed profile optimization"):

Passata 1 — Initial guess.
Si costruisce punto per punto la **massima velocità fisicamente ammissibile**, combinando due vincoli indipendenti:

- il limite di **velocità legale** (proprietà della strada, indipendente da accelerazione/decelerazione; include anche il caso del semaforo rosso, che equivale a un limite di velocità pari a 0 nel punto del semaforo);
- il **limite dovuto alla curvatura**, cioè la velocità massima compatibile con l'accelerazione laterale desiderata: $v_{max} = \sqrt{\frac{a_y^{max}}{|\rho|}}$ (dove ρ è la curvatura). In curva questo limite è più stringente; in rettilineo il limite è dato dal solo limite legale.

Questa prima stima è staticamente/istantaneamente fattibile, ma **non tiene conto del fatto che il veicolo non può cambiare velocità istantaneamente** — cioè non tiene conto della dinamica di accelerazione/decelerazione reale nel tempo.

Passata 2 — Forward pass.

Si integra in avanti la dinamica, considerando solo l'accelerazione (la frenata viene gestita nella passata successiva). Punto chiave: **dato che conosco la curvatura in ogni punto, so quanta accelerazione laterale sto "consumando", quindi quanta accelerazione longitudinale mi resta disponibile sull'ellisse d'aderenza**. Il modello di moto è un moto uniformemente accelerato tra coppie di punti consecutivi: Dominio temporale:

$$s_{i+1} = \tfrac{1}{2} a_i^x (\Delta t_i)^2 + v_i \Delta t_i + s_i \qquad v_{i+1} = a_i^x \Delta t_i + v_i$$

Risolvendo per Δt_i e sostituendo, si passa al dominio spaziale (noto Δs_i, dato dalla discretizzazione della traiettoria):

$$\Delta t_i = \frac{-v_i + \sqrt{v_i^2 + 2a_i^x \Delta s_i}}{a_i} \qquad v_{i+1} = \sqrt{v_i^2 + 2a_i^x \Delta s_i}$$

In pratica: dato che conosco velocità e accelerazione disponibile in un punto, integro in avanti e calcolo la velocità raggiungibile al punto successivo, tenendo conto che il veicolo ha bisogno di tempo/spazio per accelerare — non può "saltare" istantaneamente alla velocità massima di curvatura.

Passata 3 — Backward pass.

Simmetricamente, si integra all'indietro considerando i limiti di decelerazione (frenata) e l'accoppiamento longitudinale-laterale delle forze, partendo dalla fine del percorso verso l'inizio:

$$v_i^{(3)} = \min v^{(2)} i, v^{(3)} i, guess$$

Questo garantisce che il veicolo possa effettivamente fermarsi in tempo dove serve (es. prima di un semaforo rosso), non solo accelerare in modo fattibile.

Before Backward Pass

After Backward Pass

Figure 6.3: Evolution of the system.

Il risultato delle tre passate è il classico **profilo trapezoidale: rampe di accelerazione, tratti a velocità costante, rampe di decelerazione** — da cui il nome del modulo.

6.2.2.2 Smoothing (Quadratic Programming)

Il profilo trapezoidale è fattibile ma "spigoloso" (discontinuità nelle derivate). Per renderlo più confortevole **si risolve, a ogni istante, un problema di ottimizzazione quadratica che bilancia tre termini**:

$$v_{qp} = \underset{v_{qp}}{\text{argmin}}; \alpha \sum (v_{qp}(k) - v_{ref}(k))^2 + \beta \sum (v_{qp}(k+1) - v_{qp}(k))^2 + \gamma \sum (v_{qp}(k+2) - 2v_{qp}(k+1) + v_{qp}(k))^2$$

soggetto a $v_{lb} \leq v_{qp} \leq v_{ub}$. I tre termini sono, in ordine:

- distanza dal profilo di riferimento (non allontanarsi troppo dal trapezoidale)
- accelerazione (derivata prima, penalizza cambi bruschi di velocità)
- jerk (derivata dell'accelerazione — più alto il jerk, meno confortevole la guida, è la sensazione di "scatto")

Il vincolo superiore è fissato uguale al riferimento (la velocità massima raggiungibile), quello inferiore a zero.

Perché due step separati (trapezoidale + smoothing) invece di un solo problema di ottimo che includa da subito tutti i vincoli dinamici? Perché così facendo il primo step resta un algoritmo a complessità lineare, molto più semplice ed efficiente; unire tutto in un solo problema di ottimizzazione sarebbe teoricamente possibile ma computazionalmente più oneroso.

6.2.2.3 Valutazione quantitativa del comfort di guida

Il comfort di un tragitto in guida autonoma si valuta lungo due assi concettualmente distinti. Il primo è il **comfort dinamico**, legato al modo in cui il veicolo accelera e sterza: non basta guardare l'accelerazione longitudinale a_x e laterale a_y, ma soprattutto il **jerk** $j = \dot{a}$ (derivata dell'accelerazione), perché è lo *scatto* (jolt) percepito, non l'accelerazione in sé, a essere fastidioso — esattamente il motivo per cui il problema di smoothing QP dell'Optimizer (par.6.2.2.2) penalizza sia accelerazione sia jerk. Il secondo aspetto è il **comfort di marcia (ride comfort)**, cioè le vibrazioni ad alta frequenza trasmesse dalla strada attraverso sospensioni e sedile, tipicamente osservate sull'accelerazione verticale a_z. A questi si aggiunge la componente **geometrica**: curvature che variano bruscamente nel tempo (par. 5.1) generano accelerazione laterale/jerk elevati anche a velocità costante.

Dal punto di vista sensoristico non servono sensori dedicati: bastano gli stessi sensori propriocettivi già usati per la state estimation — una IMU (accelerometri 3 assi + giroscopio), idealmente posizionata vicino al baricentro o sul sedile/pianale per catturare ciò che percepisce davvero il passeggero, ed encoder ruota/GPS per ricostruire il profilo di velocità e correlare gli eventi di discomfort con posizione/velocità lungo il tragitto.

La valutazione quantitativa vera e propria si fa raccogliendo $a_x(t)$, $a_y(t)$, $j(t)$ (ed eventualmente $a_z(t)$) lungo tutto il tragitto e calcolando indicatori statistici: valore RMS, picco massimo, oppure percentuale di tempo in cui il segnale supera una soglia di comfort prefissata. Per il comfort vibrazionale in particolare si usa un approccio a norma frequenziale (standard ISO 2631): si applica un filtro di pesatura in frequenza al segnale di accelerazione — poiché il corpo umano è più sensibile ad alcune bande di frequenza che ad altre — e si calcola un'accelerazione RMS pesata, confrontabile tra veicoli e tragitti diversi. Le singole metriche (RMS di a_x, a_y, jerk, vibrazione pesata) possono infine essere combinate in un unico indice di comfort aggregato per l'intero tragitto, utile per confrontare due percorsi o due tarature del planner/controllore.

6.2.3 TFSM — Tactical Finite State Machine

Il TFSM gestisce le decisioni di alto livello: se il veicolo debba fermarsi o continuare. Analizza i punti della traiettoria (ciascuno può essere un nodo "normale", un semaforo, un dosso...) e la posizione del veicolo rispetto a questi. Gli stati sono:

- Normal — nessun evento particolare;
- Stop requested — uno stop è stato richiesto (es. ci si avvicina a un semaforo che potrebbe diventare rosso);

- Stop confirmed — lo stop è confermato (es. il semaforo è effettivamente rosso quando il veicolo arriva al nodo);
- Restart — si riparte dopo lo stop.

L'idea chiave è: se pianifico il profilo di velocità assumendo che il semaforo resti verde ma poi cambia in rosso, devo ricalcolare tutto perché la velocità pianificata non è più valida. Quando lo stato non è "Normal", il TFSM impone un nuovo vincolo di stop e forza un nuovo passaggio nell'Optimizer per **ricomputare il profilo ottimale tenendo conto del punto di arresto imposto**.

6.2.4 ACC — Adaptive Cruise Control*

L'ACC è il modulo che **garantisce la sicurezza mantenendo una distanza di sicurezza dal veicolo che precede**. È composto da due elementi:

1. **Spacing policy**: definisce la distanza desiderata da mantenere. Si usa una policy a time-headway:

   $$d_{desired} = t_{hw} \cdot v_{ego} + d_{offset}$$

   cioè più vado veloce, più la distanza di sicurezza richiesta è grande — semplice e computazionalmente economico.

2. **Tracking strategy**:

   il modo in cui questa distanza viene effettivamente inseguita. Si usa un **anello di controllo in cascata**: un guadagno proporzionale K_d trasforma l'errore di distanza in una velocità di riferimento $\bar{v}_{ref}$ (saturata), che viene poi inseguita da un controllore di velocità $R_v(s)$ (tipicamente un PID) che produce la velocità finale v_x.

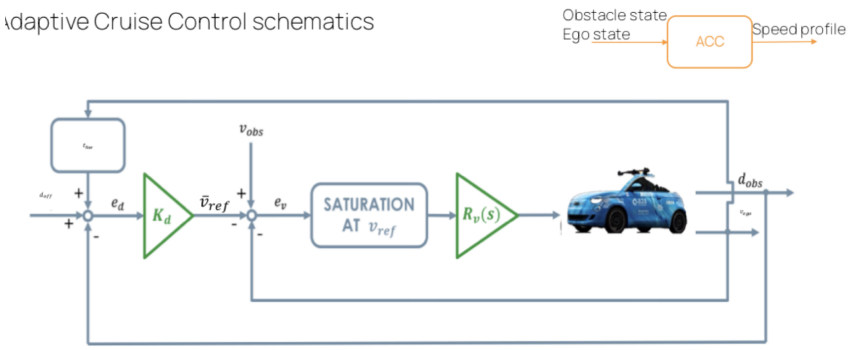


Figure 6.4: ACC scheme

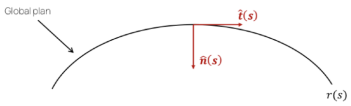
Nota architetturale importante, già menzionata sopra: **l'ACC può sovrascrivere l'optimizer**

prendendo il minimo delle due velocità proposte — è una scelta deliberata per dare priorità assoluta alla sicurezza rispetto all'ottimalità.

6.3 Lateral Planner *

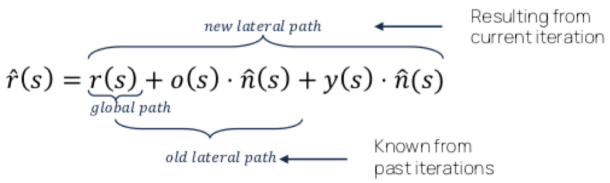
6.3.1 Impostazione del problema: parametrizzazione in Frenet

Il planner laterale modifica il global plan sulla base di informazioni locali (ostacoli, confini stradali). Il path generato viene poi inseguito dal controllore. È importante: **il lateral planner si occupa solo di generare il percorso geometrico; la velocità lungo quel percorso è compito del longitudinal planner**. Lavorare in coordinate cartesiane per descrivere uno scostamento da un percorso curvo è scomodo. Conviene invece definire, per ogni punto s del global path, un sistema di riferimento locale (frame di Frenet):



$$\hat{t}(s) = \frac{r'(s)}{|r'(s)|} \qquad \hat{n}(s) = \frac{\dot{\hat{t}}(s)}{\rho(s)}$$

dove $\hat{t}(s)$ è il versore tangente, $\hat{n}(s)$ il versore normale, $\rho(s)$ la curvatura nel punto s. In questo modo, indipendentemente da quanto sia complesso il global path, **ogni punto della soluzione può essere descritto semplicemente come uno scostamento laterale rispetto al path globale**, misurato lungo la normale:



La separazione tra $o(s)$ (offset accumulato dalle iterazioni passate) e $y(s)$ (la decisione corrente) è voluta: dà flessibilità per comporre manovre successive (es. prima sorpasso, poi rientro) sommandole in modo incrementale.
Come faccio a decidere e generare la traiettoria laterale per evitare un ostacolo? Abiamo due approcci alternativi per risolvere questo problema:

1. **closed-form, sigmoid-based**:

$$y(s) = \frac{w}{1 + e^{-a(s-c)}}$$

. Il path è descritto da una **formula chiusa a 3 parametri** (w = width of orthogonal shift, a=smoothness, c=horizontal displacement). Vi è una ricerca esasutiva.

2. **graph search**: qui il path è descritto da una sequenza di waypoint (un numero per ogni checkpoint lungo il percorso), e si costruisce un albero di possibilità esplorato con un algoritmo di ricerca su un grafo.

e Confronto sintetico:

Table 6.1: Confronto tra approccio **non** optimization-based e optimization-based.

	Pro	**Contro**
Non-optimization (sigmoid)	• Calcolo in un colpo solo. • Nessun rischio di non convergenza. • Veloce (millisecondi). • Dinamicamente fattibile per costruzione.	• Non gestisce bene scenari complessi (più veicoli). • I vincoli devono essere tradotti in forma chiusa. • Poco estendibile.
Optimization-based	• Gestisce facilmente scenari complessi. • I vincoli si integrano direttamente. • Facile aggiungere nuove considerazioni.	• Calcolato in più step. • Rischio di non convergenza. • Non velocissimo. • Non garantito dinamicamente fattibile.

6.3.2 Approccio closed-form: la sigmoide

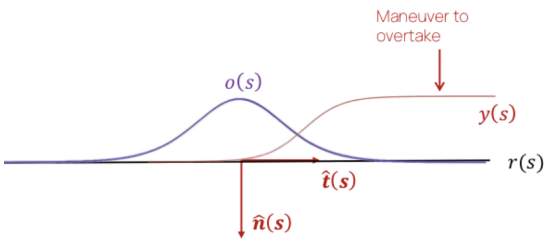


Figure 6.5: Sigmoide

Si sceglie di parametrizzare la manovra laterale (cambio corsia) con una **funzione sigmoide perché è una curva liscia la cui curvatura varia con continuità** — questo si accoppia bene con un controllore basato su curvature preview: se la curvatura cambia in modo graduale, è più facile per il controllore anticiparla e inseguirla, rispetto a una variazione brusca.

$$y(s) = \frac{w}{1 + e^{-a(s-c)}}$$

I tre parametri sono:

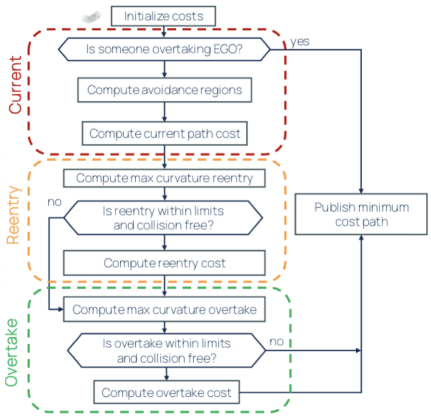
- w — ampiezza dello spostamento laterale (quanto mi sposto, es. larghezza corsia);
- a — "rigidità"/rapidità della manovra: più alto a, più brusco il cambio di traiettoria. Ha due conseguenze pratiche: un a alto è più facile da tracciare per il controllore in certe architetture (curvature preview), ma genera più accelerazione laterale a parità di velocità, quindi meno comfort;
- c — spostamento orizzontale (dove, lungo s, è centrata la manovra).

Con soli tre parametri si riduce drasticamente lo spazio delle decisioni: non sto cercando un'intera funzione, ma solo tre numeri.

6.3.2.1 Overtake decision making

A ogni istante il planner ha solo tre opzioni discrete:

1. cambiare a sinistra
2. cambiare a destra
3. restare in corsia



Avendo un numero finito e piccolo di opzioni, si può fare una **ricerca esaustiva**: si calcola il costo di ciascuna opzione e si sceglie quella minima. Il flusso è:

- si parte inizializzando i costi (current = 0, overtake = reentry = ∞);
- si verifica se qualcuno sta sorpassando me (in tal caso non posso cambiare corsia);
- si calcolano le regioni di collisione (avoidance region);
- si valuta la fattibilità del rientro e del sorpasso (curvatura massima raggiungibile, rispetto dei limiti di carreggiata, assenza di collisione);
- infine si pubblica il percorso a costo minimo.

6.3.2.2 Calcolo della regione da evitare

Si assume velocità costante di ego e ostacolo (un'ipotesi forte, esplicitamente segnalata come tale). Guardando solo la coordinata longitudinale x: la collision region start è il

punto in cui il fronte dell'ego raggiunge il retro dell'ostacolo; **la collision region end è il punto in cui l'ego ha completamente superato l'ostacolo**.

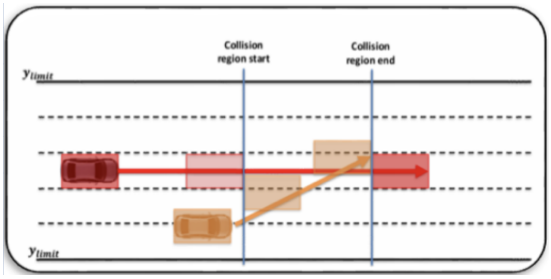


Figure 6.6: Collision Region

Se le velocità sono uguali non c'è mai collisione (le due auto restano parallele); se l'ostacolo è più lento, prima o poi ego lo raggiunge — ed è proprio questa regione, calcolata nel dominio spaziale, che il path generato deve evitare.

6.3.2.3 Funzione di costo

Per prendere la migliore decisione, l'ultimo passaggio prevede il calcolo della funzione di costo. È definita come somma di due termini:

$$Cost = Cost_v + Cost_o$$

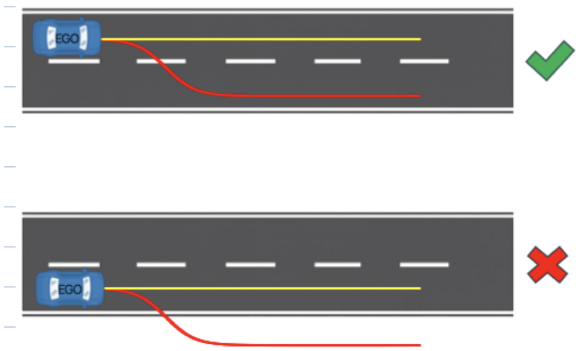
Dove:

- $Cost_v = (v_{ego} - v_{obs}) \cdot g(\Delta d)$ penalizza la deviazione rispetto alla velocità pianificata: quanto più l'ostacolo è lento rispetto a me, tanto più conviene sorpassare piuttosto che rallentare. Il fattore di scala $g(\Delta d)$, dove $\Delta d = d(s) - d_{min}$, serve a evitare di prendere decisioni troppo presto: se la distanza è grande, il costo è vicino a zero (non mi fido di una previsione così lontana nel tempo, dato che assumo velocità costante — un'ipotesi che diventa via via meno realistica più si guarda avanti); man mano che mi avvicino, il peso della differenza di velocità cresce. In pratica il sistema è progettato per decidere prima che scatti l'ACC.
- $Cost_o$ rappresenta la volontà di restare sul global path (evitare cambi di corsia inutili, e — per normativa italiana — tendere a occupare la corsia più a destra). È valutato come $o(s_N) + y(s_N)$, cioè lo scostamento laterale alla fine della manovra.

Un esempio intuitivo: se il veicolo davanti va solo 2 km/h più piano di quanto vorrei, non vale la pena sorpassare — mi conviene semplicemente rallentare leggermente e adattarmi (questo è codificato proprio nel termine $Cost_v$).

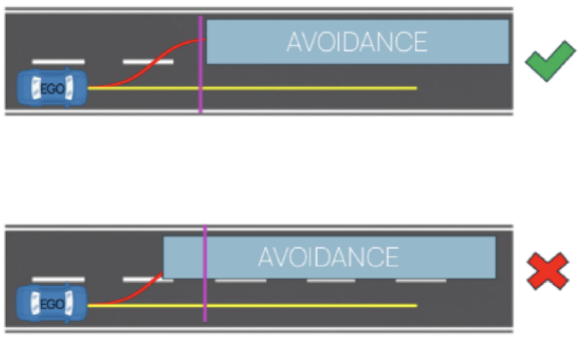
6.3.2.4 Vincoli di fattibilità (curvatura massima)

Prima di accettare una manovra di sorpasso o rientro, **si verifica che esista un valore del parametro a della sigmoide tale che la curvatura generata resti sempre entro il limite massimo fisico** $\rho_{max} = \hat{\rho} = \frac{||\hat{r}'(s) \times \hat{r}''(s)||}{||r'(s)||^3}$ (che dipende da accelerazione laterale disponibile e velocità attuale).



6.3.2.5 Preview distance

La sigmoide, essendo un'esponenziale, non "finisce" mai matematicamente — si avvicina asintoticamente al valore finale. Per verificare la collisione durante il cambio corsia, si definisce quindi una **distanza di preview, cioè la distanza necessaria alla sigmoide per raggiungere una percentuale (es. 95%) del suo valore finale**, e si controlla la collisione solo entro quella distanza. Non si guarda oltre perché, se la differenza di velocità con il veicolo davanti a me diventasse critica in un futuro più lontano, ci penserebbe comunque l'ACC a intervenire nel frattempo.



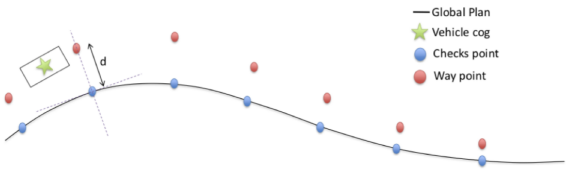
6.3.3 Approccio basato su ottimizzazione: ricerca su grafo *

Questo secondo approccio abbandona la forma chiusa della sigmoide e affronta scenari più generali (ostacoli come poligoni convessi arbitrari, non solo "veicolo lento davanti"). Il metodo, in generale, richiede sempre di rispondere a tre domande:

1. Come parametrizzo la soluzione (forma matematica del path)?
2. Cosa ottimizzo (funzione di costo)?
3. Come risolvo il problema di ottimizzazione (approccio iterativo)?

6.3.3.1 Parametrizzazione: waypoint

Si definiscono dei **checkpoint** campionati lungo il global plan, e per ognuno un insieme di possibili **waypoint** posizionati lungo la normale al checkpoint, a distanza d da esso. Il path è quindi descritto da una sequenza finita di distanze d — un insieme finito di numeri, non una funzione continua. Essendo un planner locale che dipende dall'output (limitato) della percezione, si lavora con un **receding horizon**: si campiona solo fino a una certa distanza massima.



Check points: sampled points on the global plan

Way points: points that define the local plan. A way point is located on the normal to the trajectory passing through the corresponding check point and at a **distance d** from the same check point.

Gli **ostacoli** sono rappresentati come poligoni convessi (il più piccolo poligono convesso che contiene l'ostacolo, con vertici in coordinate globali, calcolato dalla percezione).

6.3.3.2 Algoritmo

L'algoritmo si articola in tre fasi: preprocessing, optimization, post-processing.

Preprocessing.

Il problema principale che si vuole evitare è la discontinuità tra soluzioni successive: se a ogni istante ripianifico completamente da zero dalla posizione attuale del veicolo, il rischio è che la nuova soluzione sia molto diversa da quella calcolata all'istante precedente, generando un percorso irrealizzabile in termini di curvatura (salti bruschi). La soluzione è definire una testa del piano (**head of the plan**): invece di ripartire l'ottimizzazione dalla posizione istantanea del veicolo, si mantiene fissa una porzione iniziale del vecchio piano (già in fase di esecuzione/controllo) e si ottimizza solo a partire da un punto più avanti. Il ragionamento è che il controllore ha bisogno di un look-ahead stabile: se questo orizzonte di guardia continua a cambiare istante per istante, diventa inutile per il controllore stesso.

Optimization

We can first build the entire graph and then work on optimization or directly build the graph while optimizing.

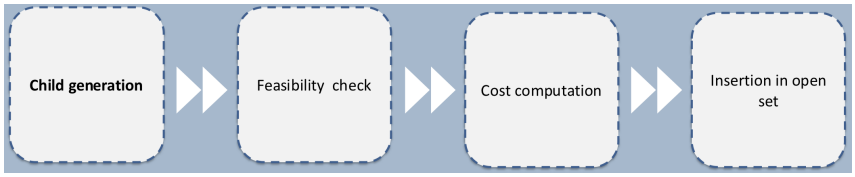
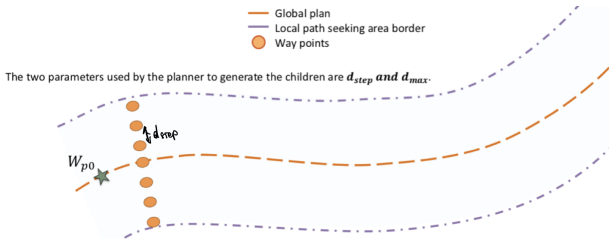


Figure 6.7: Optimization's steps

1. Child generation



A partire dal primo waypoint da ottimizzare, si genera un albero di nodi "figli": per ogni waypoint padre si generano più waypoint figli, spaziati di un passo d_{step} lungo la normale al checkpoint, entro una semi-larghezza massima d_{max}. Questi due parametri controllano la **granularità e l'ampiezza dello spazio di ricerca** — e il numero di nodi cresce molto rapidamente se si aumentano entrambi.

2. Feasibility check

Ogni nodo figlio generato viene scartato se viola almeno uno di due vincoli:

- **curvatura**: la curvatura $\frac{1}{\rho}$ implicata dalla terna padre-nodo-figlio successivo deve restare entro ρ_{max};
- **distanza dagli ostacoli**: si trova il punto più vicino del poligono dell'ostacolo al nodo, e se la distanza è inferiore a una soglia minima (MIN_{dist}, scelta tenendo conto dell'ingombro reale del veicolo più un margine di sicurezza) il nodo è scartato.

Per evitare che, essendo un approccio campionato, il segmento tra due nodi "tagli attraverso" un ostacolo pur avendo entrambi gli estremi a distanza sufficiente, **si discretizza anche il segmento padre-figlio e si ripete il controllo su punti intermedi**.

3. Cost computation

Il costo di ciascun nodo figlio somma tre contributi concettuali:

1. **distanza dal checkpoint** del global plan.
2. **coerenza** con il vecchio piano
3. una **penalità sul cambio brusco di orientamento** (heading) tra nodo padre e nodo figlio rispetto all'orientamento del global plan — in pratica per scoraggiare cambi di direzione innaturali.

Formalmente:

$$cost_{child_i} = W_d \cdot d_i^2 + W_\psi \left((\psi_{i,local} - \psi_{i-1,local}) - (\psi_{i,global} - \psi_{i-1,global})\right)^2$$

W_d indica il peso sullo scostamento: con W_d alto l'ottimizzatore è scoraggiato ad allontanarsi dal global plan, ogni metro di deviazione costa caro. Se invece W_d è basso, l'ottimizzatore si puo permettere di allontanarsi dal global plan e spostarsi lateralmente molto presto.
Analogamente il peso W_ψ regola quanto anticipare/ritardare la manovra sulla base della penalità di heading (orientamento).

4. Insertion in open set

Ogni nodo fattibile e con costo calcolato viene **inserito in una coda ordinata per costo crescente (open set)**. Il planner esamina i nodi partendo dal costo minimo; appena il primo nodo esaminato è un nodo obiettivo (goal, nel senso che appartiene all'ultima riga, alla fine del receding horizon), la ricerca si ferma e il piano si ricostruisce risalendo dai nodi padre fino al primo waypoint, altrimenti genera i figli, li valuta e li aggiunge all'open set.
Raffinamento in due iterazioni. Poiché aumentare sia d_{max} sia diminuire d_{step} fa crescere lo spazio di ricerca molto rapidamente, si preferisce una strategia a due passate:

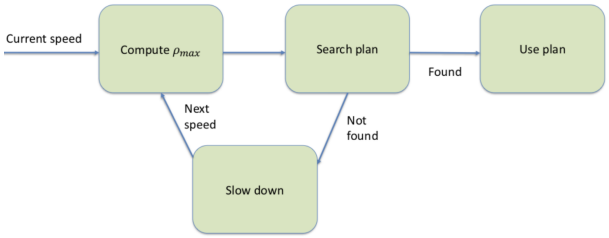
1. prima iterazione con d_{max} e d_{step} grandi trova rapidamente una soluzione "grezza" (con qualche cambio di direzione non ottimale);
2. una seconda iterazione usa quella soluzione come **nuova linea centrale** e restringe l'area di ricerca (raffinando d_{step} e riducendo d_{max}), risolvendo un problema più piccolo attorno alla soluzione già trovata.

I dati sperimentali del corso mostrano che questa strategia a due passate riduce sensibilmente i tempi medi di calcolo rispetto a una singola iterazione con passo fine su tutta

l'area.

Post-processing

Si concatenano i nodi della soluzione ottima, si interpola il risultato per ottenere punti equispaziati (utile per il tipo di controllore usato), e **si prepara la traiettoria finale** .



Se **non** si trova un piano che **eviti l'ostacolo** rispettando i vincoli di curvatura alla velocità corrente, ci sono due possibilità:

- l'ostacolo è realmente inevitabile (occupa tutta la carreggiata)
- si sta semplicemente andando troppo veloce per la manovra necessaria.

In quest'ultimo caso si rallenta e si ricalcola ρ_{max} alla nuova velocità (più bassa è la velocità, più curvatura è disponibile), tentando di nuovo la ricerca del piano.

6.3.4 Estensione a ostacoli dinamici

Fin qui si è ragionato solo nel dominio dello spazio. Con ostacoli in movimento questo non basta più: **l'ostacolo occupa posizioni diverse in istanti diversi**, quindi occorre aggiungere la dimensione temporale. L'idea è restare nel dominio spaziale per la costruzione dell'albero (stesso algoritmo di prima), ma **etichettare ogni "riga" di nodi generata** (cioè ogni checkpoint lungo s) **con un tempo stimato** t_i, calcolato assumendo che l'ego vehicle proceda a velocità costante: $\Delta t_i = \frac{\Delta s_i}{v_i}$. Questo disaccoppiamento (velocità nota e fissata, non parte delle variabili decisionali) è ciò che rende il problema trattabile: se si dovesse pianificare simultaneamente velocità e posizione laterale, l'associazione spazio-tempo dipenderebbe essa stessa dalle variabili di decisione, complicando enormemente il problema.

Per la predizione della posizione futura dell'ostacolo si usa un **modello a curvatura coordinata (coordinated turn model)**, assumendo:

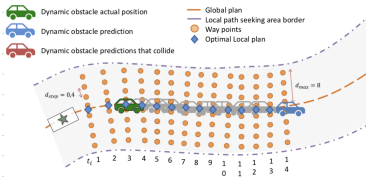
- velocità lineare costante

- velocità angolare costante

$$x_{i+1} = x_i + \Delta x_i, \quad \Delta x_i = v^i_{x,obst}\Delta t_i, \quad v^i_{x,obst} = |v_{obst}| \cos \psi^i_{obst}$$

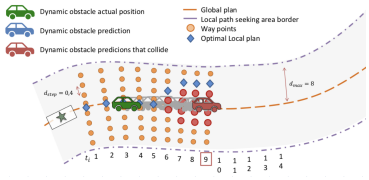
(analogo per y e ψ). Conoscendo tempo e velocità dell'ostacolo, per ogni riga temporale della griglia **si sa dove l'ostacolo si troverà**, e si può marcare come infattibile solo la porzione di spazio effettivamente occupata in quell'istante — non l'intera "scia" statica dell'ostacolo. Abbiamo diversi scenari:

6.3.4.1 Same direction, same velocity



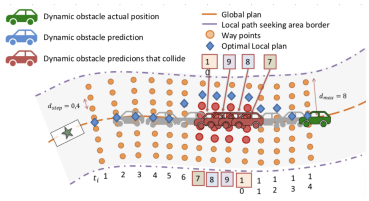
$v_{obst} = v_{ego}$: la distanza resta costante. Poiché l'associazione spazio-tempo tiene conto del moto reale, **il planner "non vede mai" l'ostacolo come qualcosa da evitare** — non raggiungerò mai la sua posizione. Un planner statico, al contrario, vedrebbe l'ostacolo occupare quello spazio per sempre e pianificherebbe (inutilmente) di evitarlo o frenerebbe senza motivo.

6.3.4.2 Same direction, different velocity



ostacolo più lento $v_{obst} < v_{ego}$: la distanza si riduce nel tempo, e **prima o poi ego raggiunge l'ostacolo**. La regione da evitare che emerge dal grafo è più grande della semplice sagoma del veicolo — tanto più grande quanto più piccola è la velocità relativa.

6.3.4.3 same direction, opposite velocity



la regione di collisione emerge naturalmente dall'intersezione spazio-temporale.

Il costo dell'ottimizzazione resta identico a prima; cambia solo come si stabilisce dove si trova l'ostacolo (associazione tempo-spazio anziché posizione statica).

Un punto pratico aggiuntivo: bisogna mantenere memoria di un ostacolo anche quando esce dal campo visivo dei sensori durante una manovra (es. mentre lo si sta sorpassando) — la cosiddetta **object persistence**: un oggetto non può "sparire" improvvisamente

Evento	Planner statico	Planner dinamico
Ostacolo al centro corsia con velocità pari a quella ego	**Frena inutilmente** fino a fermarsi, perché considera l'ostacolo come occupante permanente dello spazio e non tiene conto del fatto che non ci sarà collisione futura.	Riconosce che non avverrà alcuna collisione e **continua a velocità costante**.
Ostacolo più lento e non sorpassabile	Il **planner statico si ferma prima perché sovrastima la vicinanza della collisione.**	Predice correttamente il punto di collisione futuro e **si ferma più lontano (circa 25 m), con una frenata meno brusca.**
Ostacolo sorpassabile a velocità relativa bassa	Riesce comunque a pianificare la manovra, ma con **margine molto ridotto**, perché non anticipa il moto dell'ostacolo. Con velocità relative maggiori rischierebbe di pianificare la manovra troppo tardi.	Anticipa il movimento dell'ostacolo e pianifica la manovra considerando la dinamica futura, garantendo un margine maggiore.

Table 6.2: Confronto tra planner statico e dinamico in diversi scenari di ostacolo.

solo perché temporaneamente non visibile. Se il planner non trova alcuna soluzione anche assumendo il rallentamento, il fallback finale è delegare all'ACC, che adatta semplicemente la velocità a quella del veicolo davanti (stessa struttura vista nel longitudinal planner).
Un'ultima considerazione strutturale: se ogni veicolo pianifica assumendo che gli altri mantengano un comportamento fisso e "faccia ognuno per sé", il sistema può facilmente finire in situazioni di stallo (deadlock) in cui nessuna soluzione fattibile viene trovata — un limite intrinseco di questo tipo di predizione semplificata.

6.3.5 Predizione del comportamento altrui

La qualità della predizione del comportamento degli altri utenti della strada è un **punto critico per la bontà del planning in scenari complessi**.

6.3.5.1 Scenari semplici

Parliamo di scenari semplici per esempio quando citiamo sorpasso in autostrada, immissione, attraversamento pedonale: **si assume che l'altro veicolo/pedone mantenga velocità e/o accelerazione costante**, e si "proietta" questo comportamento nel futuro.
Ipotesi classiche:

- velocità costante
- accelerazione costante

Soluzioni "mapless", basate su filtraggio e predizione a ciclo chiuso classici. Il limite evidente è che, in alcuni contesti, assumere un comportamento costante è chiaramente irrealistico (un'auto che sta per svoltare non mantiene affatto traiettoria e velocità costanti).

6.3.5.2 Scenari complessi

incroci, rotatorie: qui la mappa stessa limita il numero di comportamenti plausibili — non esistono infinite possibilità, ma un **numero finito di manovre coerenti con la topologia della strada**. Due approcci:

1. **Approccio classico (probabilistico)**: Si definiscono più modelli di comportamento (es. uno per "va dritto", uno per "svolta a destra"), ciascuno usato in parallelo per predire la posizione futura. **Ad ogni nuova misura si confronta la predizione di ciascun modello con la misura reale, correggendo il modello e — implicitamente — capendo quale dei modelli è più plausibile**: è uno schema a filtro di Kalman multi-modello. Ogni modello di moto può essere più o meno informato: si va da un modello "poco informativo" (velocità nulla, yaw rate nullo — il caso in cui non si sa nulla) a un modello a curvatura coordinata dove il yaw rate dipende dalla curvatura della manovra prevista.
   Il limite di questo approccio è che scala male: funziona bene per predire un solo veicolo, ma diventa molto più complesso quando bisogna modellare le interazioni fra più utenti contemporaneamente.
2. **Approccio Machine Learning**: L'idea è **addestrare una rete su grandi quantità di dati** per predire direttamente il comportamento. Il vantaggio è che gestisce meglio la complessità delle interazioni multi-agente; gli svantaggi dichiarati sono il bisogno di grandi quantità di dati e questioni aperte di adattabilità a scenari non visti in training.
   Una volta che si dispone di una predizione corretta, la fase di pianificazione vera e propria è gestibile bene con i metodi già visti — la vera difficoltà sta nell'essere robusti agli errori di stima e predizione.

7 | Global Planning *

7.1 Dal global planning "stradale" al global planning "da corsa"

Nella lezione precedente il global planning era stato definito come il percorso di riferimento per andare dalla posizione attuale a un obiettivo, basato su **conoscenza a priori dell'ambiente** (informazioni statiche: mappa stradale, lunghezza, consumo, traffico). Per veicoli passeggeri, l'ambiente è la rete stradale e gli obiettivi tipici sono minimizzare lunghezza, durata, pedaggi, energia, traffico — sostanzialmente un problema di routing. Ora cambiamo il contesto: si considera il global planning dal **punto di vista di una vettura da corsa**. Cambiano ambiente e obiettivo: l'ambiente è il tracciato (racetrack, con i suoi confini interno/esterno), e l'obiettivo tipico e dominante è minimizzare il tempo sul giro (lap time) — un problema molto più specifico e circoscritto rispetto al routing generico.

NB: pianificare traiettorie da corsa non è utile solo in pista — **la stessa logica si applica su strada pubblica in manovre di emergenza orientate alla sicurezza** (safety-oriented manoeuvres), dove serve sfruttare al massimo l'aderenza del veicolo per evitare un pericolo. *Cosa serve per trovare il piano migliore in ambito racing?* Bisogna trovare la **migliore traiettoria basata su percorso e velocità insieme** — cioè non basta scegliere una bella linea geometrica, **bisogna anche sapere quanto veloce percorrerla, perché le due cose sono legate dai limiti fisici del veicolo**.

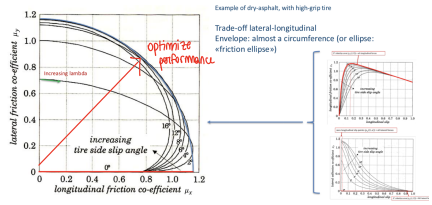
7.1.1 Il limite fisico: la friction ellipse

Il punto di partenza fisico è che **sono le forze degli pneumatici a far muovere l'auto sul tracciato, e queste forze sono limitate**. Si considerano due direzioni:

- longitudinale (accelerazione/frenata)
- laterale (sterzata/curva)

La quantità totale di forza disponibile è limitata, quello che si può fare è:

1. esercitare una forza minore della massima disponibile
2. decidere in quale direzione applicarla — ma non si può "avere di più" del totale disponibile.



Se si plottano insieme il coefficiente di attrito longitudinale e quello laterale, l'inviluppo di tutte le combinazioni fisicamente raggiungibili è, appunto, la friction ellipse.

1st requirement: Per un ambiente racing, dove l'obiettivo è le prestazioni pure, **si vuole idealmente restare sul bordo di questa ellisse in ogni istante** — cioè sfruttare sempre il massimo di aderenza disponibile, in qualunque direzione serva in quel momento.

2nd requirement: il modo in cui si "consuma" questa ellisse dipende da quale percorso si sta seguendo (più stretta la curva, più aderenza laterale serve), quindi bisogna anche capire **qual è il percorso migliore da seguire** — non solo quanto veloce andare su un percorso dato. Derivazione geometrica che lega raggio di curvatura, velocità e accelerazione laterale massima:

$$M\frac{v_{max}^2}{R} = F_z\mu_y^{max} \Rightarrow v_{max} = \sqrt{\frac{RF_z\mu_y^{max}}{M}} = \sqrt{Ra_y^{max}} = \sqrt{\frac{a_{yMax}}{\rho}}$$

dove R è il raggio di curvatura, μ_y^{max} il coefficiente di attrito laterale massimo.

In pratica: **la velocità massima in curva cresce con la radice del raggio di curvatura** — curve più larghe permettono velocità più alte a parità di aderenza. Insieme alle relazioni cinematiche $\dot{\psi} = v/R$ e $\rho = 1/R$ (curvatura come inverso del raggio), questo è il ponte concettuale tra "geometria del percorso" e "velocità massima sostenibile".

7.1.2 Due filosofie per progettare la traiettoria

Vi sono due approcci alternativi per il design della traiettoria da corsa, con un compromesso speculare a quello già visto per il lateral planning (closed-form vs optimization-based).

7.1.2.1 Metodo dinamico (Dynamic method)

Si risolve direttamente un **problema di controllo ottimo, sfruttando esplicitamente un modello dinamico del veicolo**. È un metodo computazionalmente più complesso, ma con un vantaggio importante: ottimizza direttamente il tempo sul giro (non una proxy), ed è **più robusto perché tiene conto di tutti i parametri reali del veicolo**. Lo svantaggio, però, è proprio questo: essendo basato su un modello dettagliato del veicolo, **la soluzione trovata è specifica per quel veicolo** — cambiando auto, va ripetuto tutto.

7.1.2.2 Metodo geometrico (Geometric method)

Fornisce una soluzione più generale: formulazione più semplice, **indipendente dai parametri del veicolo**, e disaccoppia il problema laterale (che percorso seguire) da quello longitudinale (quanto veloce), ottimizzandoli in due passi consecutivi. Il vantaggio è la semplicità e la generalità; lo svantaggio è che si perde un po' di robustezza, proprio perché non si tiene conto fin da subito di tutti i dettagli dinamici del veicolo specifico.

Si vuole minimizzare il tempo sul giro:

$$T_{lap} = \int_0^{s_{end}} \frac{1}{v(s)}, ds$$

dove s è l'ascissa curvilinea lungo il percorso e $v(s)$ il profilo di velocità. Guardando questa espressione, per minimizzare T_{lap} serve minimizzare due cose contemporaneamente:

1. velocità inversa (cioè massimizzare v)
2. la lunghezza del percorso su cui si integra (un percorso più corto significa un intervallo di integrazione più piccolo).

Ma la velocità stessa è vincolata dalla curvatura del percorso:

$$v(s) \leq v_{max}(s) = \sqrt{\frac{a^y_{max}}{|\rho(s)|}}$$

Massimizzare la velocità richiede minimizzare la curvatura (curve più dolci = più veloci), ma minimizzare la lunghezza del percorso tipicamente richiede tagliare le curve in modo più stretto (più curvatura). I due obiettivi sono in contrasto tra loro.

Come risolvere questo compromesso? Divide et impera. Invece di ottimizzare simultaneamente percorso e velocità (un problema con due tipi di variabili intrecciate), si applica

un **approccio di rilassamento**: si spezza il problema in due sotto-problemi risolti in sequenza.

- **Step A**— calcolare prima il percorso, senza preoccuparsi affatto della velocità (solo geometria: lunghezza + curvatura).
- **Step B** — dato quel percorso, guidarlo alla massima velocità possibile (qui il percorso è ormai fissato/dato, e si applica lo stesso schema di ottimizzazione della velocità già visto per il longitudinal planner).

Alla fine si può calcolare il tempo sul giro risultante per confrontare soluzioni diverse e scegliere la migliore. È un compromesso: non è garantito essere l'ottimo globale assoluto (che richiederebbe ottimizzare le due cose insieme), ma è molto più trattabile computazionalmente.

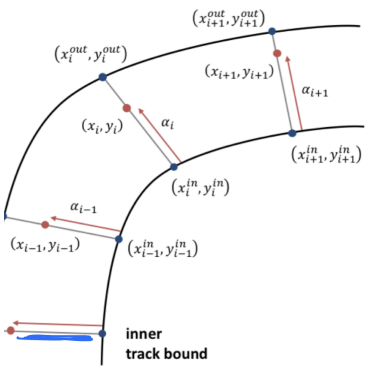
Step A — Ottimizzazione del percorso (racing line)

Il primo passo si scompone a sua volta in due domande:

1. *come parametrizzo il percorso?*
2. *come scelgo il migliore?*

Parametrizzazione

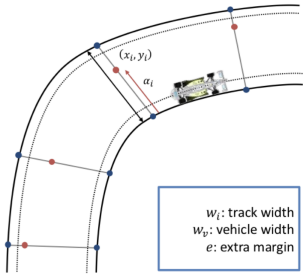
Si assume di conoscere il tracciato tramite le coordinate dei suoi confini interno ed esterno, (x_i^{in}, y_i^{in}) e (x_i^{out}, y_i^{out}), per $i \in [0, N]$.



Prima idea (scartata): coordinate cartesiane dirette. Descrivere ogni punto del percorso con la sua (x, y) diretta sembra naturale, ma ha due problemi seri:

- **Verificare i vincoli è scomodo**: per controllare che un punto candidato sia dentro i confini del tracciato, bisognerebbe verificare che sia "sopra" la curva interna e "sotto" quella esterna — geometricamente non banale da esprimere come vincolo semplice in un problema di ottimizzazione.
- **Dimensionalità doppia**: con n campioni lungo il percorso servono $2n$ variabili (una x e una y per ciascuno) — il problema di ottimizzazione ha il doppio delle incognite necessarie.

Soluzione: **parametrizzazione tramite** α. Esattamente come nel local planning si erano usati checkpoint e offset laterali, qui si riusa la stessa idea adattiva: **per ogni checkpoint** i**, si definisce il punto della racing line come combinazione lineare tra il confine interno e quello esterno**:



$$x_i = x_i^{in} + \alpha_i(x_i^{out} - x_i^{in}) \qquad y_i = y_i^{in} + \alpha_i(y_i^{out} - y_i^{in})$$

dove α_i è una coordinata laterale normalizzata: un unico numero per checkpoint invece di una coppia (x, y). Questa scelta porta due vantaggi diretti, che risolvono esattamente i due problemi sopra:

- **Dimensionalità dimezzata**: con n campioni servono solo n variabili (α_i), non $2n$.
- **Vincoli banali**: se $\alpha_i \in [0, 1]$, il punto è automaticamente all'interno dei confini del tracciato — non serve più nessun controllo geometrico complicato, basta un vincolo di intervallo (box constraint) sulla singola variabile scalare.

Il vincolo reale non è esattamente $\alpha_i \in [0, 1]$, ma leggermente più stretto:

$$\alpha_i \in [\Delta\alpha_i, 1 - \Delta\alpha_i] \qquad \Delta\alpha_i = \frac{w_v/2 + e}{w_i}$$

dove w_i è la larghezza del tracciato in quel punto, w_v la larghezza del veicolo, e un margine extra di sicurezza.

Il perché di questo margine è spiegato con tre osservazioni pratiche:

1. la racing line rappresenta il baricentro del veicolo (COG), non i suoi bordi — quindi se il COG è esattamente sul confine, metà macchina è già fuori pista;
2. il controllore che insegue la traiettoria non garantisce mai un errore di inseguimento nullo (ci sarà sempre un po' di scarto reale);
3. la larghezza del tracciato può variare punto per punto. Sottraendo mezza larghezza veicolo più un margine extra da entrambi i lati, si garantisce che l'intera vettura, anche con un po' di errore di tracking, resti effettivamente dentro i bordi.

La funzione di costo deve penalizzare **entrambi** gli obiettivi in conflitto — lunghezza e curvatura — combinandoli con due pesi:

$$J(\alpha_0, \ldots, \alpha_N) = k_l, J_l(\alpha_0, \ldots, \alpha_N) + k_\rho, J_\rho(\alpha_0, \ldots, \alpha_N)$$

Curvatura. Definiamo la **curvatura** come la somma (integrale discreto) del quadrato della curvatura in ogni segmento:

$$J_\rho(\alpha_0, \ldots, \alpha_N) = \sum_{i=1}^{N-1} (\rho_i)^2$$

Il quadrato evita di dover gestire il valore assoluto (la curvatura può essere positiva o negativa a seconda del verso della curva, ma quello che conta ai fini del costo è la sua entità, non il segno), e nei problemi di ottimizzazione è preferibile evitare funzioni con valore assoluto perché non sono derivabili ovunque (hanno un punto angoloso a zero), mentre il quadrato è liscio e ben comportato per i metodi di ottimizzazione numerica.

La curvatura in ogni punto i è calcolata per differenziazione numerica a partire da tre punti consecutivi (il punto dipende dalle variabili di ottimizzazione stesse, cioè dagli α):

$$\Delta\theta_i = \arctan\left(\frac{y_{i+1} - y_i}{x_{i+1} - x_i}\right) - \arctan\left(\frac{y_i - y_{i-1}}{x_i - x_{i-1}}\right) \qquad \rho_i = 2\frac{\Delta\theta_i}{\Delta s_i}$$

cioè: **quanto cambia l'orientamento del segmento tra un punto e il successivo, diviso per la distanza percorsa** — la definizione discreta classica di curvatura.

Lunghezza. J_l è semplicemente la somma delle distanze euclidee tra punti consecutivi del percorso (lunghezza totale approssimata).

- se $k_\rho = 0$, il problema si riduce a trovare il percorso di **lunghezza minima** (ignorando completamente la curvatura);
- se $k_l = 0$, si trova il percorso di **curvatura minima** (ignorando completamente la lunghezza, tipicamente un percorso più largo/dolce).

È il rapporto k_l/k_ρ che permette di scegliere un compromesso intermedio tra questi due estremi.

Nei rettilinei la racing line ottimale resta vicina al bordo esterno; in curva si sposta verso l'interno. Nota sulla struttura del problema: poiché il tracciato è un circuito chiuso, un vincolo aggiuntivo (menzionato esplicitamente) è che il primo e l'ultimo punto della soluzione devono coincidere — altrimenti la traiettoria non si "richiude" correttamente su un giro completo.

Step B — Ottimizzazione della velocità sul percorso dato

Una volta risolto lo Step A (il percorso è ormai fissato), il secondo step chiede: *qual è la massima velocità sostenibile su questo percorso specifico, rispettando i limiti fisici del veicolo?* É concettualmente lo stesso algoritmo a tre passate (trapezoidale) visto in SubSec 6.2.2.1.

Input: il percorso pianificato (ascissa curvilinea s e profilo di curvatura $\rho(s)$) e la velocità iniziale v_0. **Output**: il profilo di velocità di riferimento $v(s)$. L'algoritmo, identico nella struttura a quello già visto, si articola nelle stesse tre passate:

1. **Initial guess**: Si massimizza la velocità considerando solo il vincolo di accelerazione laterale (limite superiore, saturazione completa dell'ellisse in direzione laterale):

$$v_i^{(1)} = \sqrt{\frac{a_{max}^y}{|\rho_i|}}$$

   Concettualmente: in ogni punto, *quanto veloce potrei andare se potessi istantaneamente avere la velocità giusta per quella curvatura, ignorando il fatto che serve tempo/spazio per accelerare o frenare fino a lì.* Un dettaglio specifico al contesto racing: la velocità apice (speed limit while cornering, al punto di massima curvatura) si ottiene assumendo accelerazione/decelerazione longitudinale nulla in quel punto — cioè al vertice della curva tutta l'aderenza disponibile è dedicata alla componente laterale:

$$v_{max} = \sqrt{a_{max}^y/|\rho|}$$

   mentre altrove lungo la curva (non all'apice), assumendo $v \leq v_{max}$, resta un margine di accelerazione longitudinale residua disponibile, calcolabile a partire dal vincolo dell'ellisse:

$$a^x = \pm a_{max}^x \sqrt{1 - \left(\frac{a^y}{a_{max}^y}\right)^2}$$

2. **Forward pass**. Si integra in avanti nello spazio, considerando i limiti di accelerazione longitudinale e l'accoppiamento tra forze longitudinali e laterali (esattamente le stesse equazioni di moto uniformemente accelerato viste nella lezione precedente, in dominio spaziale):

$$a_{i-1}^y = (v_{i-1}^{(2)})^2 \cdot \rho_{i-1} \qquad a_{i-1}^x = a_{max}^x \sqrt{1 - \left(\frac{a_{i-1}^y}{a_{max}^y}\right)^2}$$

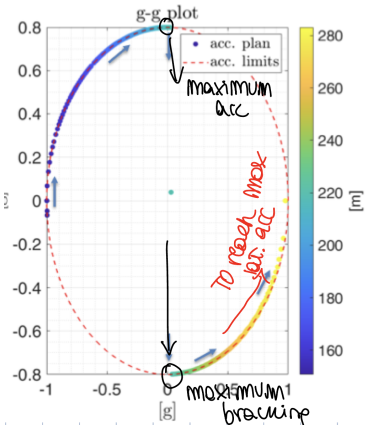
$$v_{i,guess}^{(2)} = \sqrt{(v_{i-1}^{(2)})^2 + 2a_{i-1}^x \Delta s_{i-1}} \qquad v_i^{(2)} = \min\left(v_i^{(1)}, v_{i,guess}^{(2)}\right)$$

3. **Backward pass.** Simmetricamente all'indietro, con i limiti di decelerazione longitudinale:

$$v_i^{(3)} = \min\left(v_i^{(2)}, v_{i,guess}^{(3)}\right)$$

Un dettaglio tecnico interessante: perché il risultato sia sia ottimo che fattibile, occorre che la velocità iniziale coincida con quella finale ($v_0^{(1)} = v_0^{(3)}$, requisito naturale su un circuito chiuso). Questo si garantisce inizializzando l'algoritmo nel punto di massima curvatura di tutto il tracciato ($i_0 = \text{argmax}|\rho_i|$), perché è lì che la velocità ammissibile è per costruzione la più bassa di tutto il giro — partire da quel punto garantisce che il resto dell'algoritmo non debba mai "sforare" quel limite minimo.

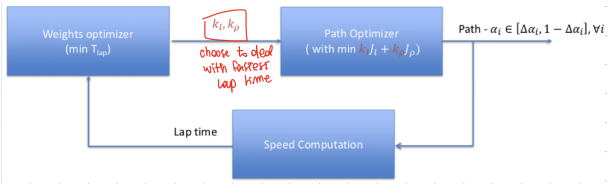
Il modo standard per visualizzare se il pilota (o il planner) sta effettivamente sfruttando tutta l'aderenza disponibile è plottando le coppie (a^x, a^y) effettivamente pianificate sopra l'ellisse limite. **Punti vicini al bordo dell'ellisse indicano che si sta sfruttando il massimo di aderenza in quella direzione** (massima accelerazione o massima frenata, a seconda della zona del grafico).



L'ellisse descrive il limite di aderenza pneumatico-strada, ma **in accelerazione c'è un ulteriore vincolo: la potenza del motore**. Il motore non può erogare una forza di accelerazione arbitrariamente alta anche se l'aderenza lo permetterebbe — quindi la parte "superiore" dell'ellisse (lato accelerazione) va troncata a un limite di potenza.

In frenata, invece, si può assumere che la potenza frenante sia sostanzialmente illimitata (il limite resta solo l'aderenza) — un'asimmetria fisica reale tra accelerare e frenare. Il metodo di calcolo resta comunque valido anche con l'ellisse troncata: si applica semplicemente lo stesso algoritmo con un vincolo aggiuntivo sulla parte superiore.

weight optimizer



A questo punto si hanno due componenti risolte separatamente:

- Path optimizer: dato il problema (i pesi k_l, k_ρ), lo risolve matematicamente restituendo la sequenza ottima di α_i;
- Speed computation: dato un percorso, calcola numericamente (con l'algoritmo a tre passate, non in forma chiusa) il profilo di velocità e quindi il tempo sul giro.

Manca un terzo livello: *come si scelgono i pesi k_l e k_ρ?* Qui entra in gioco il weight optimizer, un anello esterno che cerca la combinazione di pesi che minimizza il T_{lap} finale, usando il tempo sul giro calcolato dallo speed computation come segnale di retroazione per scegliere pesi via via migliori.

7.1.2.3 Hybrid approach

L'approccio più usato in pratica è ibrido: si parte con il metodo geometrico (semplice, generale) e poi, alla fine, si raffina con un passaggio dinamico — così da non dover partire subito con la necessità di conoscere tutti i parametri specifici del veicolo, ma ottenendo comunque una soluzione più accurata nella fase finale.